STA137FinalProject

Caitlyn Koyabu

2025-11-22

```
library(forecast)
```

```
## Warning: package 'forecast' was built under R version 4.3.3
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
load("/Users/caitlynkoyabu/Downloads/Data_File_finalproject.Rdata")
head(finalPro_data)
```

```
## # A tibble: 6 x 9
##   Country             Code   Year    GDP Growth   CPI Imports Exports Population
##   <fct>               <fct> <dbl>  <dbl>  <dbl> <dbl>   <dbl>   <dbl>      <dbl>
## 1 Central African Re~ CAF    1960 1.12e8 NA        NA    34.2    23.3    1503508
## 2 Central African Re~ CAF    1961 1.23e8  4.95     NA    35.8    26.5    1529227
## 3 Central African Re~ CAF    1962 1.24e8 -3.71     NA    37.7    24.6    1556661
## 4 Central African Re~ CAF    1963 1.29e8 -0.707    NA    38.5    25.2    1585763
## 5 Central African Re~ CAF    1964 1.42e8  2.08     NA    40.8    28.4    1616516
## 6 Central African Re~ CAF    1965 1.51e8  0.948    NA    37.7    27.1    1648833
```

```
summary(finalPro_data)
```

```
##                      Country        Code         Year           GDP          
##  Central African Republic:58   CAF    :58   Min.   :1960   Min.   :1.122e+08  
##  Afghanistan              : 0   ABW    : 0   1st Qu.:1974   1st Qu.:3.057e+08  
##  Albania                  : 0   AFG    : 0   Median :1988   Median :9.348e+08  
##  Algeria                  : 0   AGO    : 0   Mean   :1988   Mean   :9.394e+08  
##  American Samoa           : 0   ALB    : 0   3rd Qu.:2003   3rd Qu.:1.367e+09  
##  Andorra                  : 0   AND    : 0   Max.   :2017   Max.   :2.196e+09  
##  (Other)                  : 0   (Other): 0                                     
##      Growth               CPI            Imports         Exports      
##  Min.   :-36.69995   Min.   : 36.77   Min.   :18.00   Min.   :10.68  
##  1st Qu.: -0.00087   1st Qu.: 49.63   1st Qu.:25.03   1st Qu.:16.19  
##  Median :  2.05413   Median : 71.42   Median :29.68   Median :21.74  
##  Mean   :  1.19893   Mean   : 73.58   Mean   :30.83   Mean   :20.66  
##  3rd Qu.:  4.60753   3rd Qu.: 86.50   3rd Qu.:36.58   3rd Qu.:25.14  
##  Max.   :  9.48173   Max.   :186.86   Max.   :46.90   Max.   :34.31  
##  NA's   :1           NA's   :22
```

```
##    Population
##  Min.   :1503508
##  1st Qu.:1985984
##  Median :2842456
##  Mean   :2974790
##  3rd Qu.:3963152
##  Max.   :4659080
##
```

No missing data and no duplicate rows.

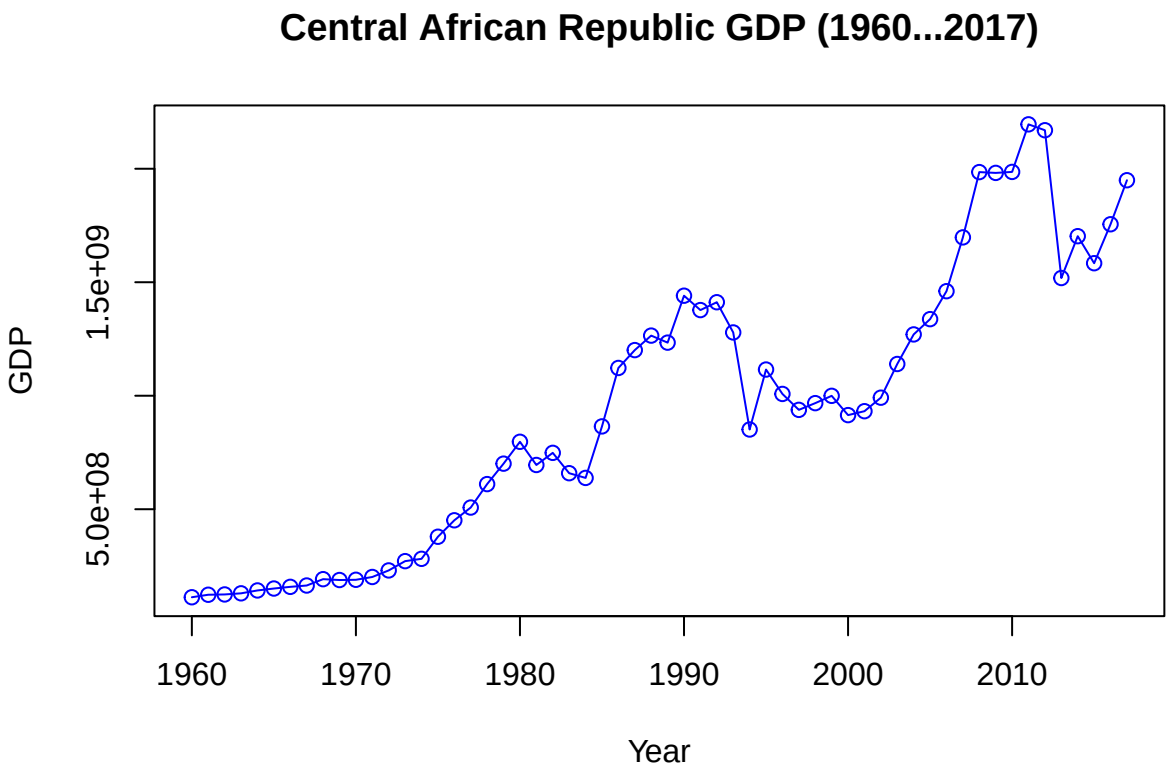
Plot the raw data

```
# Extract variables
gdp <- ts(finalPro_data$GDP, start = 1960, frequency = 1)
exports <- ts(finalPro_data$Exports, start = 1960, frequency = 1)
population <- ts(finalPro_data$Population, start = 1960, frequency = 1)

plot(gdp,
     type = "o",
     col = "blue",
     ylab = "GDP",
     xlab = "Year",
     main = "Central African Republic GDP (1960-2017)")
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Central African Republic GDP (1960-2017)' in 'mbcsToSbcs': dot
## substituted for <e2>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Central African Republic GDP (1960-2017)' in 'mbcsToSbcs': dot
## substituted for <80>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Central African Republic GDP (1960-2017)' in 'mbcsToSbcs': dot
## substituted for <93>
```

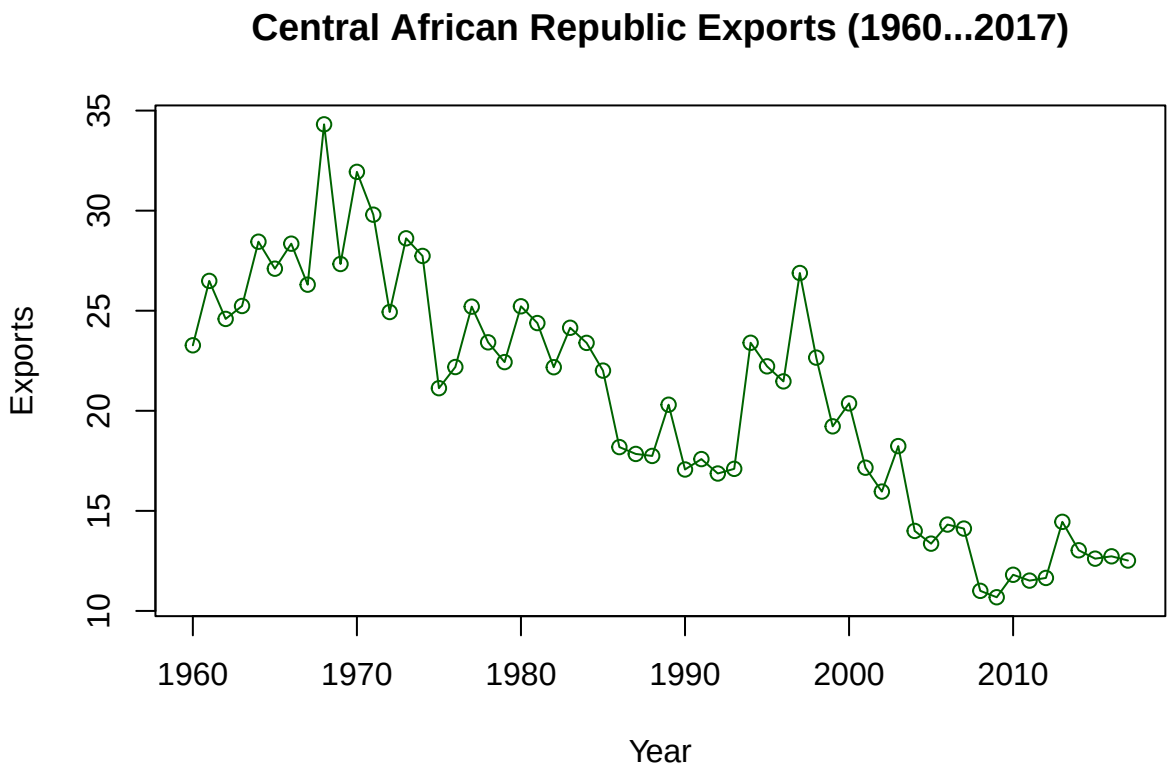



```
plot(exports,
     type = "o",
     col = "darkgreen",
     ylab = "Exports",
     xlab = "Year",
     main = "Central African Republic Exports (1960-2017)")
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Central African Republic Exports (1960-2017)' in 'mbcsToSbcs': dot
## substituted for <e2>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Central African Republic Exports (1960-2017)' in 'mbcsToSbcs': dot
## substituted for <80>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Central African Republic Exports (1960-2017)' in 'mbcsToSbcs': dot
## substituted for <93>
```

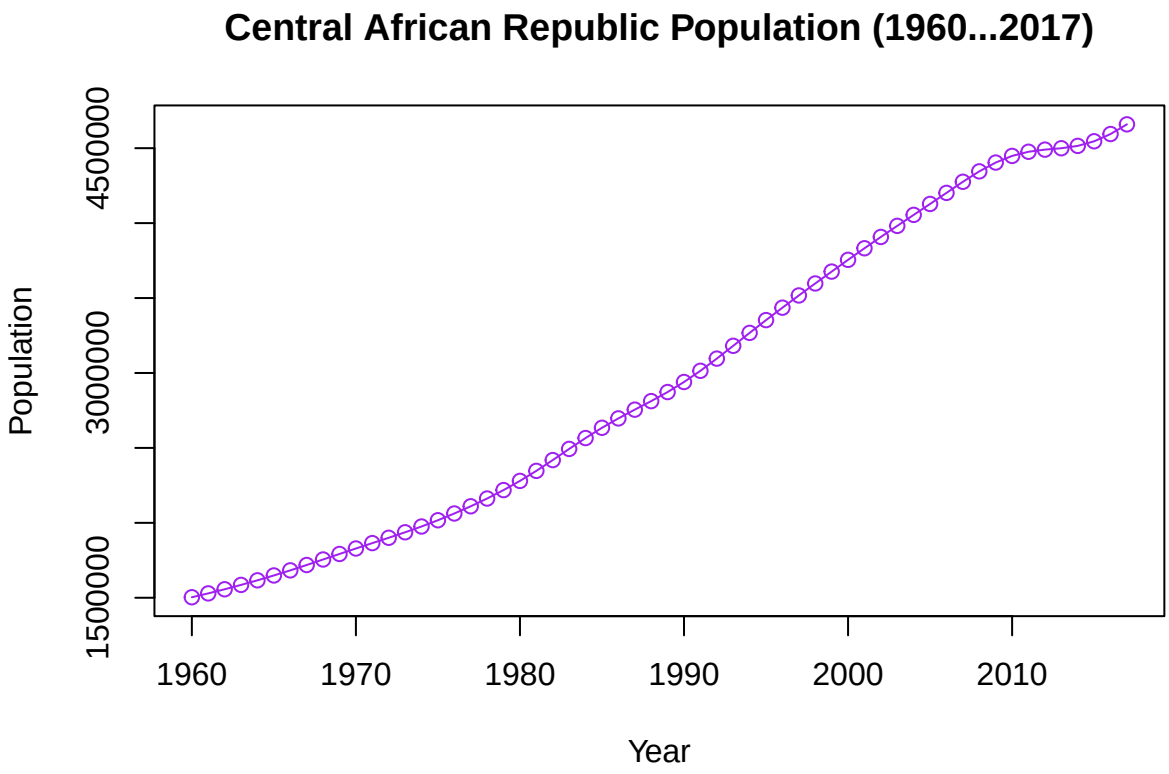



```
plot(population,
     type = "o",
     col = "purple",
     ylab = "Population",
     xlab = "Year",
     main = "Central African Republic Population (1960-2017)")
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Central African Republic Population (1960-2017)' in 'mbcsToSbcs':
## dot substituted for <e2>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Central African Republic Population (1960-2017)' in 'mbcsToSbcs':
## dot substituted for <80>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Central African Republic Population (1960-2017)' in 'mbcsToSbcs':
## dot substituted for <93>
```



Check Outliers - (Z-score and Boxplots)

```
gdp_z <- scale(gdp)
which(abs(gdp_z) > 3)
```

```
## integer(0)
```

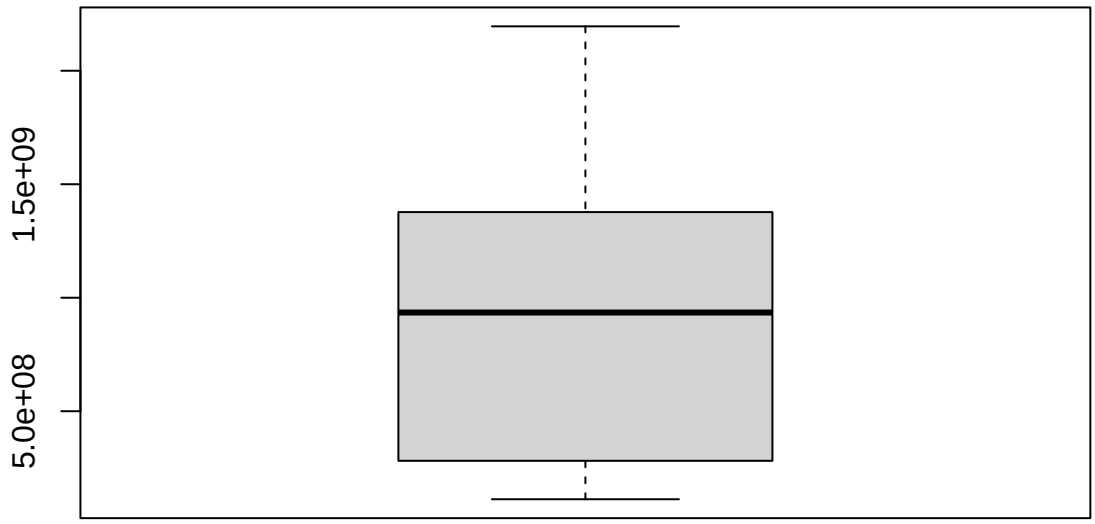
```
exports_z <- scale(exports)
which(abs(exports_z) > 3)
```

```
## integer(0)
```

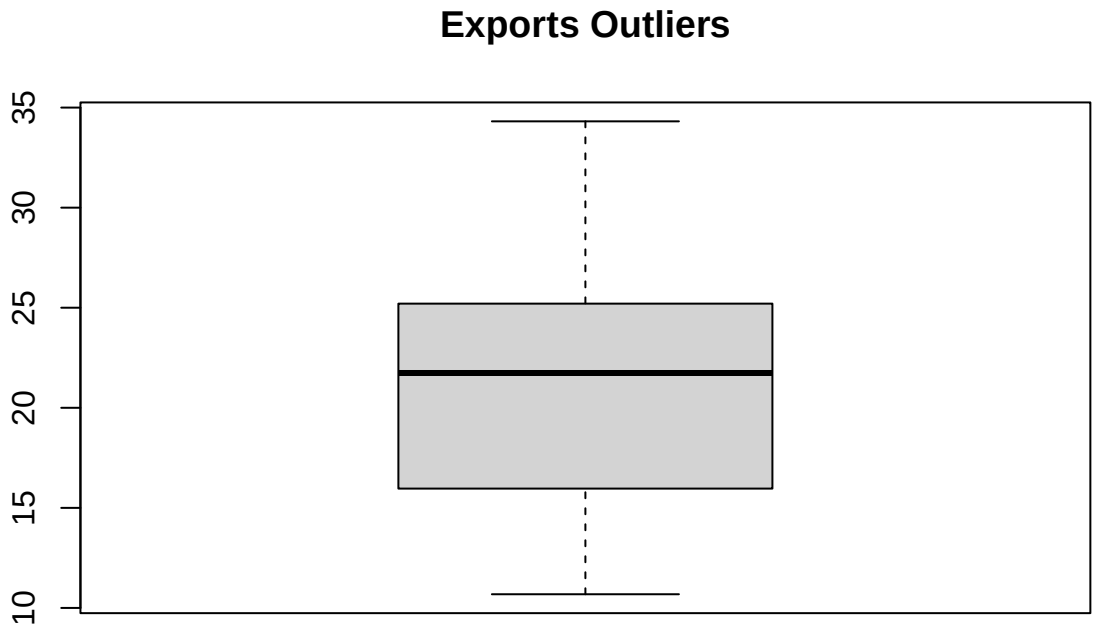
```
pop_z <- scale(population)
which(abs(pop_z) > 3)
```

```
## integer(0)
```

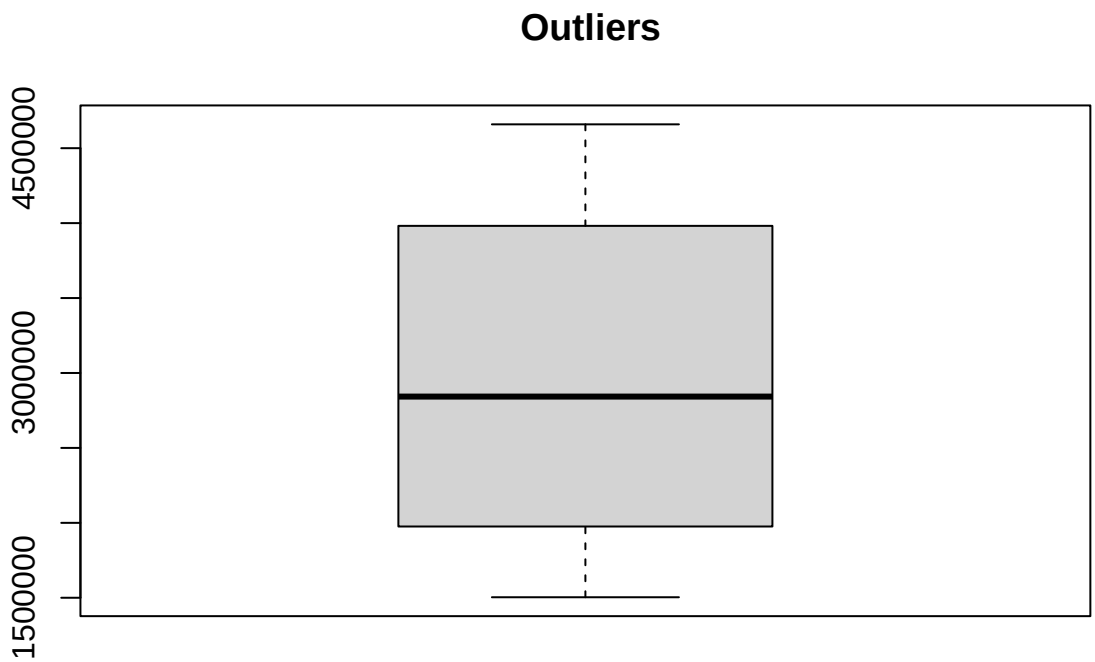
```
boxplot(gdp, main="GDP Outliers")
```




```
boxplot(exports, main="Exports Outliers")
```




```
boxplot(population, main=" Outliers")
```




```
tsoutliers(gdp)
```

```
## $index
## integer(0)
##
## $replacements
## numeric(0)
```

```
tsoutliers(exports)
```

```
## $index
## integer(0)
##
## $replacements
## numeric(0)
```

```
tsoutliers(population)
```

```
## $index
## integer(0)
##
## $replacements
## numeric(0)
```

GDP Analysis: The Central African Republic's GDP displays an upward trend from 1960 to 2017, with noticeable fluctuations during the 1980s to the mid 1990s and the 2010s to mid 2010s. There is no seasonality trend to be observed because the data we observe is annual. Overall, the trend of the data is relatively smooth with the occasional drops and spikes in the plot, but overall, the Central African Republic's GDP increases over time.

Export Analysis: The Central African Republic's Exports is more volatile than the GDP plot and doesn't show as smooth as a trend. There are several sharp spike and drops, indicating that there could be uncertainty and instability in trade activity. However, it still seems to be having a downward trend when plotting the data. Therefore, the Central African Republic's Exports decreases over time.

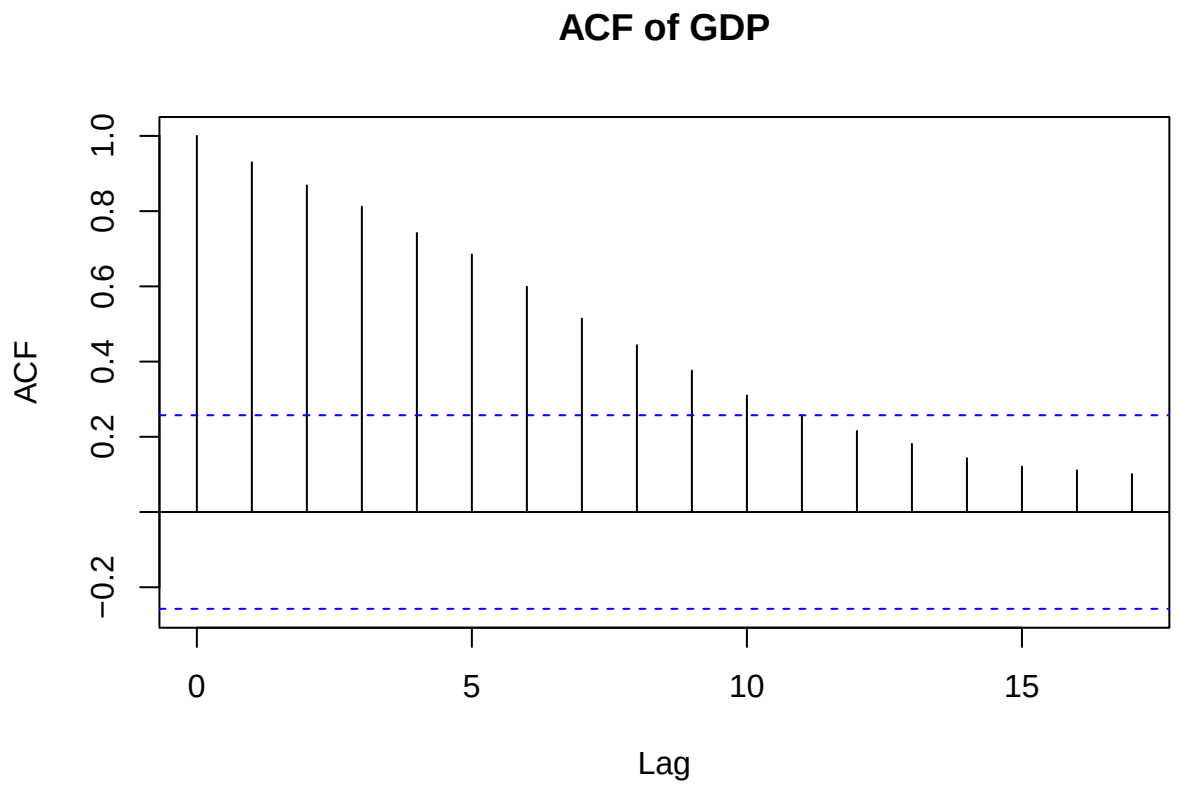
Population Analysis: The Central African Republic's Population displays a smooth consistent increasing trend. Over time, the Central African Republic's Population increases.

Since there are trends for The Central African Republic's GDP, Exports, and Population data, this means that we have non-stationary data. We want to get rid of the trends, so we can work with stationary data. Therefore, we will take first differences of the data until the data is stationary.

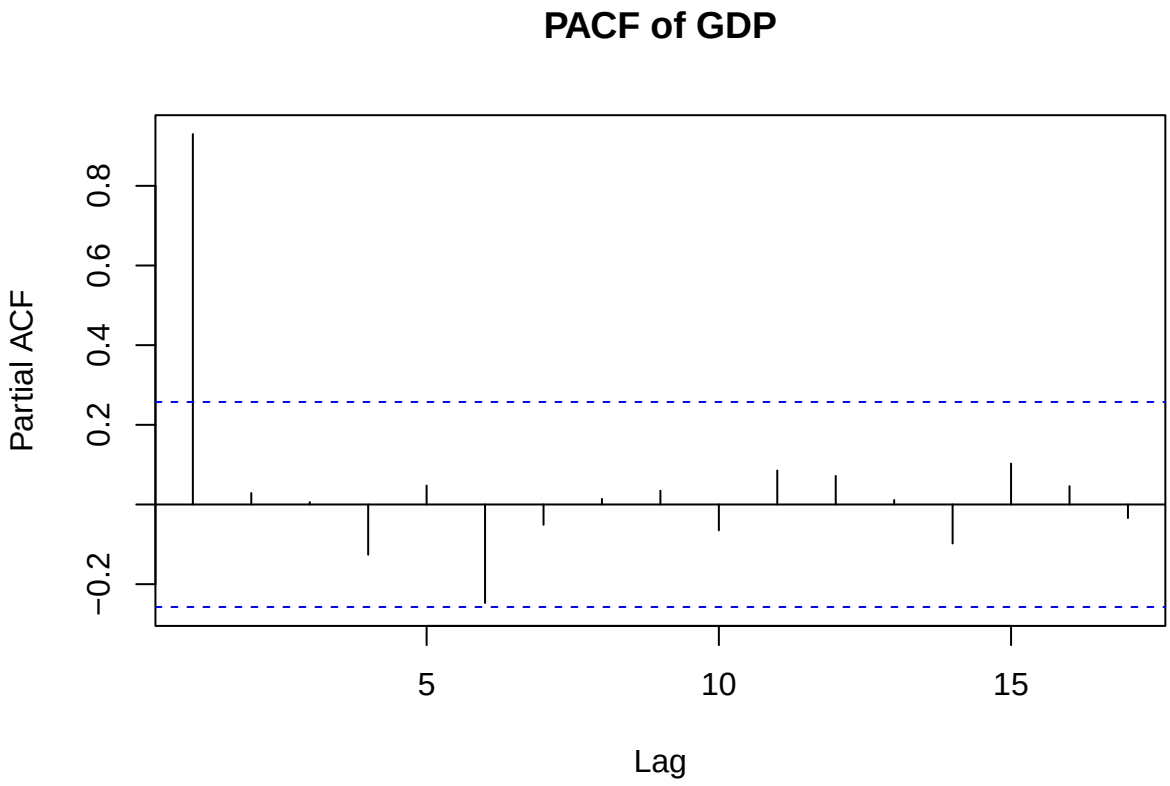
Checking for seasonlity in the plots, we can look repeating patterns at regular intervals (yearly cycles) and we can see that there's no regular pattern occuring as we observe each year from 1960s to 2017s. Therefore, we can conclude that there is no seasonality for GDP, exports, and population plots.

ACF/PACF - Raw

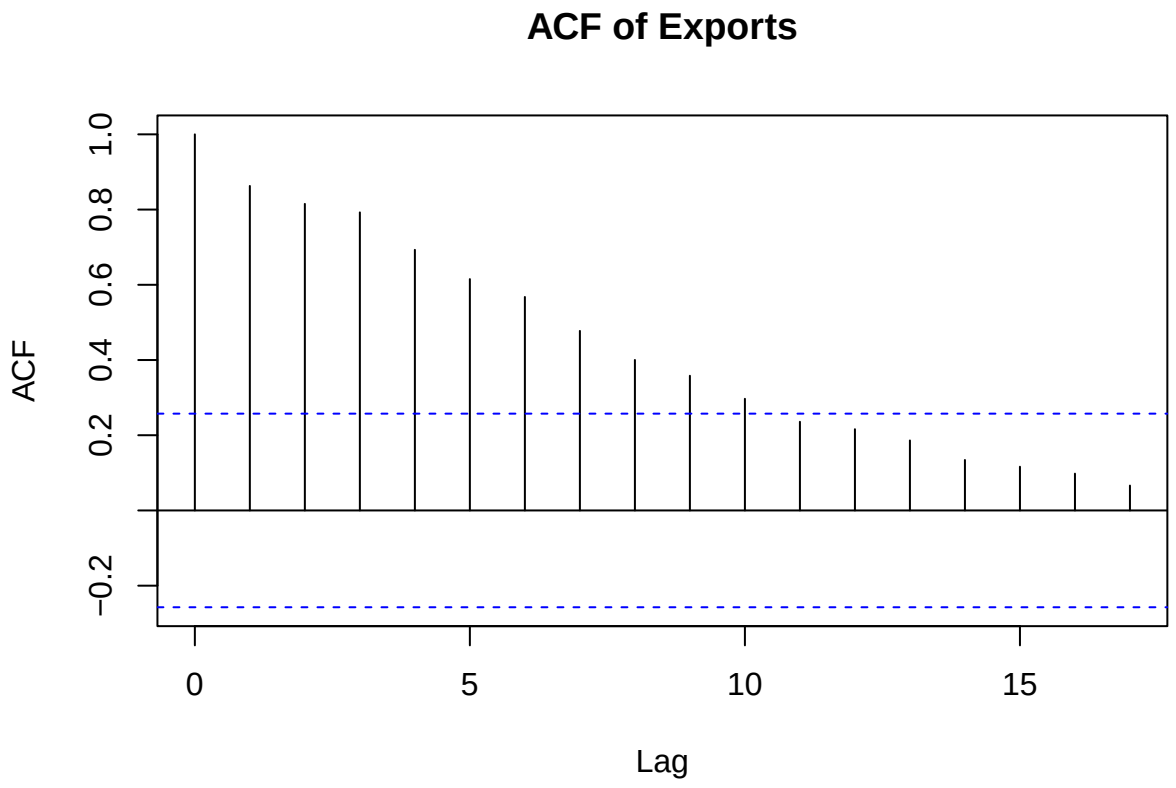
```
acf(gdp, main = "ACF of GDP")
```



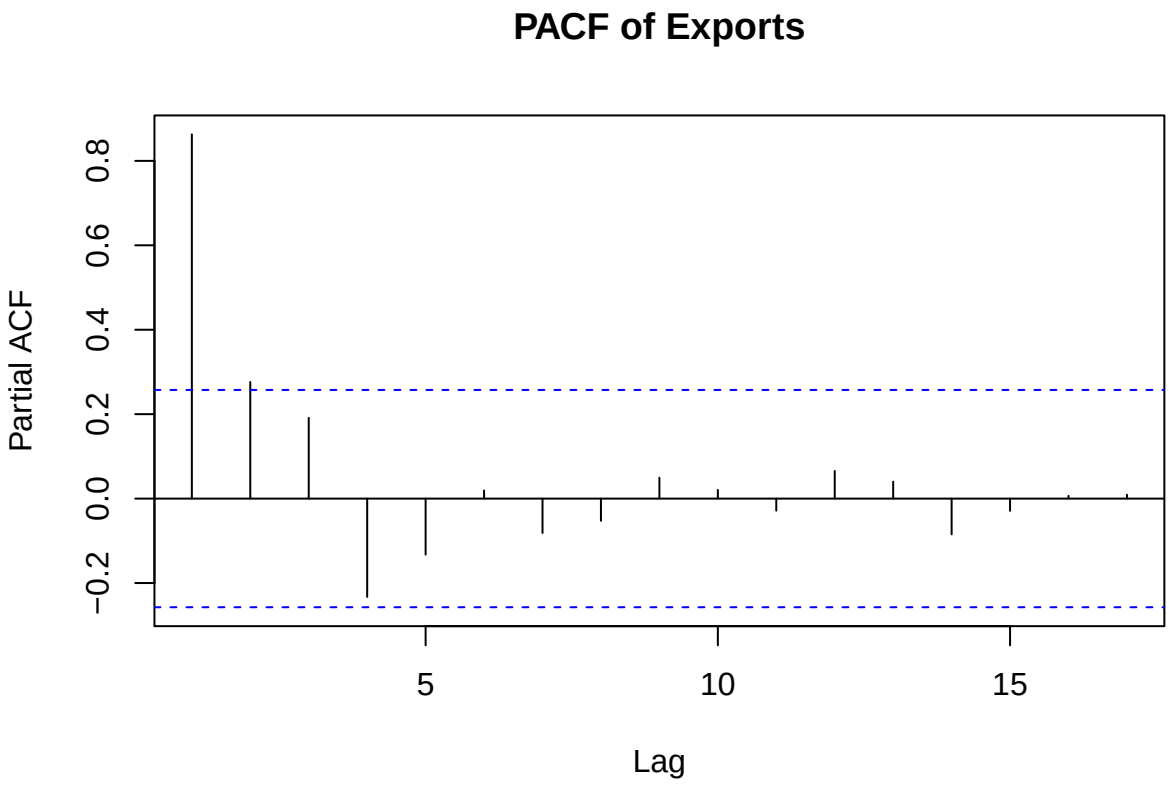
```r
pacf(gdp, main = "PACF of GDP")
```



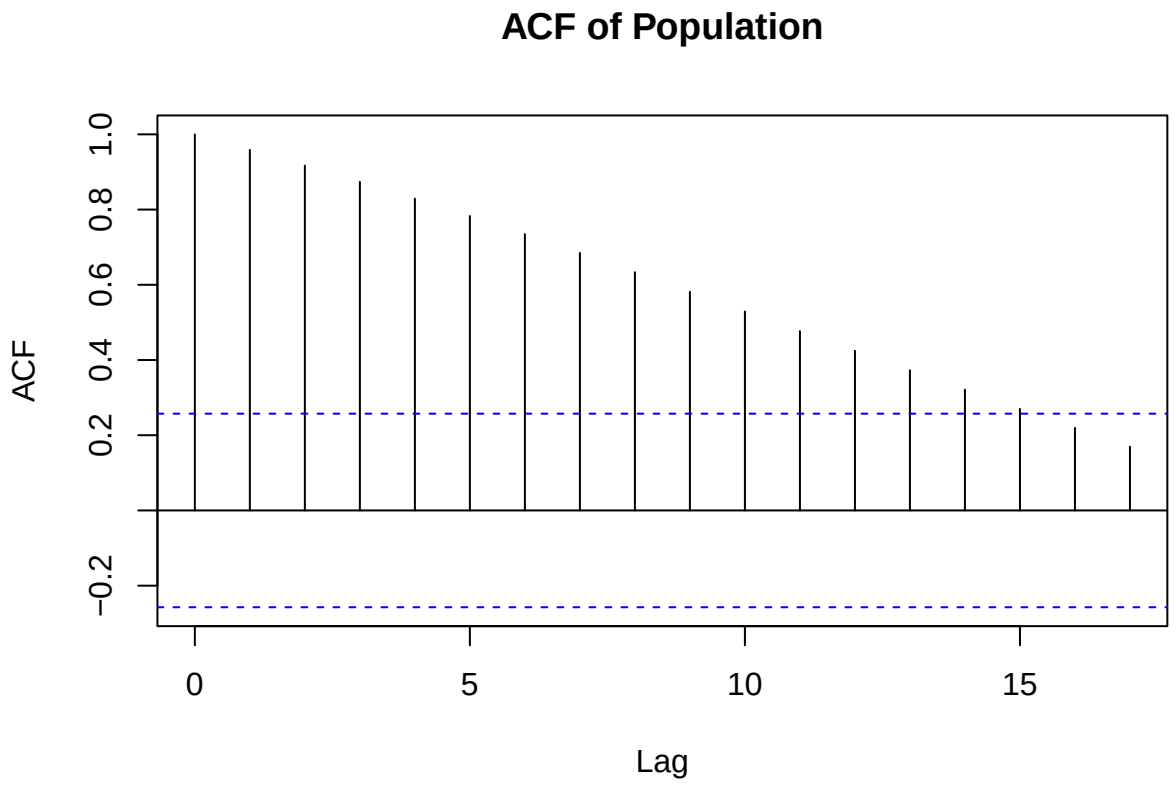

```r
acf(exports, main = "ACF of Exports")
```



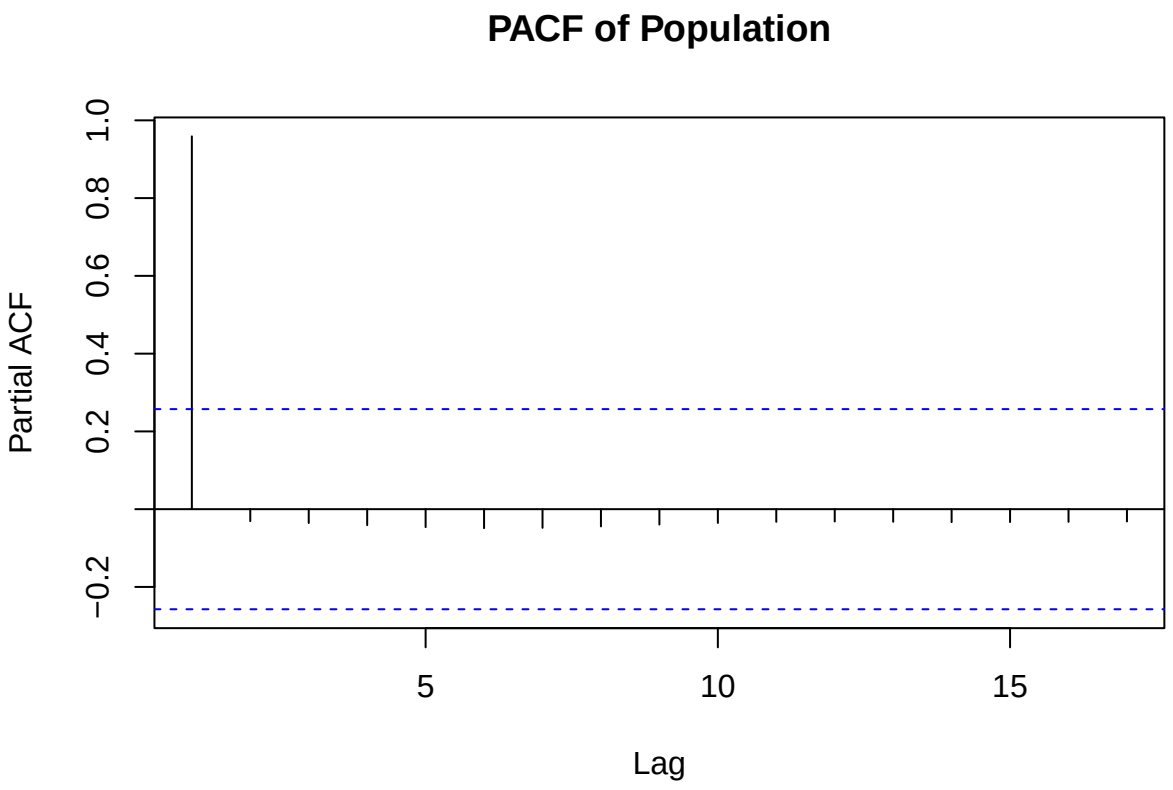

```
pacf(exports, main = "PACF of Exports")
```




```
acf(population, main = "ACF of Population")
```




```
pacf(population, main = "PACF of Population")
```



Box-Cox Transform Data

```
lambda_gdp <- BoxCox.lambda(gdp)
lambda_exports <- BoxCox.lambda(exports)
lambda_population <- BoxCox.lambda(population)

gdp_bc <- BoxCox(gdp, lambda_gdp)
exports_bc <- BoxCox(exports, lambda_exports)
population_bc <- BoxCox(population, lambda_population)

lambda_gdp
```

```
## [1] -0.2054777
```

```
lambda_exports
```

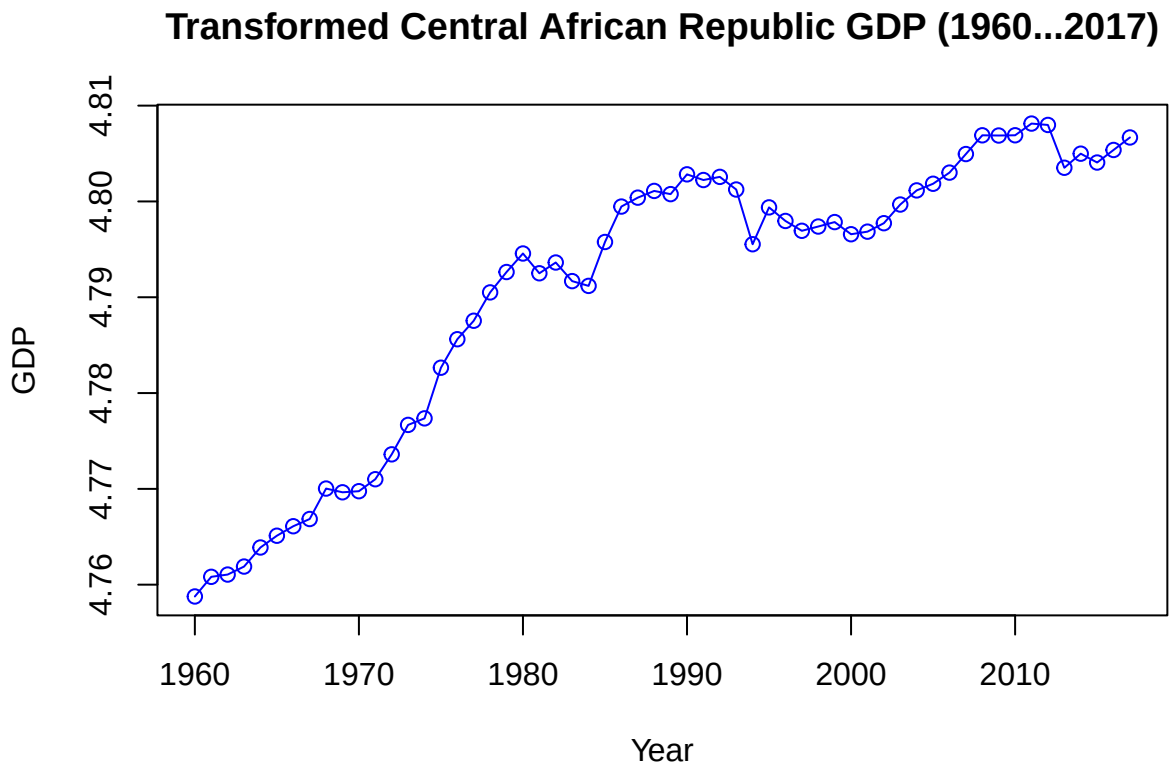
```
## [1] -0.3650884
```

```
lambda_population
```

```
## [1] 0.2206428
```

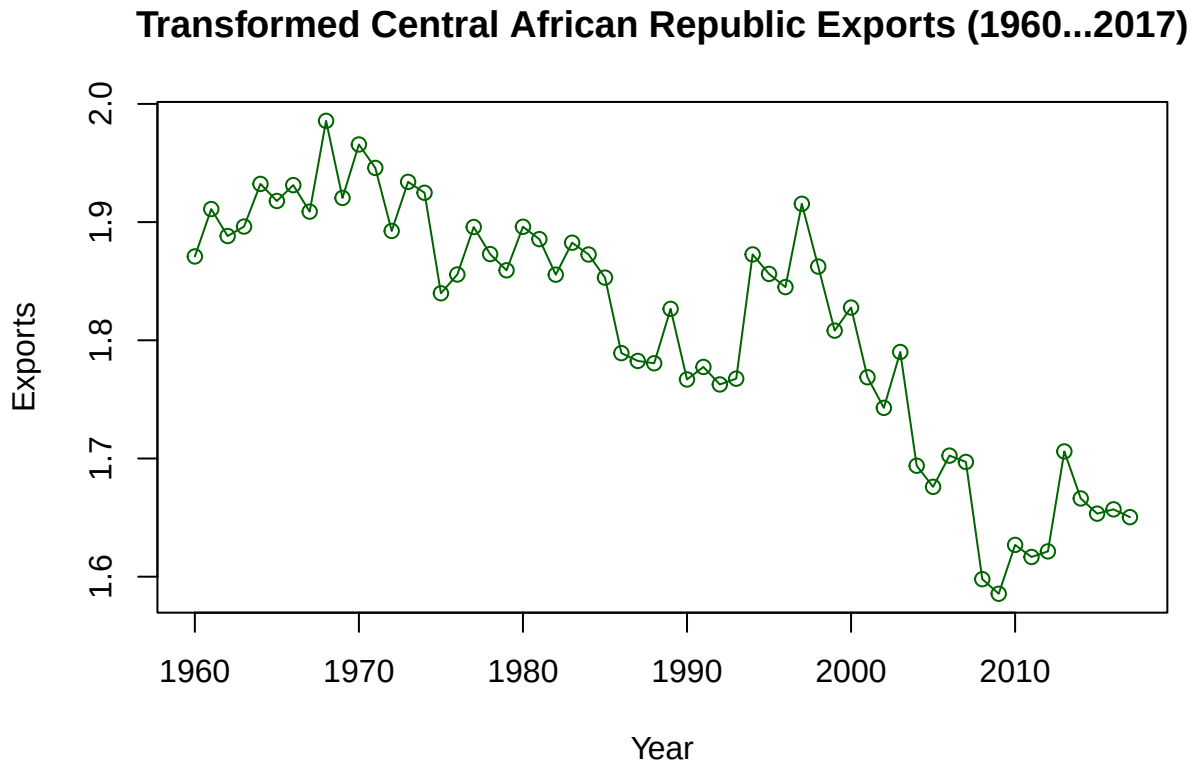
```
plot(gdp_bc,
     type = "o",
     col = "blue",
     ylab = "GDP",
     xlab = "Year",
     main = "Transformed Central African Republic GDP (1960-2017)")
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic GDP (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <e2>

## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic GDP (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <80>

## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic GDP (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <93>
```




```
plot(exports_bc,
     type = "o",
     col = "darkgreen",
     ylab = "Exports",
     xlab = "Year",
     main = "Transformed Central African Republic Exports (1960-2017)")
```

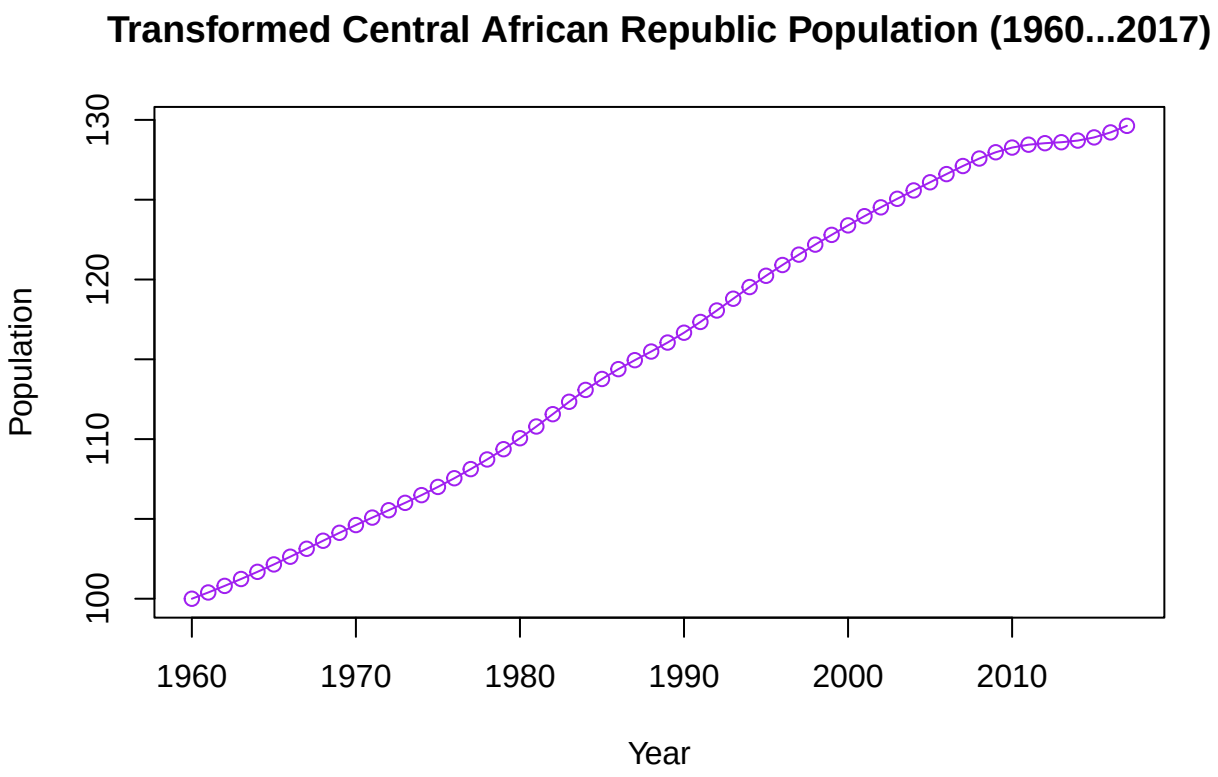
```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Exports (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <e2>

## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Exports (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <80>

## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Exports (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <93>
```




```
plot(population_bc,
     type = "o",
     col = "purple",
     ylab = "Population",
     xlab = "Year",
     main = "Transformed Central African Republic Population (1960-2017)")
```

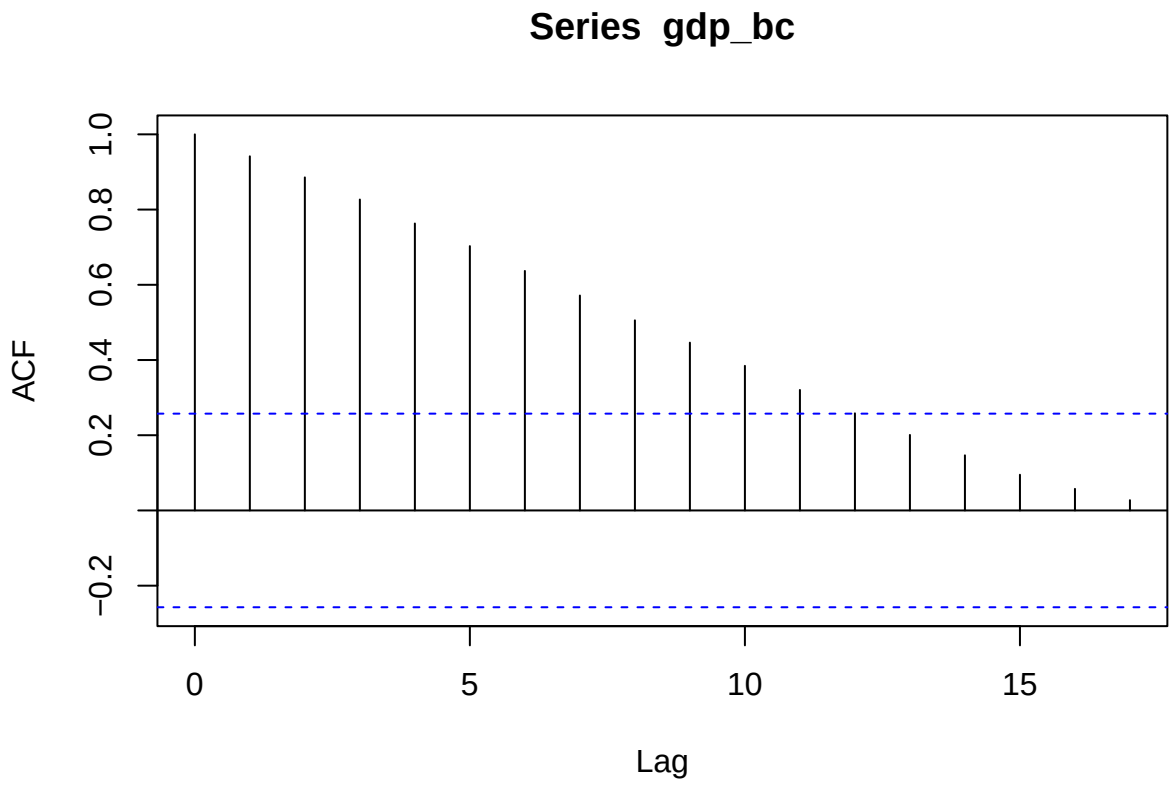
```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Population (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <e2>

## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Population (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <80>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Population (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <93>
```

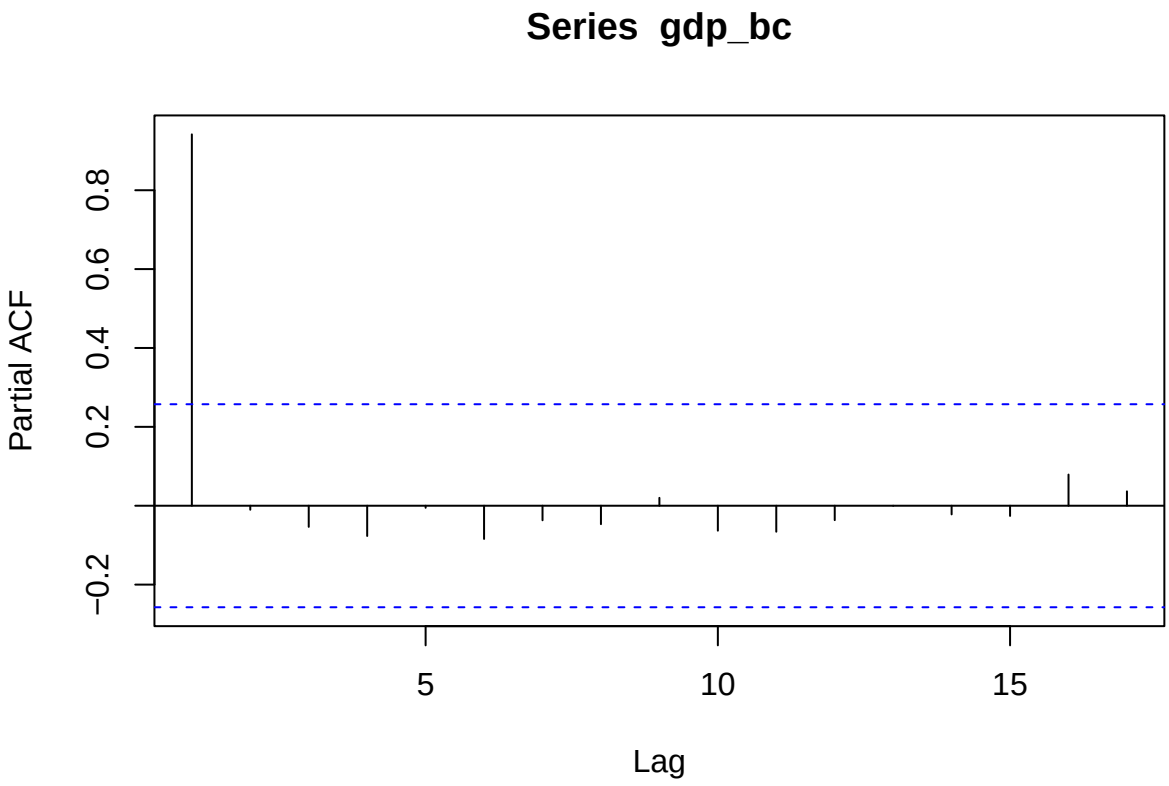


ACF/PACF Box-Cox Transform Data

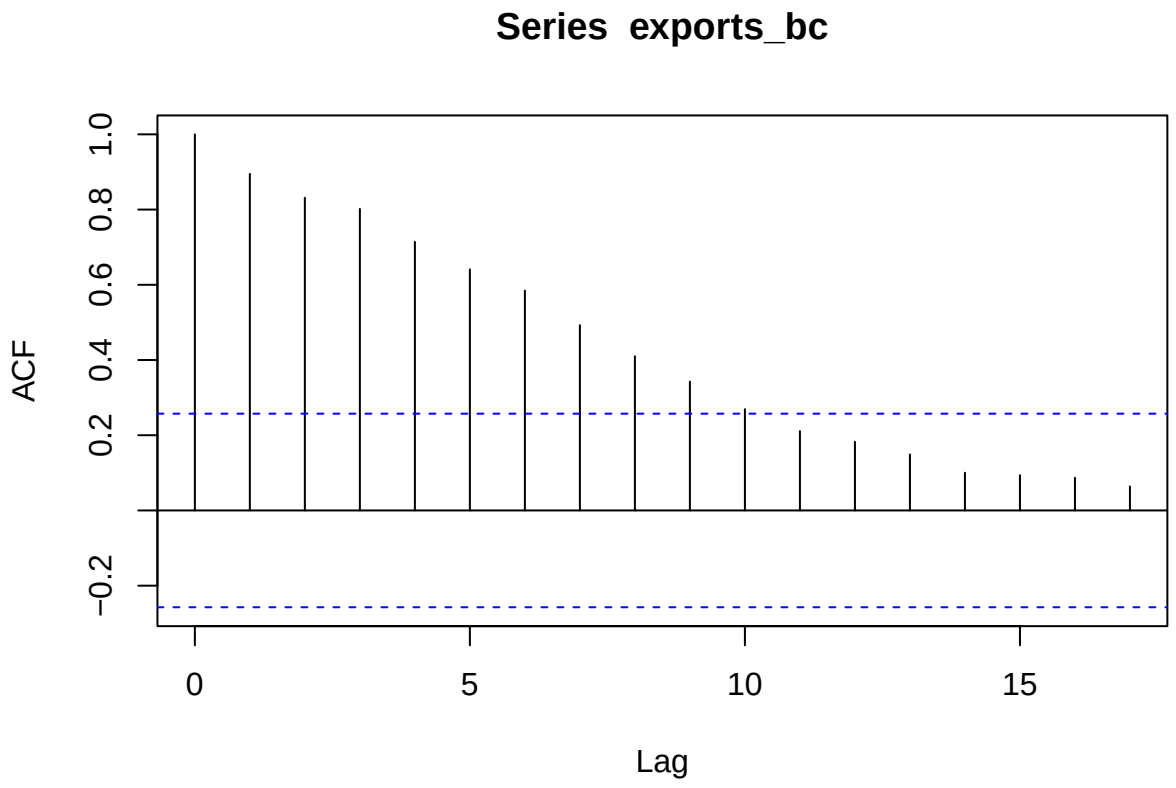
```
acf(gdp_bc)
```



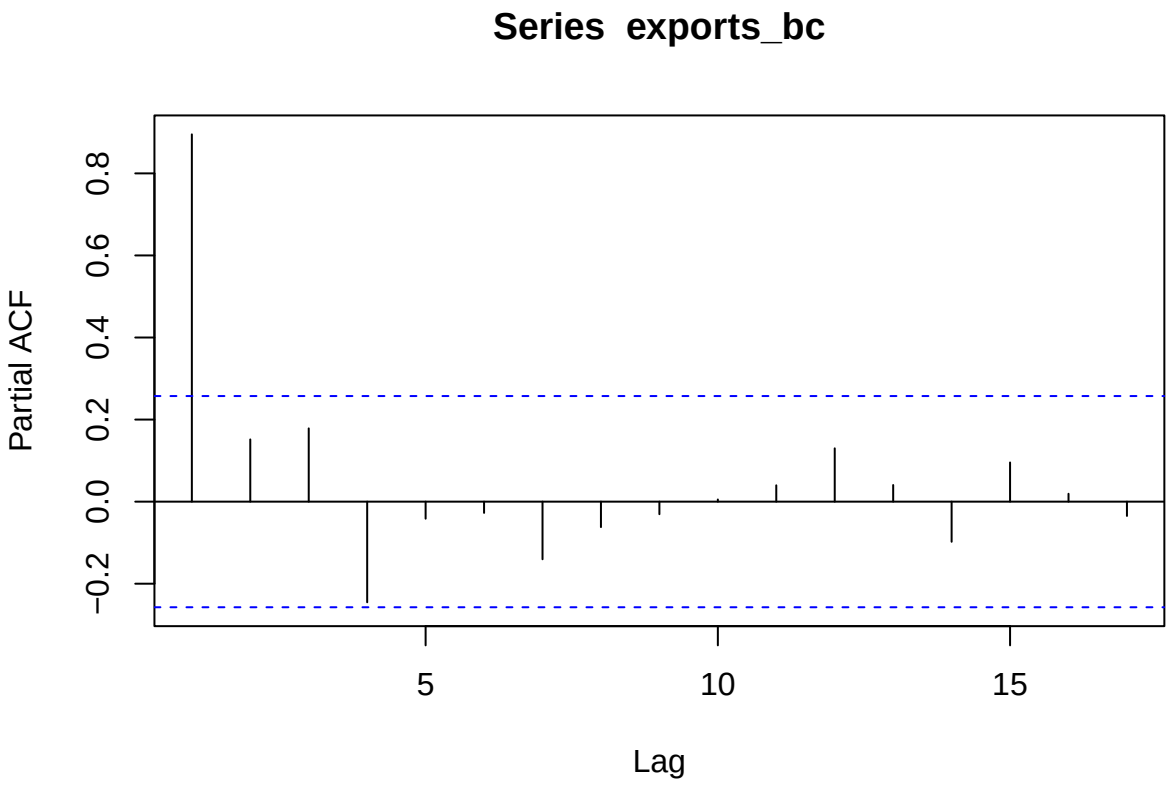

```
pacf(gdp_bc)
```



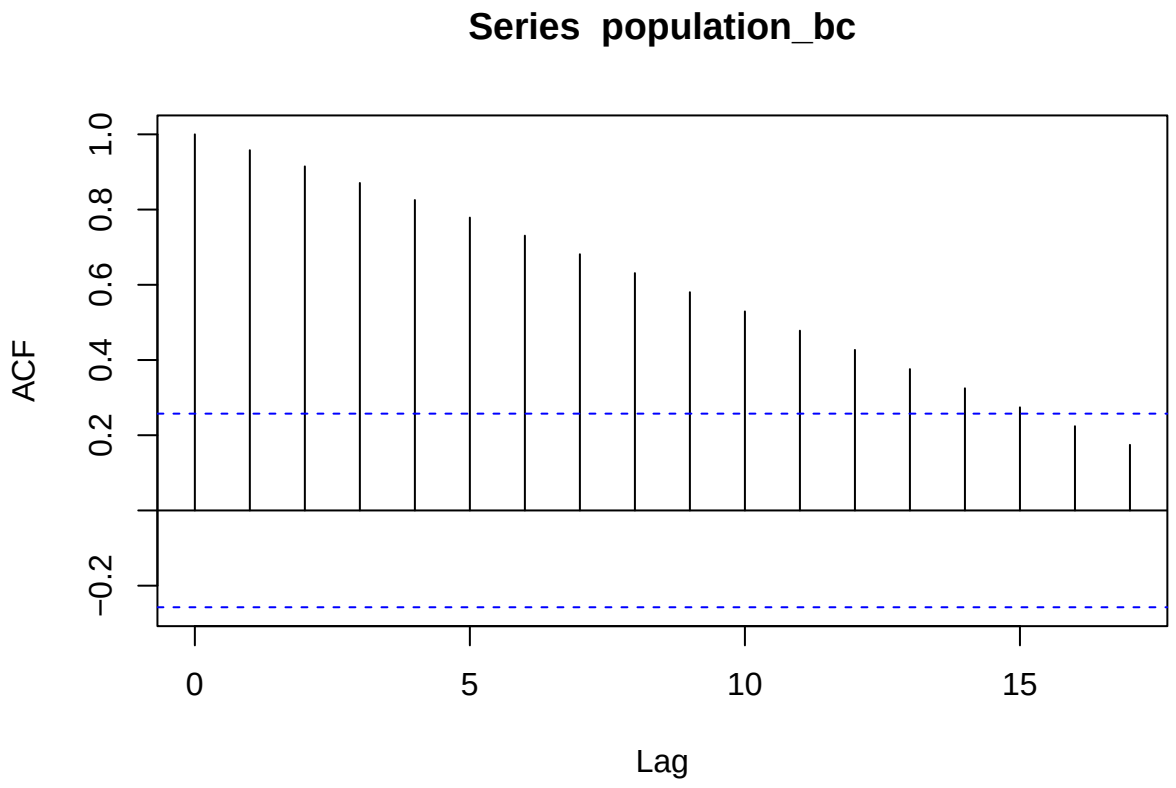

```
acf(exports_bc)
```



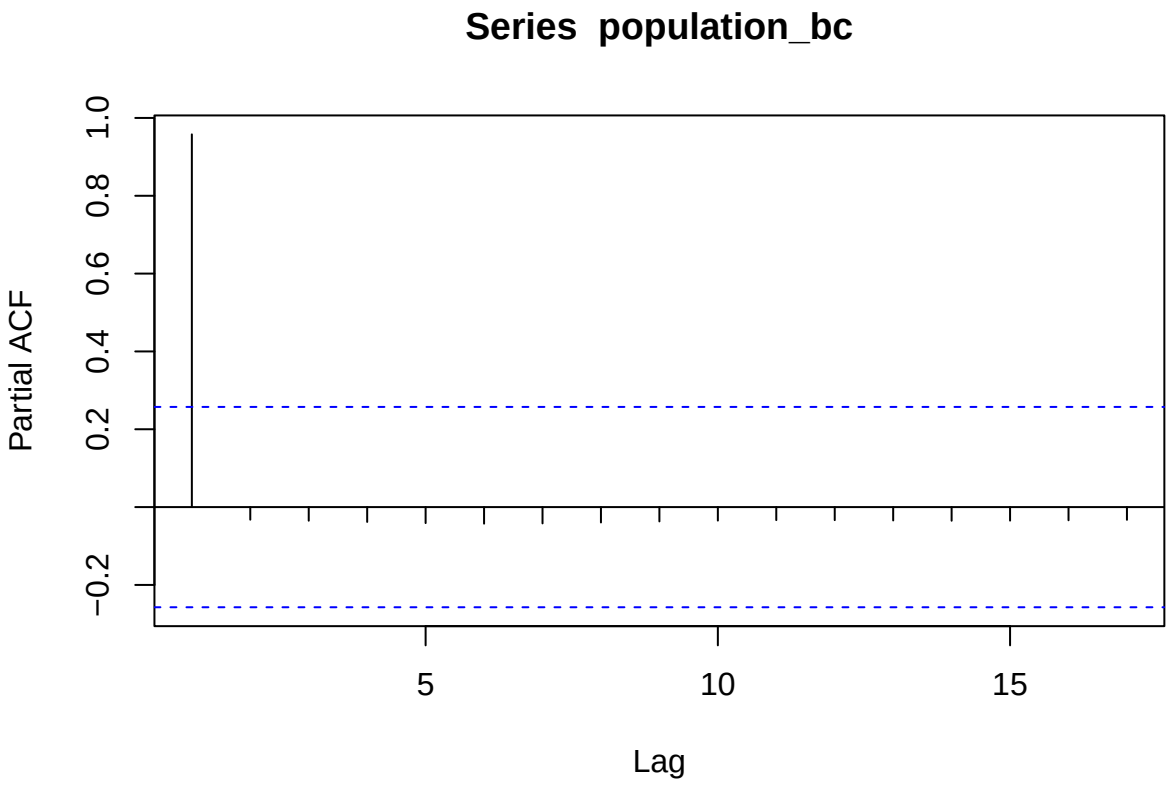

```
pacf(exports_bc)
```




```
acf(population_bc)
```




```
pacf(population_bc)
```

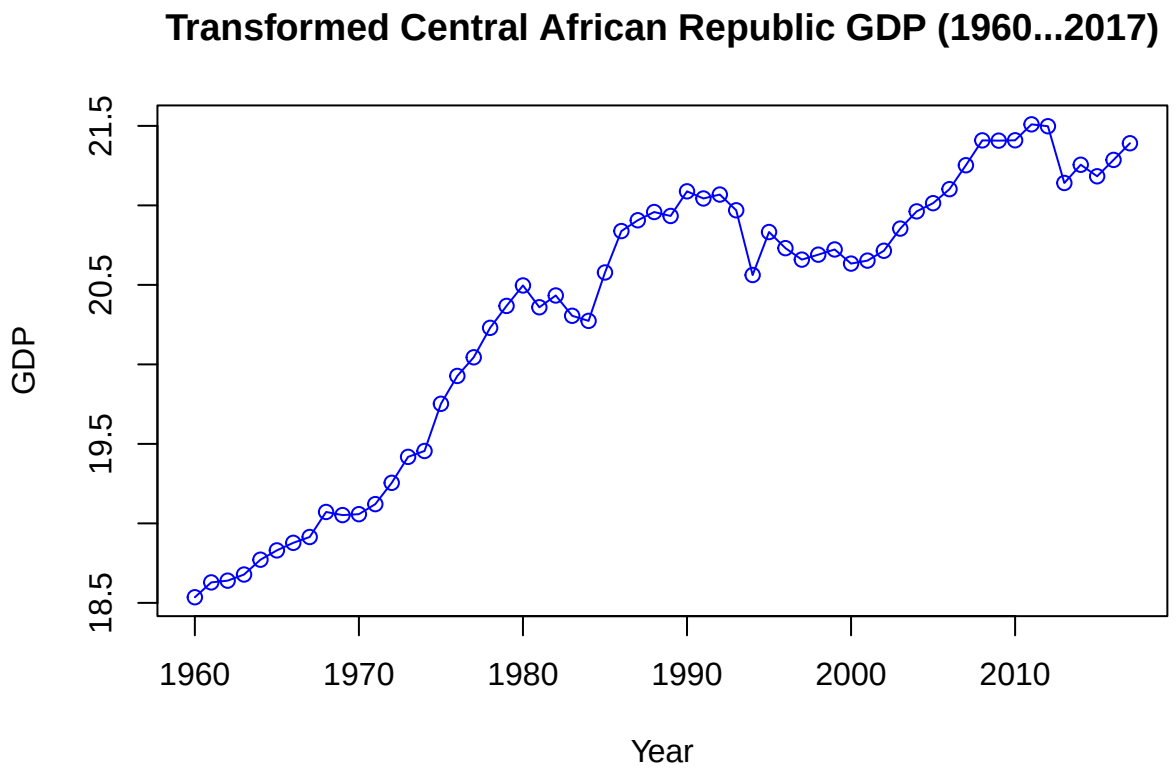


Performs well but not as interpretable. This is expected because Box-Cox has better performance.

Log-Transform Data

```
log_gdp <- log(gdp)
log_exports <- log(exports)
log_population <- log(population)

plot(log_gdp,
     type = "o",
     col = "blue",
     ylab = "GDP",
     xlab = "Year",
     main = "Transformed Central African Republic GDP (1960-2017)")
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic GDP (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <e2>

## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic GDP (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <80>

## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic GDP (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <93>
```

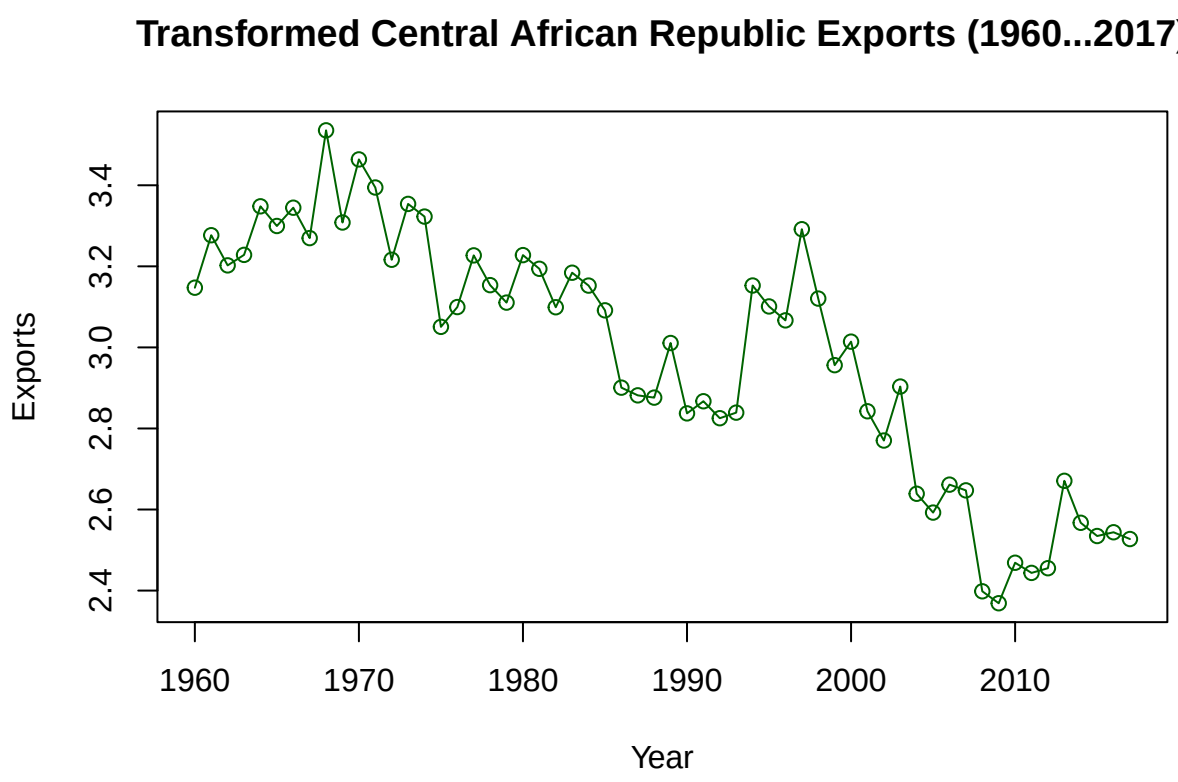



```
plot(log_exports,
     type = "o",
     col = "darkgreen",
     ylab = "Exports",
     xlab = "Year",
     main = "Transformed Central African Republic Exports (1960-2017)")
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Exports (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <e2>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Exports (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <80>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Exports (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <93>
```

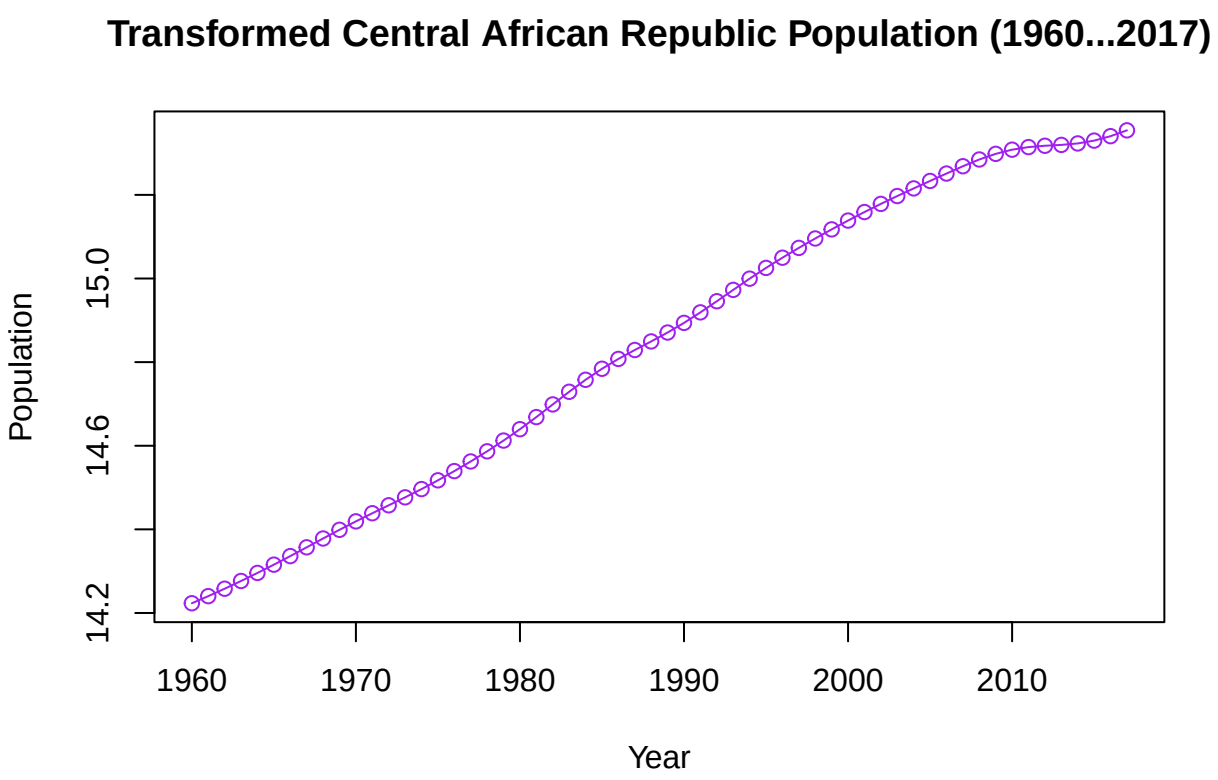



```
plot(log_population,
     type = "o",
     col = "purple",
     ylab = "Population",
     xlab = "Year",
     main = "Transformed Central African Republic Population (1960-2017)")
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Population (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <e2>
```

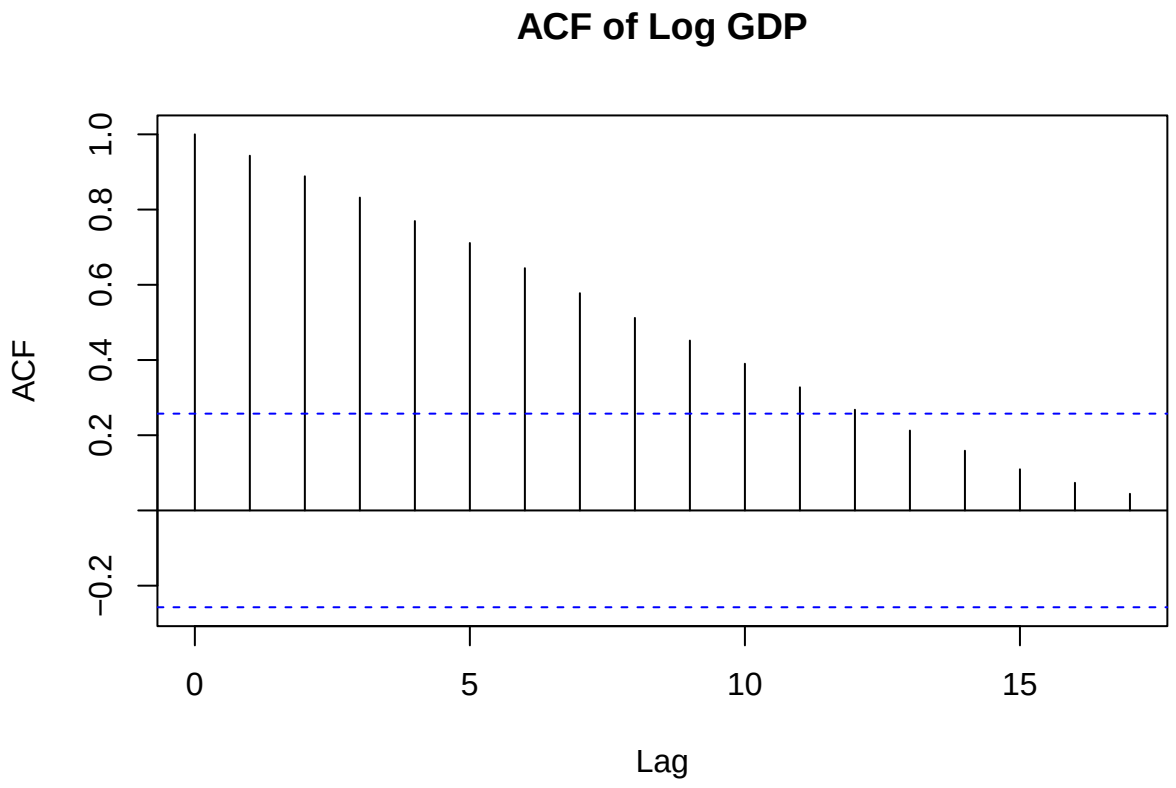
```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Population (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <80>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Population (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <93>
```

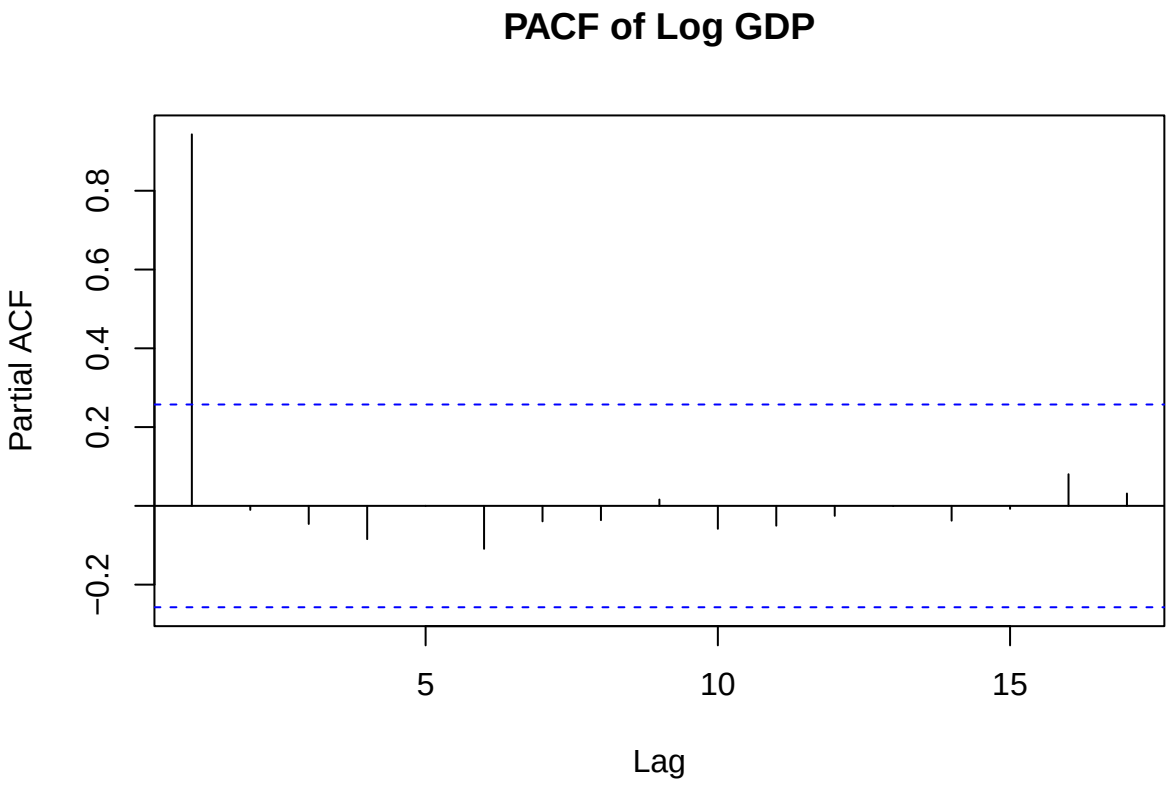


ACF/PACF - Log Transformed Data

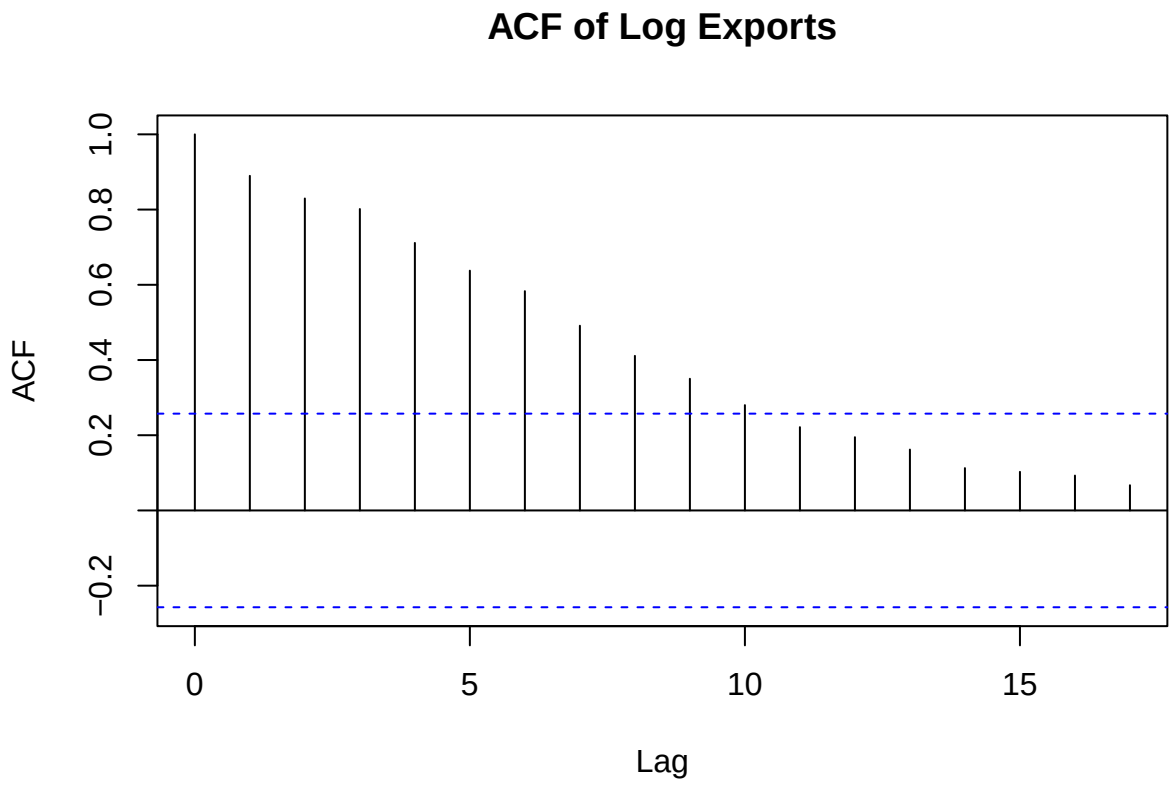
```
acf(log_gdp, main = "ACF of Log GDP")
```



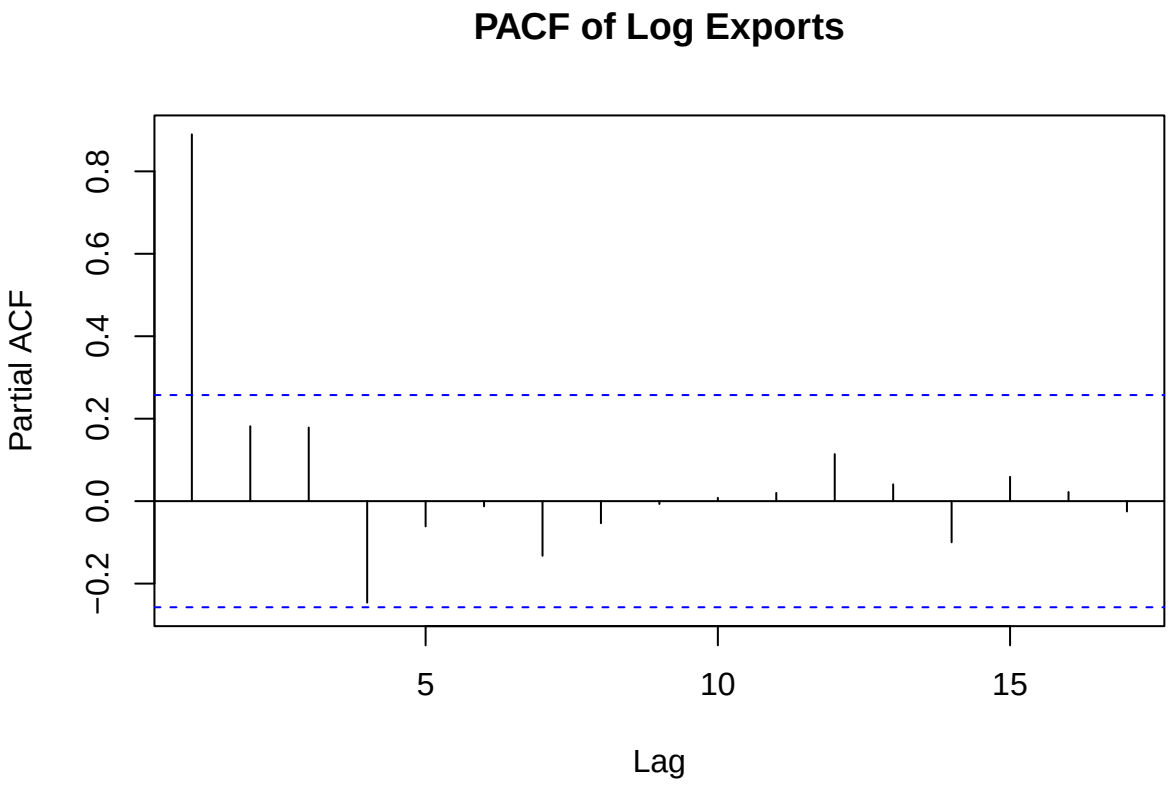

```
pacf(log_gdp, main = "PACF of Log GDP")
```



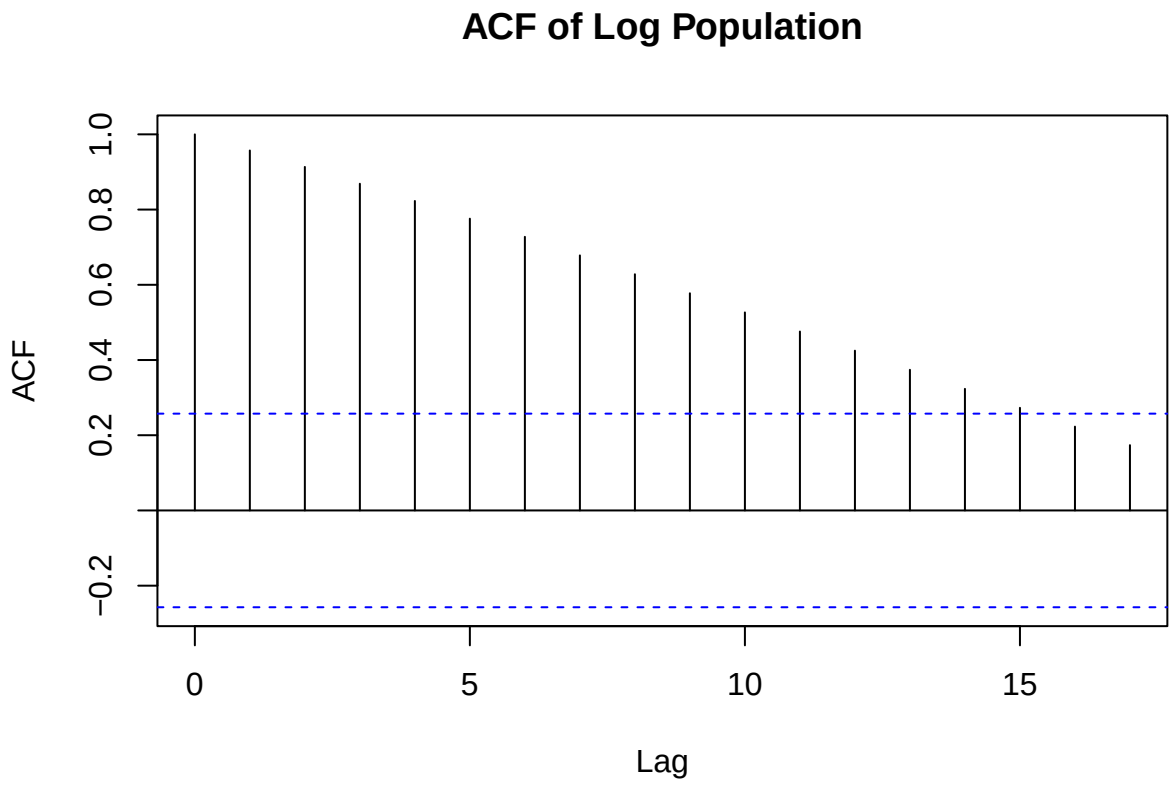

```
acf(log_exports, main = "ACF of Log Exports")
```



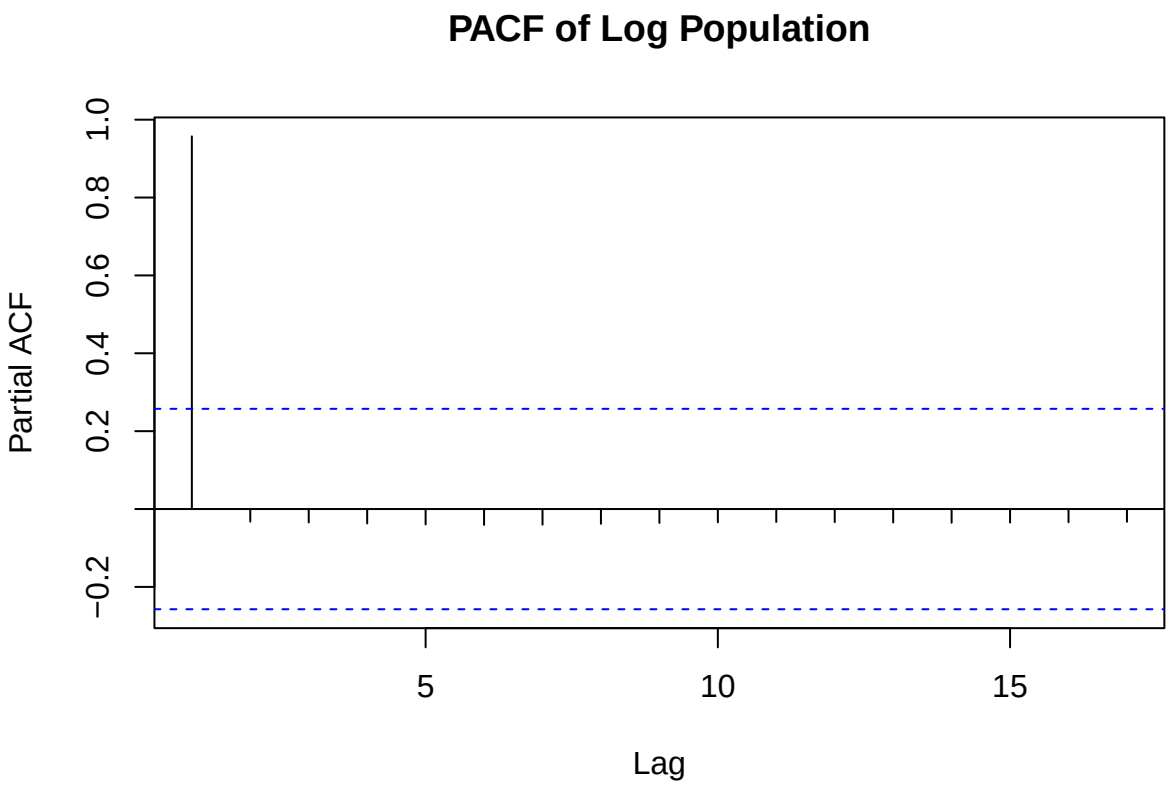

```
pacf(log_exports, main = "PACF of Log Exports")
```




```r
acf(log_population, main = "ACF of Log Population")
```




```
pacf(log_population, main = "PACF of Log Population")
```



Log transformation improves plots slightly, but still needs differencing since log transformation plot is non-stationary.The transformed Central African Republic GDP and Population both have upwards trend and the transformed exports data has a downwards trend. This means we have non-stationary data and will still need to do differencing.

Log-Outliers

```
log_gdp_z <- scale(log_gdp)
which(abs(log_gdp_z) > 3)
```

```
## integer(0)
```

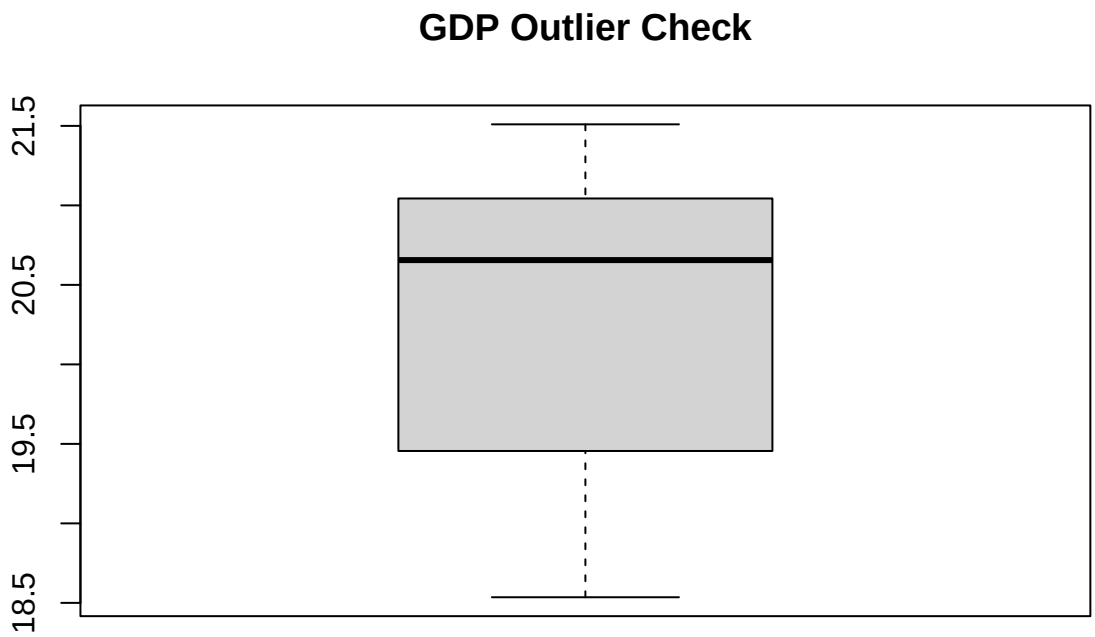
```
log_exports_z <- scale(log_exports)
which(abs(log_exports_z) > 3)
```

```
## integer(0)
```

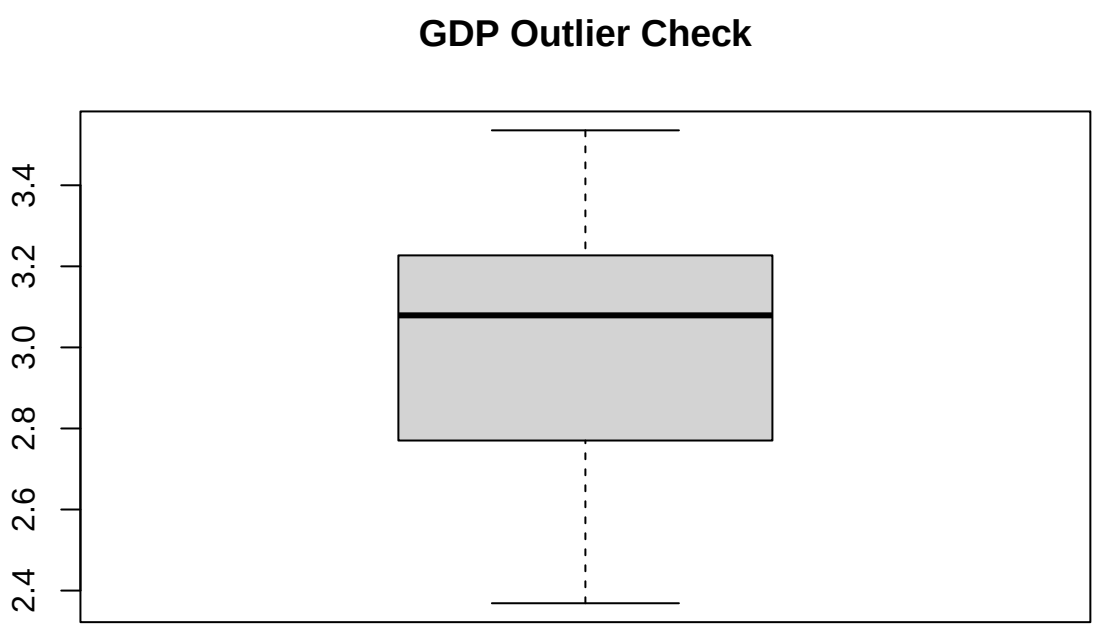
```
log_pop_z <- scale(log_population)
which(abs(log_pop_z) > 3)
```

```
## integer(0)
```

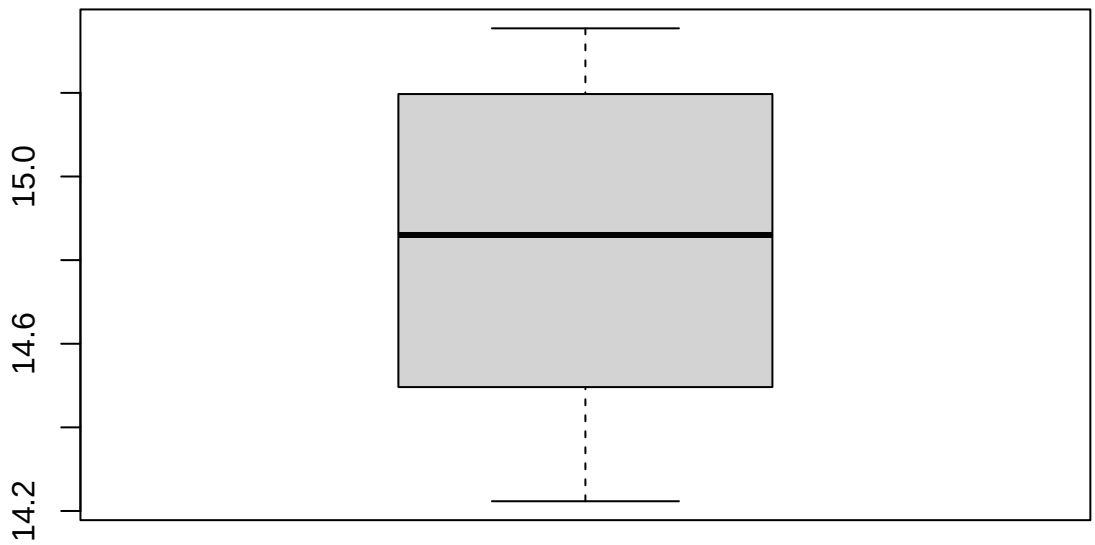
```
boxplot(log_gdp, main="GDP Outlier Check")
```




```
boxplot(log_exports, main="GDP Outlier Check")
```




```
boxplot(log_population, main="GDP Outlier Check")
```




```
tsoutliers(log_gdp)
```

```
## $index
## integer(0)
##
## $replacements
## numeric(0)
```

```
tsoutliers(log_exports)
```

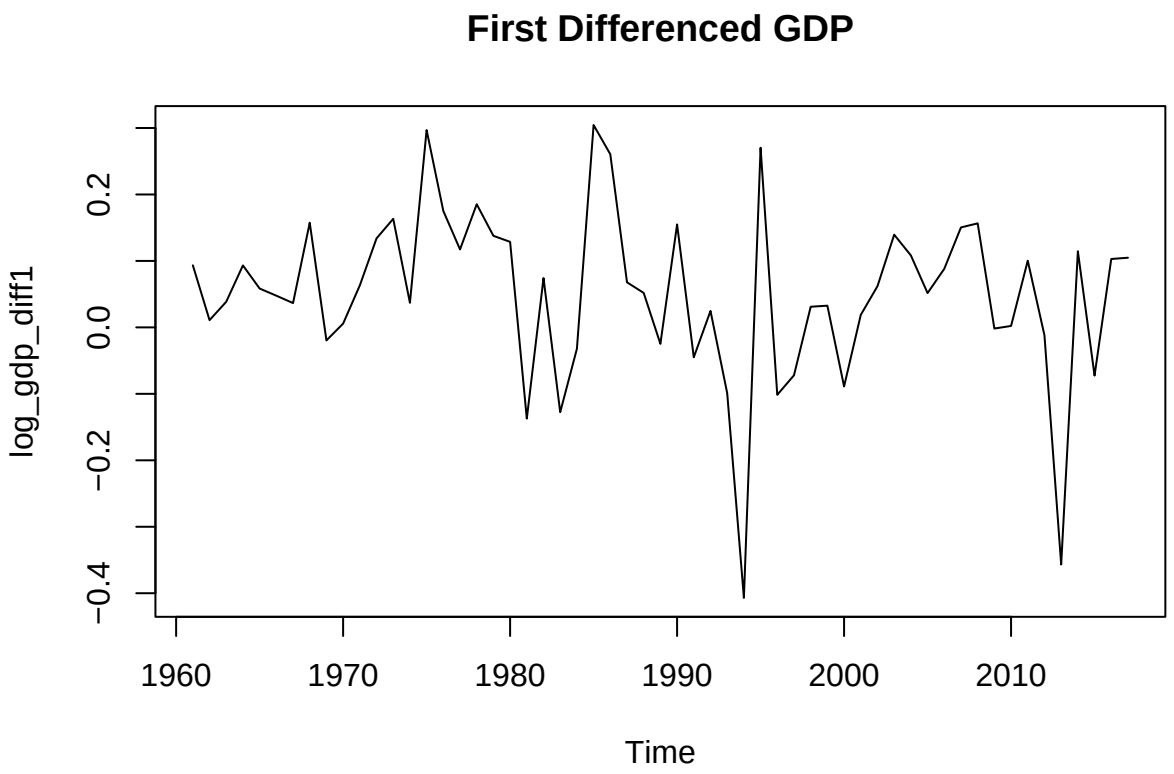
```
## $index
## integer(0)
##
## $replacements
## numeric(0)
```

```
tsoutliers(log_population)
```

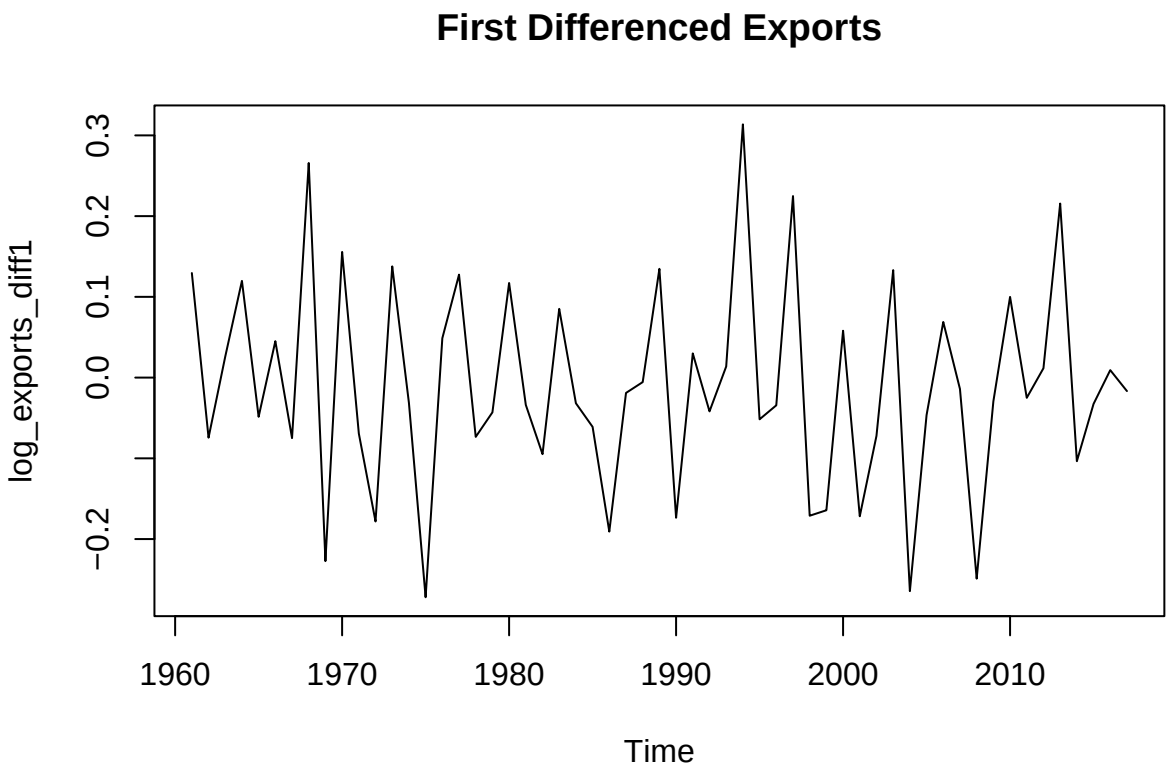
```
## $index
## integer(0)
##
## $replacements
## numeric(0)
```

Stationary - (Differencing) Log Transformed Data Differencing can help stabilize the mean of a time series by removing changes in the level of a time series, and therefore eliminating (or reducing) trend and seasonality.

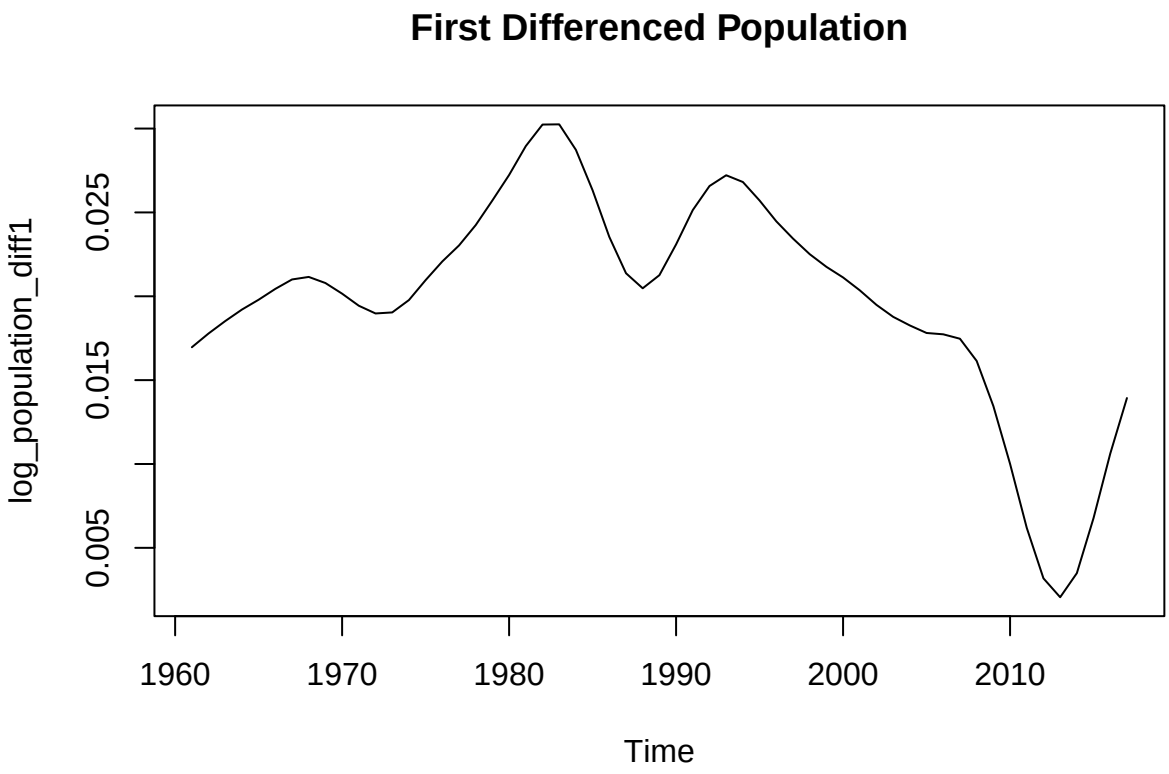
```
log_gdp_diff1 <- diff(log_gdp, differences = 1)
plot(log_gdp_diff1, type = "l", main = "First Differenced GDP")
```



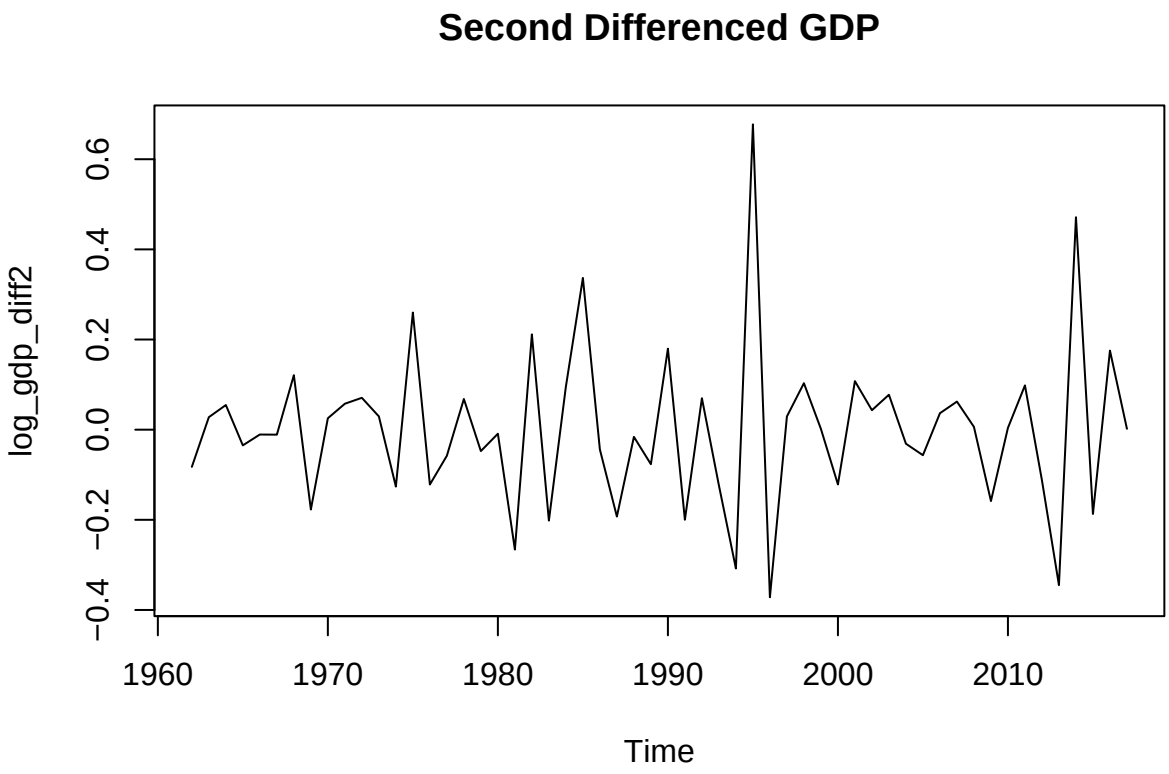

```
log_exports_diff1 <- diff(log_exports, differences = 1)
plot(log_exports_diff1, type = "l", main = "First Differenced Exports")
```



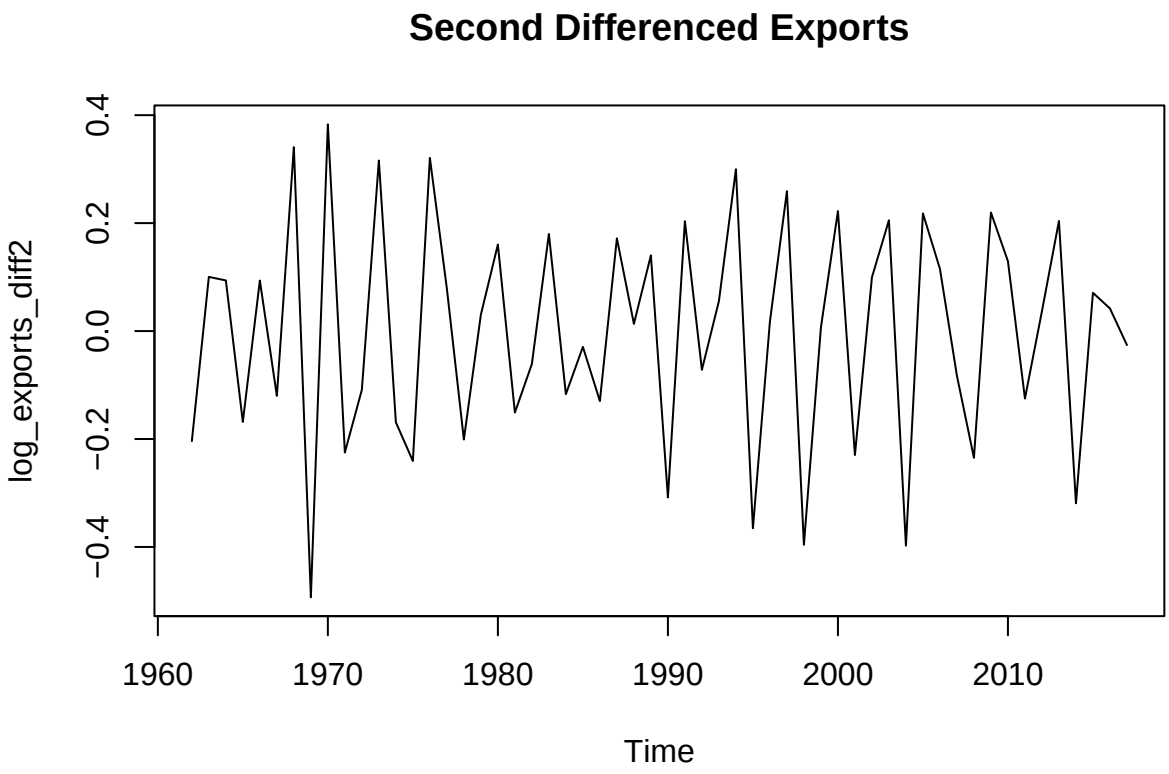

```r
log_population_diff1 <- diff(log_population, differences = 1)
plot(log_population_diff1, type = "l", main = "First Differenced Population")
```



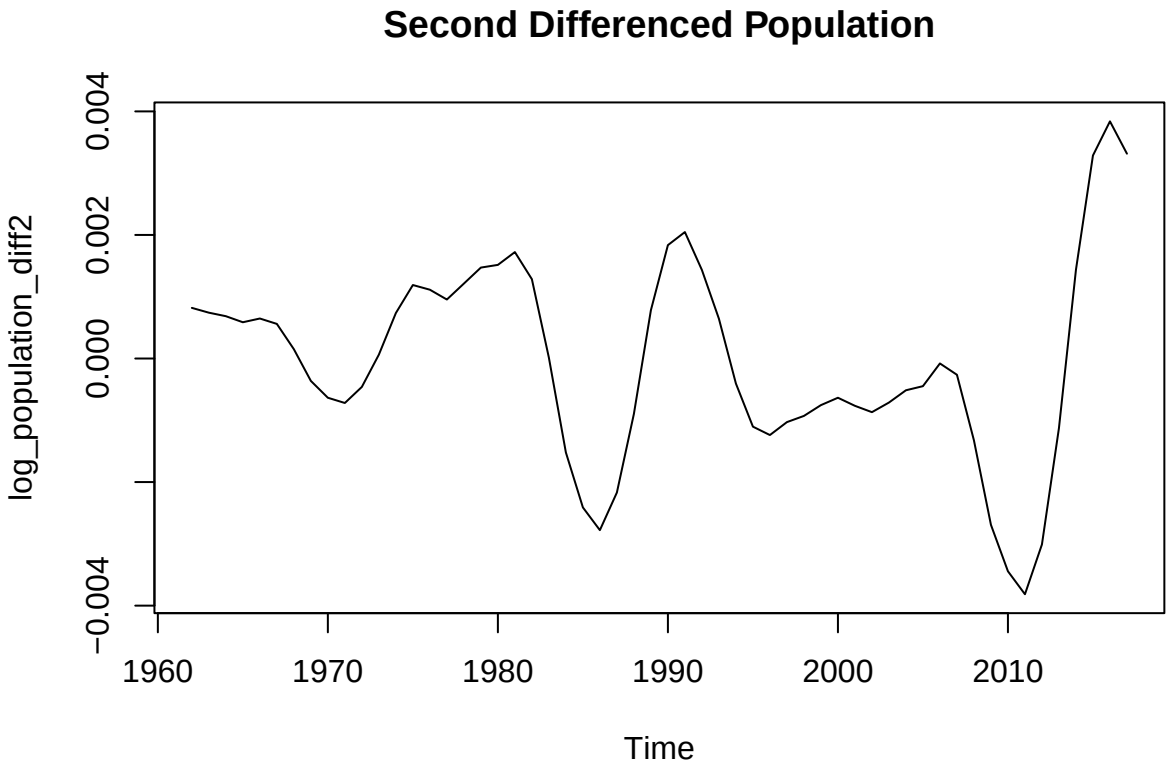

```r
log_gdp_diff2 <- diff(log_gdp, differences = 2)
plot(log_gdp_diff2, type = "l", main = "Second Differenced GDP")
```




```
log_exports_diff2 <- diff(log_exports, differences = 2)
plot(log_exports_diff2, type = "l", main = "Second Differenced Exports")
```




```
log_population_diff2 <- diff(log_population, differences = 2)
plot(log_population_diff2, type = "l", main = "Second Differenced Population")
```



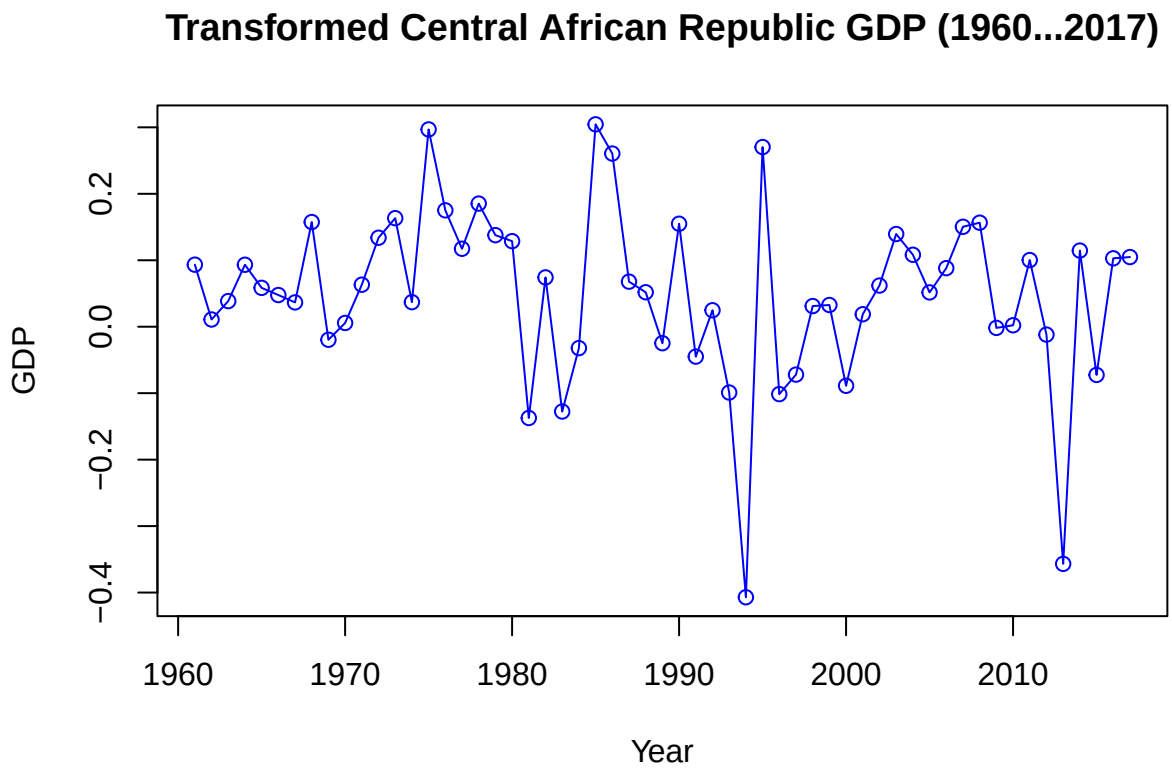
Log Transformed Difference 1:

```
plot(log_gdp_diff1,
     type = "o",
     col = "blue",
     ylab = "GDP",
     xlab = "Year",
     main = "Transformed Central African Republic GDP (1960-2017)")
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic GDP (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <e2>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic GDP (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <80>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic GDP (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <93>
```

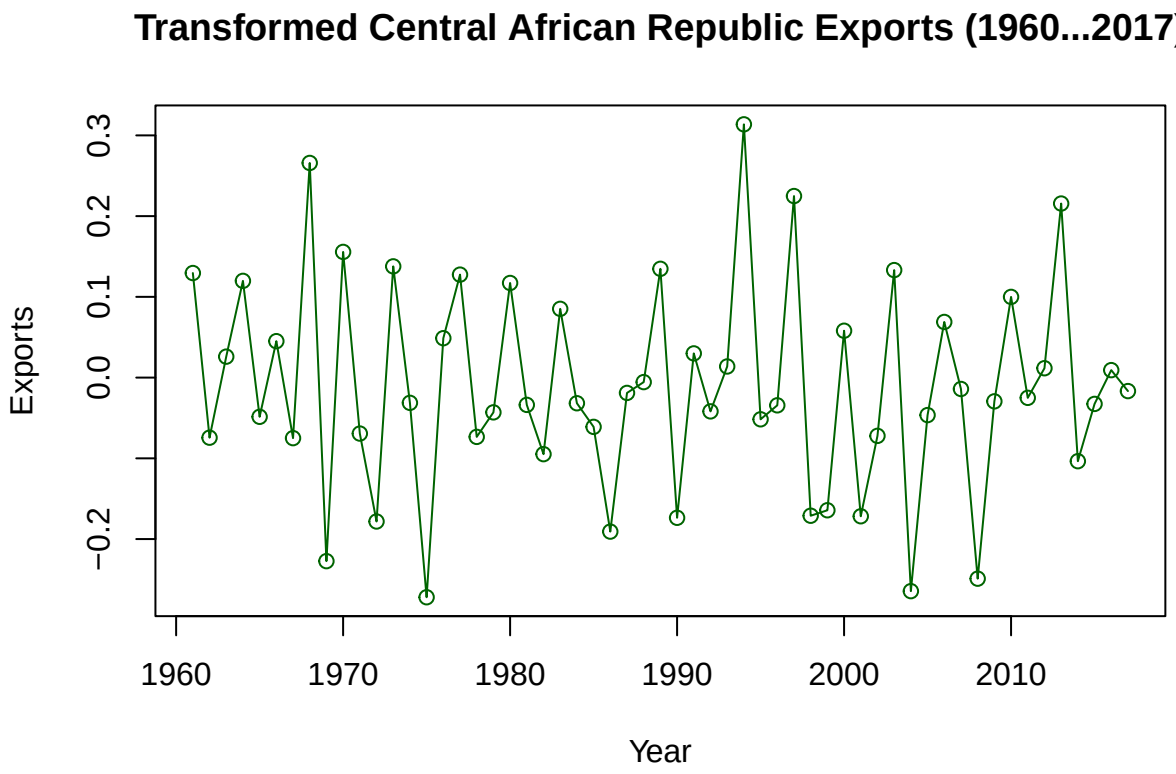



```
plot(log_exports_diff1,
     type = "o",
     col = "darkgreen",
     ylab = "Exports",
     xlab = "Year",
     main = "Transformed Central African Republic Exports (1960-2017)")
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Exports (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <e2>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Exports (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <80>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Exports (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <93>
```

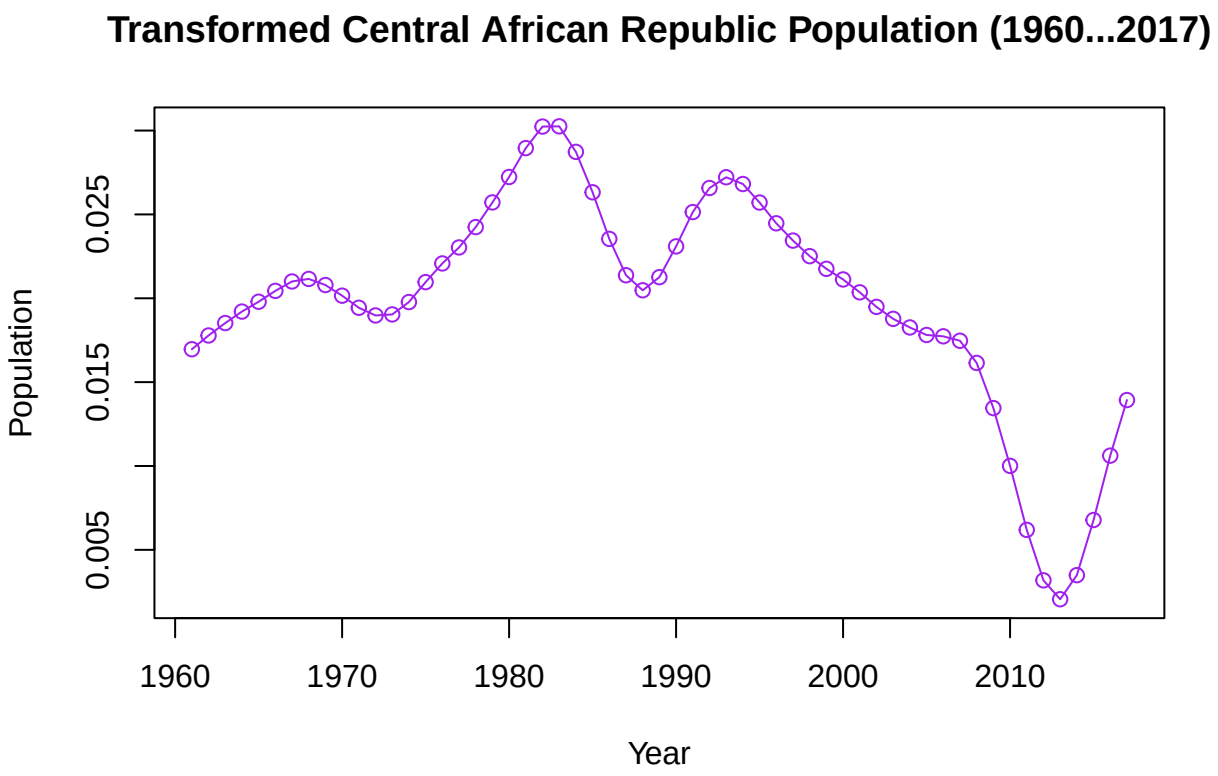



```
plot(log_population_diff1,
     type = "o",
     col = "purple",
     ylab = "Population",
     xlab = "Year",
     main = "Transformed Central African Republic Population (1960-2017)")
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Population (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <e2>
```

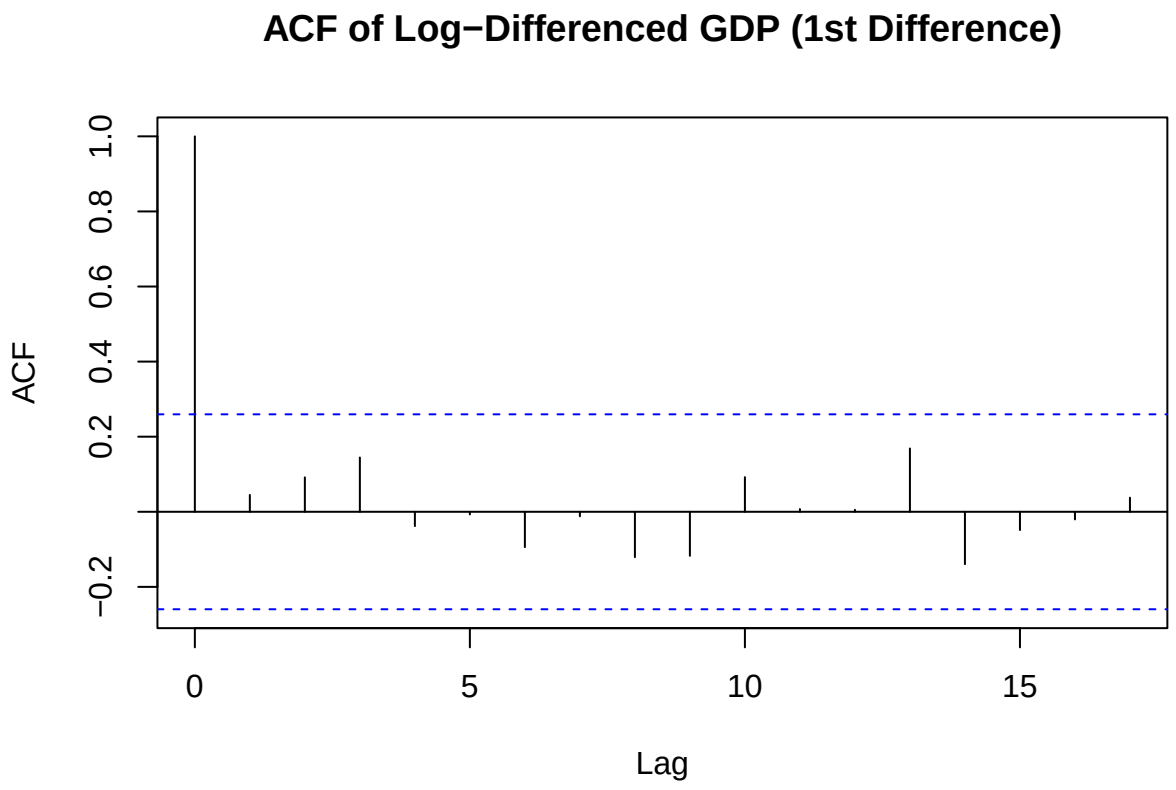
```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Population (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <80>
```

```
## Warning in title(main = main, xlab = xlab, ylab = ylab, ...): conversion
## failure on 'Transformed Central African Republic Population (1960-2017)' in
## 'mbcsToSbcs': dot substituted for <93>
```

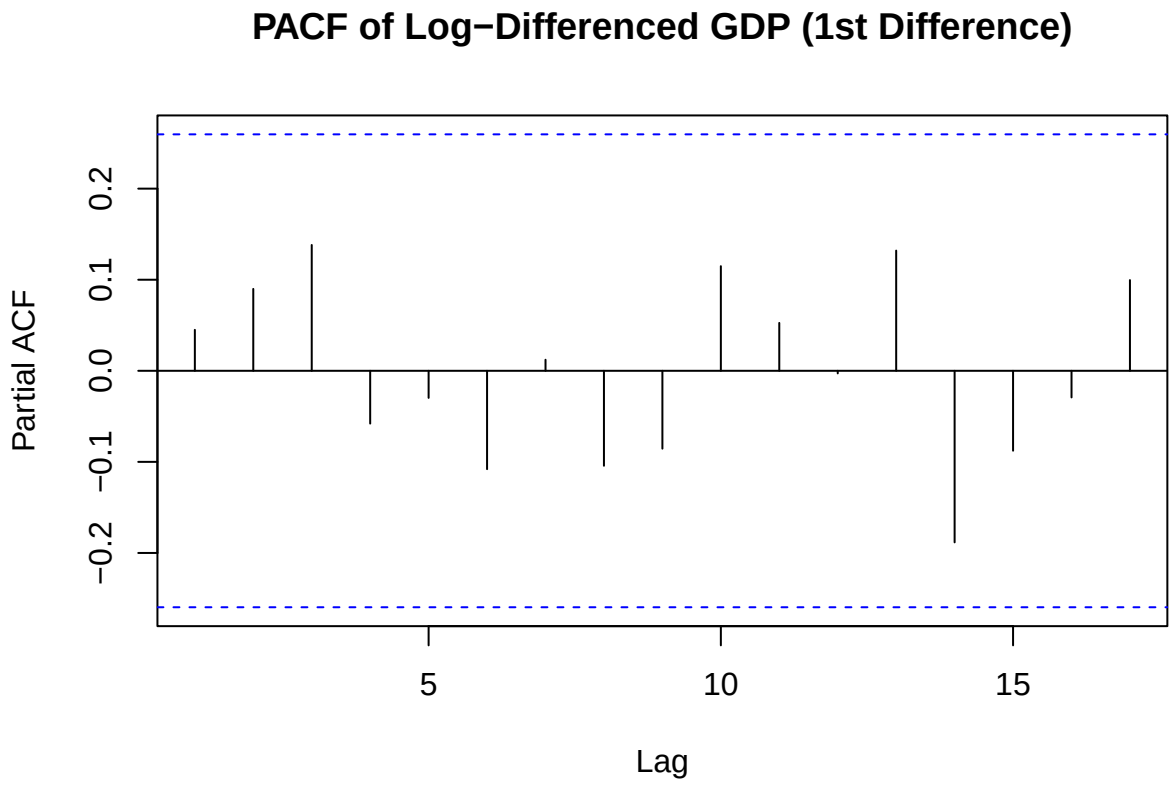


ACF/PACF Log Transformation:

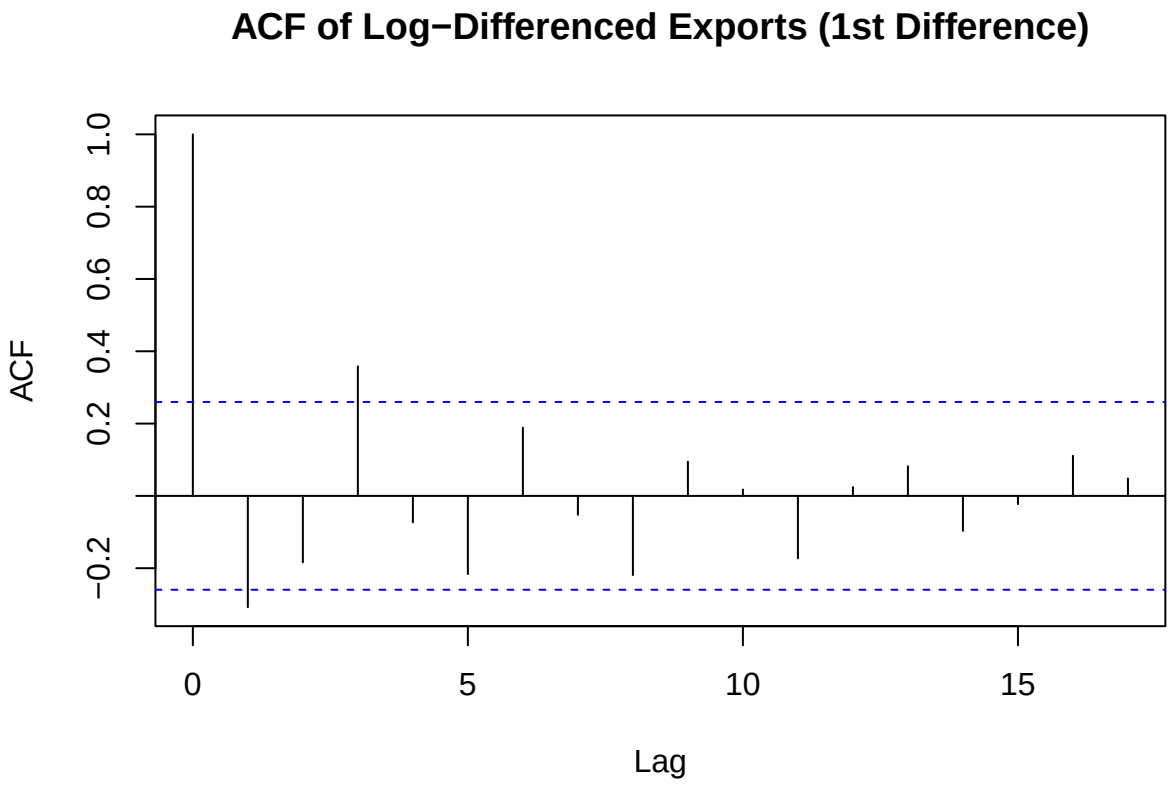
```
acf(log_gdp_diff1, main = "ACF of Log-Differenced GDP (1st Difference)")
```



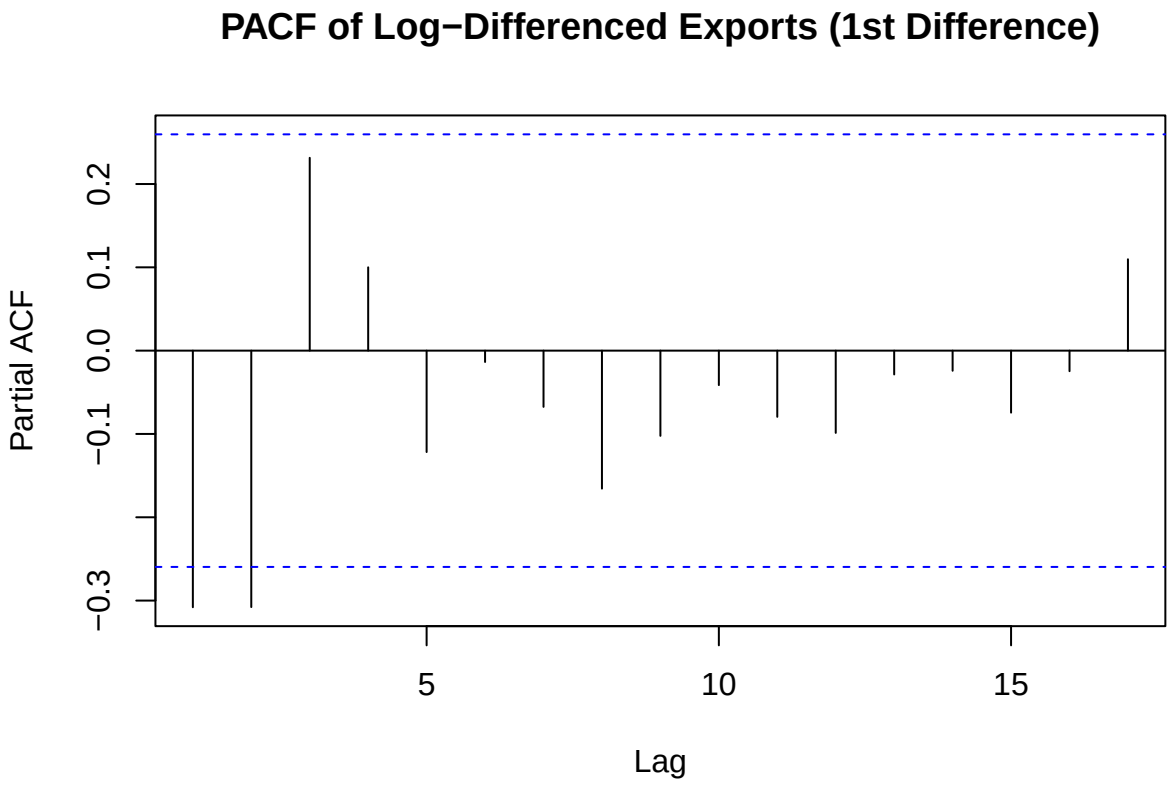

```
pacf(log_gdp_diff1, main = "PACF of Log-Differenced GDP (1st Difference)")
```



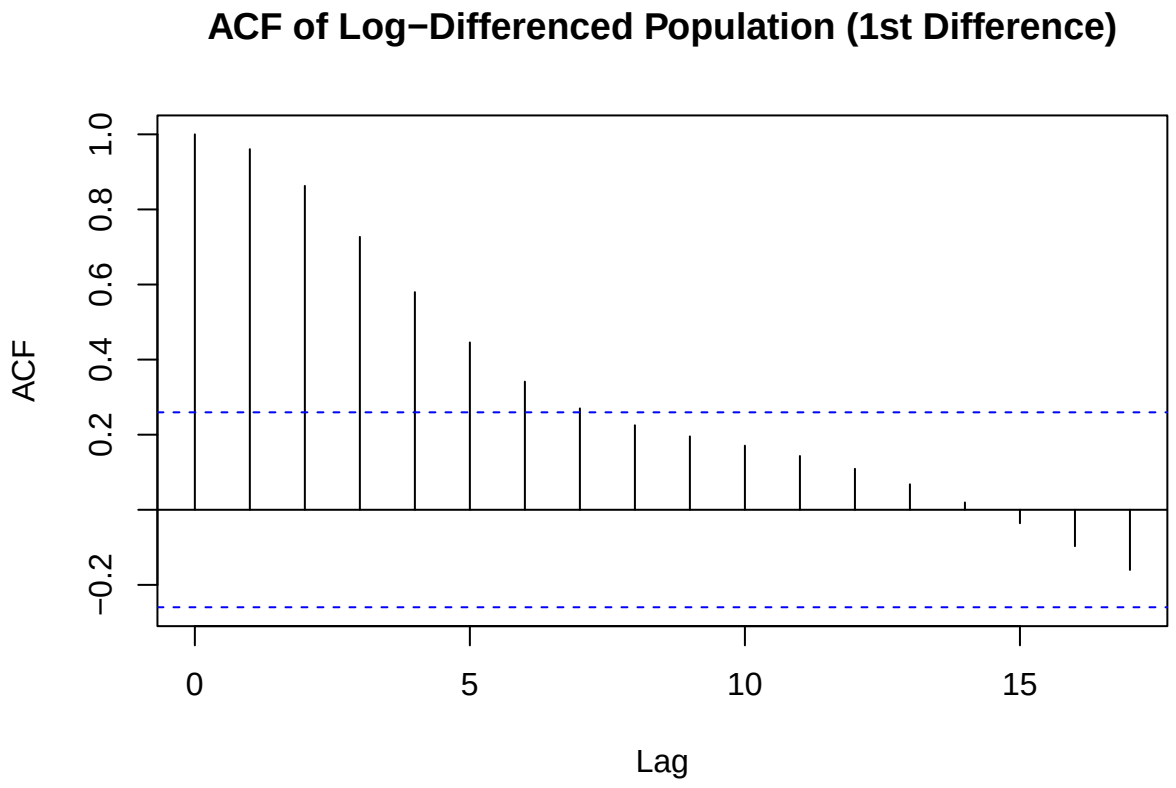

```
acf(log_exports_diff1, main = "ACF of Log-Differenced Exports (1st Difference)")
```



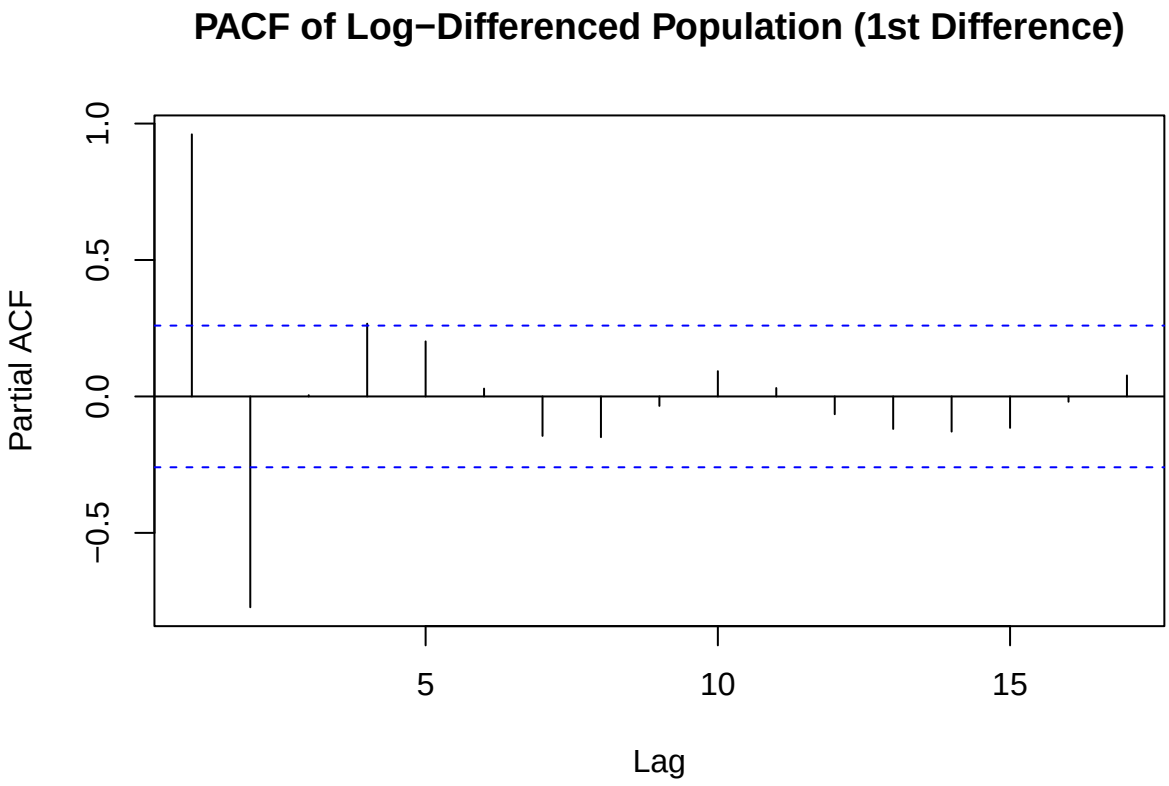

```
pacf(log_exports_diff1, main = "PACF of Log-Differenced Exports (1st Difference)")
```




```
acf(log_population_diff1, main = "ACF of Log-Differenced Population (1st Difference)")
```




```
pacf(log_population_diff1, main = "PACF of Log-Differenced Population (1st Difference)")
```




```
# acf(log_gdp_diff2)
# pacf(log_gdp_diff2)
#
# acf(log_exports_diff2)
# pacf(log_exports_diff2)
#
# acf(log_population_diff2)
# pacf(log_population_diff2)
```

With difference of 1: GDP: MA(1), MA(2) or later ARMA(1,1), ARMA(2,1)

Exports: MA(2), AR(2), or later MA(1), AR(1), ARMA(1,1), ARMA(1,2), ARMA(2,1), ARMA(2,2)

Population: AR(1), AR(2), or later ARMA(1,1), ARMA(1,2)

Prefer simpler models

Just for reference:

```
library(tseries)
```

```
## Warning: package 'tseries' was built under R version 4.3.3
```

```
adf.test(log_gdp)
```

```
##
```

```
##  Augmented Dickey-Fuller Test
##
## data:  log_gdp
## Dickey-Fuller = -1.9149, Lag order = 3, p-value = 0.609
## alternative hypothesis: stationary
```

```
adf.test(log_exports)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  log_exports
## Dickey-Fuller = -2.9724, Lag order = 3, p-value = 0.1821
## alternative hypothesis: stationary
```

```
adf.test(log_population)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  log_population
## Dickey-Fuller = -2.1218, Lag order = 3, p-value = 0.5255
## alternative hypothesis: stationary
```

```
adf.test(log_gdp_diff1)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  log_gdp_diff1
## Dickey-Fuller = -3.3099, Lag order = 3, p-value = 0.07899
## alternative hypothesis: stationary
```

```
adf.test(log_exports_diff1)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  log_exports_diff1
## Dickey-Fuller = -3.263, Lag order = 3, p-value = 0.08637
## alternative hypothesis: stationary
```

```
adf.test(log_population_diff1)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  log_population_diff1
## Dickey-Fuller = -1.4801, Lag order = 3, p-value = 0.7844
## alternative hypothesis: stationary
```

```r
adf.test(log_gdp_diff2)
```

```
## Warning in adf.test(log_gdp_diff2): p-value smaller than printed p-value

##
##  Augmented Dickey-Fuller Test
##
## data:  log_gdp_diff2
## Dickey-Fuller = -5.1059, Lag order = 3, p-value = 0.01
## alternative hypothesis: stationary
```

```r
adf.test(log_exports_diff2)
```

```
## Warning in adf.test(log_exports_diff2): p-value smaller than printed p-value

##
##  Augmented Dickey-Fuller Test
##
## data:  log_exports_diff2
## Dickey-Fuller = -5.6121, Lag order = 3, p-value = 0.01
## alternative hypothesis: stationary
```

```r
adf.test(log_population_diff2)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  log_population_diff2
## Dickey-Fuller = -3.2152, Lag order = 3, p-value = 0.09399
## alternative hypothesis: stationary
```

Finding a Candidate Model GDP:

```r
#arma(p,d,q)

#GDP
gdp_m1 <- Arima(log_gdp, order = c(1,1,0), include.mean = TRUE)   # MA(1)
gdp_m2 <- Arima(log_gdp, order = c(0,1,2), include.mean = TRUE)   # MA(2)


#Exports
exports_m1 <- Arima(log_exports, order = c(0,1,2), include.mean = TRUE)   # MA(2)
exports_m2 <- Arima(log_exports, order = c(2,1,0), include.mean = TRUE)   # AR(2)
exports_m3 <- Arima(log_exports, order = c(2,1,2), include.mean = TRUE)   # ARMA(2,2)


#Population
population_m1 <- Arima(log_population, order = c(2,1,0), include.mean = TRUE)   # AR(2)
population_m2 <- Arima(log_population, order = c(1,1,0), include.mean = TRUE)   # AR(1)
```

AIC/BIC Choose BIC because it penalizes model complexity more.

```r
summary(gdp_m1)
```

```
## Series: log_gdp
## ARIMA(1,1,0)
##
## Coefficients:
##          ar1
##       0.1642
## s.e.  0.1306
##
## sigma^2 = 0.01885:  log likelihood = 32.8
## AIC=-61.61   AICc=-61.38   BIC=-57.52
##
## Training set error measures:
##                       ME      RMSE      MAE       MPE     MAPE     MASE
## Training set 0.04174331 0.1349037 0.102239 0.2077372 0.499746 0.963075
##                     ACF1
## Training set -0.1324105
```

```r
summary(gdp_m2)
```

```
## Series: log_gdp
## ARIMA(0,1,2)
##
## Coefficients:
##          ma1     ma2
##       0.0827  0.1508
## s.e.  0.1416  0.1327
##
## sigma^2 = 0.01889:  log likelihood = 33.24
## AIC=-60.48   AICc=-60.03   BIC=-54.35
##
## Training set error measures:
##                       ME      RMSE        MAE       MPE      MAPE      MASE
## Training set 0.04070831 0.1338413 0.09857835 0.2025987 0.4821589 0.9285919
##                      ACF1
## Training set -0.05876644
```

```r
summary(exports_m1)
```

```
## Series: log_exports
## ARIMA(0,1,2)
##
## Coefficients:
##          ma1     ma2
##       -0.4281  0.1207
## s.e.   0.1836  0.2222
##
## sigma^2 = 0.01515:  log likelihood = 39.46
## AIC=-72.92   AICc=-72.46   BIC=-66.79
##
```

```
## Training set error measures:
##                       ME      RMSE        MAE        MPE     MAPE      MASE
## Training set -0.01552691 0.1198489 0.08964931 -0.6554892 3.038706 0.8866345
##                     ACF1
## Training set 0.01643604
```

```
summary(exports_m2)
```

```
## Series: log_exports
## ARIMA(2,1,0)
##
## Coefficients:
##           ar1      ar2
##       -0.3816  -0.2829
## s.e.   0.1269   0.1255
##
## sigma^2 = 0.01437:  log likelihood = 40.92
## AIC=-75.84   AICc=-75.39   BIC=-69.71
##
## Training set error measures:
##                       ME      RMSE        MAE        MPE     MAPE      MASE
## Training set -0.01794603 0.1167409 0.08800061 -0.7426934 2.961359 0.8703288
##                     ACF1
## Training set 0.04665926
```

```
summary(exports_m3)
```

```
## Series: log_exports
## ARIMA(2,1,2)
##
## Coefficients:
##           ar1      ar2     ma1     ma2
##       -0.6924  -0.8006  0.3985  0.5571
## s.e.   0.1455   0.1879  0.2067  0.2985
##
## sigma^2 = 0.01354:  log likelihood = 43.49
## AIC=-76.98   AICc=-75.81   BIC=-66.77
##
## Training set error measures:
##                       ME      RMSE        MAE        MPE     MAPE      MASE
## Training set -0.01395682 0.1112344 0.08445742 -0.5805899 2.849815 0.8352866
##                      ACF1
## Training set -0.03795421
```

```
summary(population_m1)
```

```
## Series: log_population
## ARIMA(2,1,0)
##
## Coefficients:
##          ar1      ar2
##       1.8788  -0.8846
```

```
## s.e.  0.0561   0.0566
##
## sigma^2 = 4.275e-06:  log likelihood = 324.38
## AIC=-642.77   AICc=-642.32   BIC=-636.64
##
## Training set error measures:
##                        ME        RMSE          MAE         MPE        MAPE
## Training set 0.0003968695 0.002013452 0.0007831319 0.002742284 0.005319975
##                     MASE      ACF1
## Training set 0.03946759 0.1138815
```

```
summary(population_m2)
```

```
## Series: log_population
## ARIMA(1,1,0)
##
## Coefficients:
##          ar1
##       0.9954
## s.e.  0.0054
##
## sigma^2 = 6.199e-06:  log likelihood = 283.93
## AIC=-563.87   AICc=-563.65   BIC=-559.78
##
## Training set error measures:
##                        ME        RMSE         MAE         MPE       MAPE
## Training set 0.0003061378 0.002446392 0.001494217 0.002199244 0.01007951
##                     MASE      ACF1
## Training set 0.07530424 0.4145039
```

Analysis: GDP

ARIMA(1,1,0)

Coefficients: ar1 0.1642 s.e. 0.1306

sigma^2 = 0.01885: log likelihood = 32.8 AIC=-61.61 AICc=-61.38 BIC=-57.52

Exports Series: log_exports ARIMA(2,1,0)

Coefficients: ar1 ar2 -0.3816 -0.2829 s.e. 0.1269 0.1255

sigma^2 = 0.01437: log likelihood = 40.92 AIC=-75.84 AICc=-75.39 BIC=-69.71

Population Series: log_population ARIMA(2,1,0)

Coefficients: ar1 ar2 1.8788 -0.8846 s.e. 0.0561 0.0566

sigma^2 = 4.275e-06: log likelihood = 324.38 AIC=-642.77 AICc=-642.32 BIC=-636.64

Coeffs:

```
get_arima_coefs <- function(model) {
  if (is.null(model$var.coef)) stop("Model does not have variance-covariance matrix")
  coefs <- coef(model)
  se <- sqrt(diag(model$var.coef))
  tval <- coefs / se
  pval <- 2 * (1 - pnorm(abs(tval)))
```

```
  data.frame(
    Estimate = coefs,
    SE = se,
    t.value = tval,
    p.value = pval,
    row.names = names(coefs)
  )
}

# Example for GDP
get_arima_coefs(gdp_m1)
```

```
##      Estimate        SE t.value   p.value
## ar1 0.1641636 0.1305829 1.25716 0.2086958
```

```
get_arima_coefs(exports_m2)
```

```
##       Estimate        SE   t.value     p.value
## ar1 -0.3815858 0.1268795 -3.007467 0.002634349
## ar2 -0.2829136 0.1254718 -2.254798 0.024146001
```

```
get_arima_coefs(population_m1)
```

```
##       Estimate         SE   t.value p.value
## ar1  1.8788207 0.05613192  33.47152       0
## ar2 -0.8846124 0.05656786 -15.63808       0
```

Model Diagnostics (Checking residual)

```
library(astsa)
```

```
## Warning: package 'astsa' was built under R version 4.3.3
```

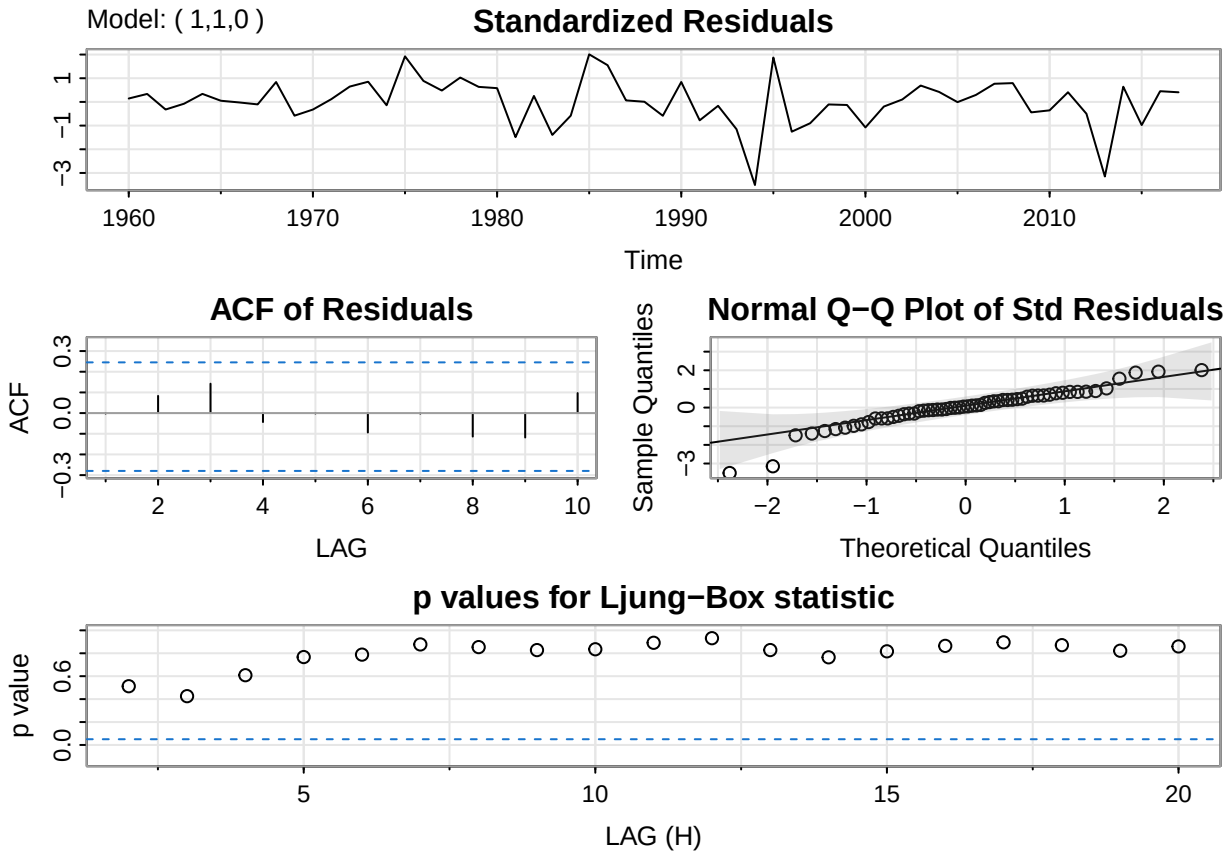
```
##
## Attaching package: 'astsa'
```

```
## The following object is masked _by_ '.GlobalEnv':
##
##     gdp
```

```
## The following object is masked from 'package:forecast':
##
##     gas
```

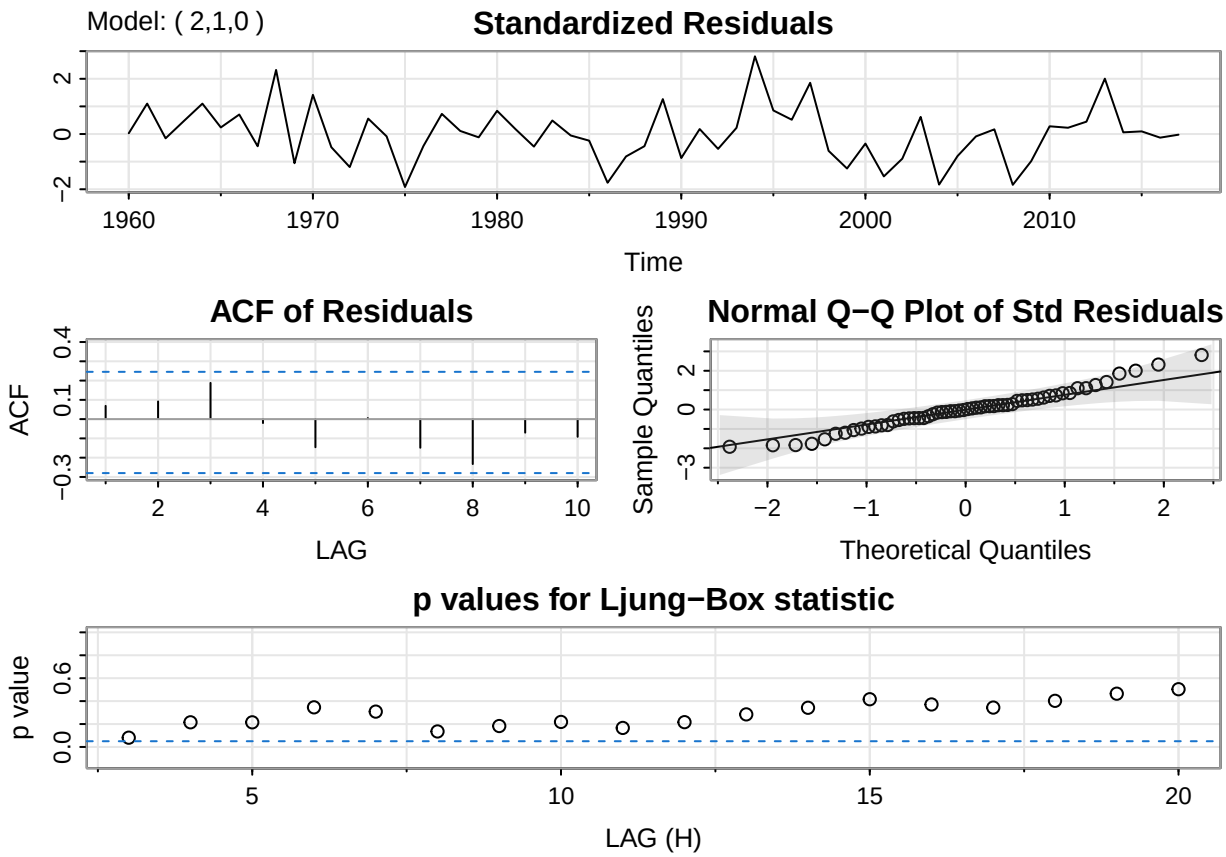
```
#GDP:
arima_110 <- sarima(log(gdp), 1,1,0)
```

```
## initial  value -2.043604
## iter   2 value -2.044628
## iter   3 value -2.044635
## iter   4 value -2.044638
## iter   4 value -2.044638
## iter   4 value -2.044638
## final  value -2.044638
## converged
## initial  value -2.052438
## iter   2 value -2.052450
## iter   3 value -2.052458
## iter   3 value -2.052458
## iter   3 value -2.052458
## final  value -2.052458
## converged
## <><><><><><><><><><><><><><>
##
## Coefficients:
##          Estimate     SE t.value p.value
## ar1        0.0444 0.1315  0.3380  0.7367
## constant   0.0502 0.0178  2.8207  0.0067
##
## sigma^2 estimated as 0.01649082 on 55 degrees of freedom
##
## AIC = -1.161776  AICc = -1.157878  BIC = -1.054247
##
```



```r
#Exports:
arima_210_exports <- sarima(log(exports), 2,1,0)
```

```
## initial  value -2.046549
## iter   2 value -2.136489
## iter   3 value -2.140824
## iter   4 value -2.143878
## iter   5 value -2.145964
## iter   6 value -2.146586
## iter   7 value -2.146587
## iter   8 value -2.146587
## iter   8 value -2.146587
## final  value -2.146587
## converged
## initial  value -2.150777
## iter   2 value -2.150816
## iter   3 value -2.150825
## iter   4 value -2.150826
## iter   5 value -2.150826
## iter   5 value -2.150826
## iter   5 value -2.150826
## final  value -2.150826
## converged
## <><><><><><><><><><><><><><>
##
## Coefficients:
##          Estimate     SE t.value p.value
## ar1       -0.4035 0.1263 -3.1957  0.0023
## ar2       -0.3037 0.1247 -2.4351  0.0182
## constant  -0.0117 0.0091 -1.2895  0.2027
##
## sigma^2 estimated as 0.01347641 on 54 degrees of freedom
##
## AIC = -1.323425  AICc = -1.31548  BIC = -1.180053
##
```

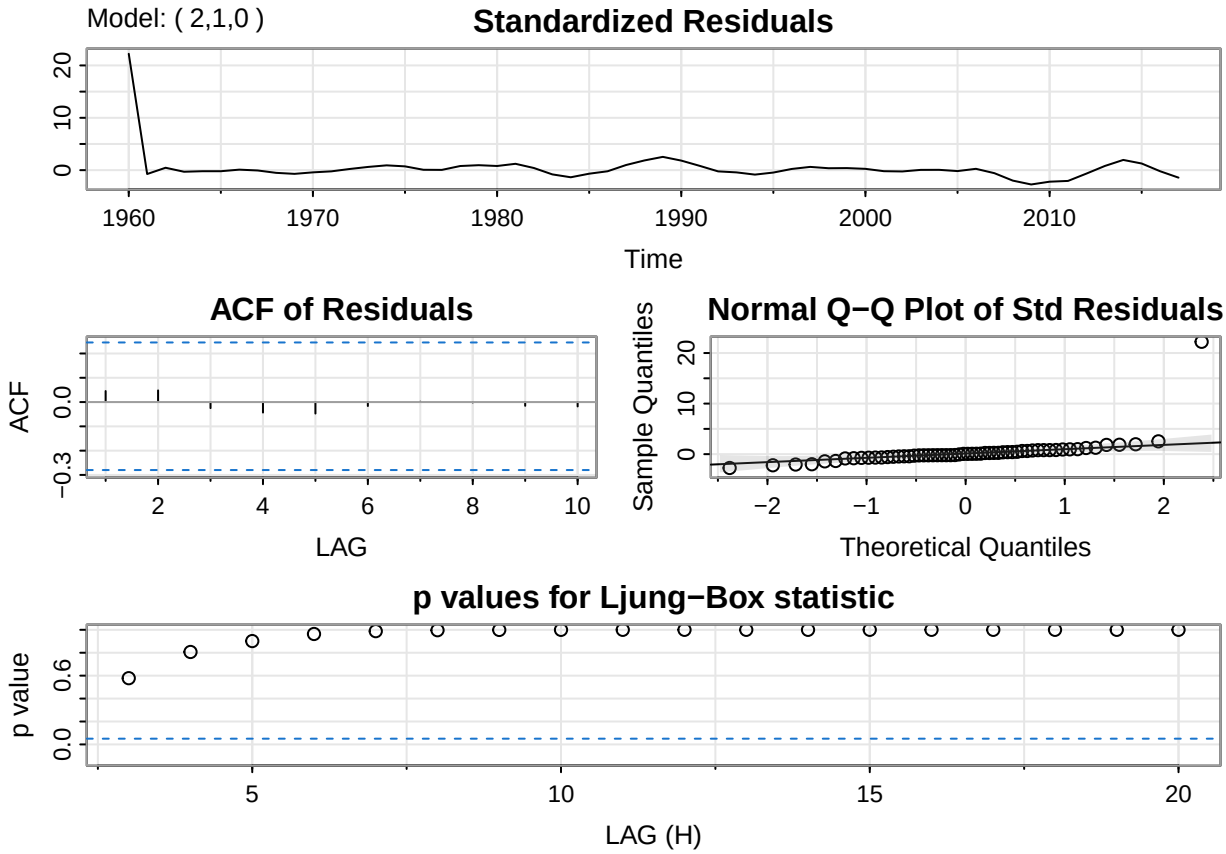


Exports: AR(1) (-0.4035) p-value is 0.0023 so that suggests that coefficient is significant AR(2) (-0.3037) p-value is 0.0182 so that suggests that coefficient is significant constant (-0.0117) p-value is 0.2027, so not significant

```
#Population:
arima_210_pop <- sarima(log(population), 2,1,0)
```

```
## initial  value -5.030781
## iter   2 value -5.114043
## iter   3 value -6.094349
## iter   4 value -6.144149
## iter   5 value -6.557846
## iter   6 value -7.233697
## iter   7 value -7.273859
## iter   8 value -7.332591
## iter   9 value -7.333733
## iter  10 value -7.343975
## iter  11 value -7.348005
## iter  12 value -7.348103
## iter  13 value -7.348105
## iter  13 value -7.348105
## iter  13 value -7.348105
## final  value -7.348105
## converged
## initial  value -7.289435
## iter   2 value -7.290530
## iter   3 value -7.293845
```

```
## iter   4 value -7.296074
## iter   5 value -7.296795
## iter   6 value -7.296905
## iter   7 value -7.296906
## iter   7 value -7.296906
## iter   7 value -7.296906
## final  value -7.296906
## converged
## <><><><><><><><><><><><><><><>
##
## Coefficients:
##          Estimate     SE  t.value p.value
## ar1        1.8622 0.0443  42.0719       0
## ar2       -0.9296 0.0444 -20.9352       0
## constant   0.0204 0.0013  16.0619       0
##
## sigma^2 estimated as 4.084694e-07 on 54 degrees of freedom
##
## AIC = -11.61558  AICc = -11.60764  BIC = -11.47221
##
```



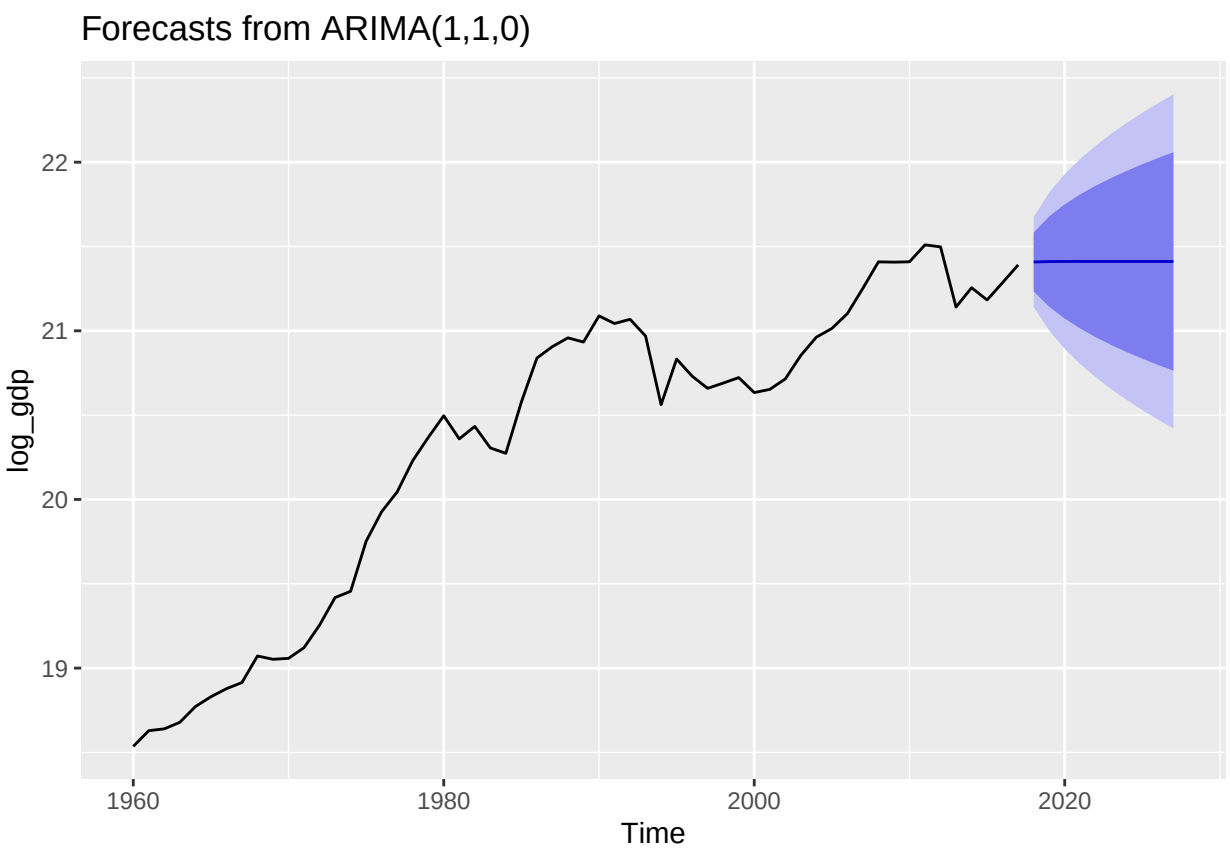
Population: AR(1) (1.8622) p-value is 0 so that suggests that coefficient is significant AR(2) (-0.9296) p-value is 0 so that suggests that coefficient is significant constant (0.0204) p-value is 0, so significant

The final models- GDP: ARIMA(1,1,0) - m1 Exports: ARIMA(2,1,0) - m2 Population: ARIMA(2,1,0) - m1

Forecasts:

```
#change to arima
log_gdp_arima110 <- arima(log_gdp, order= c(1,1,0))
log_exports_arima210 <- arima(log_exports, order= c(2,1,0))
log_population_arima210 <- arima(log_population, order= c(2,1,0))
```

```
library(forecast)
forecast_gdp <- forecast(log_gdp_arima110, h=2, level = 95)
autoplot(forecast(log_gdp_arima110))
```




```
forecast_gdp$mean
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
## [1] 21.40800 21.41082
```

```
forecast_gdp$lower
```

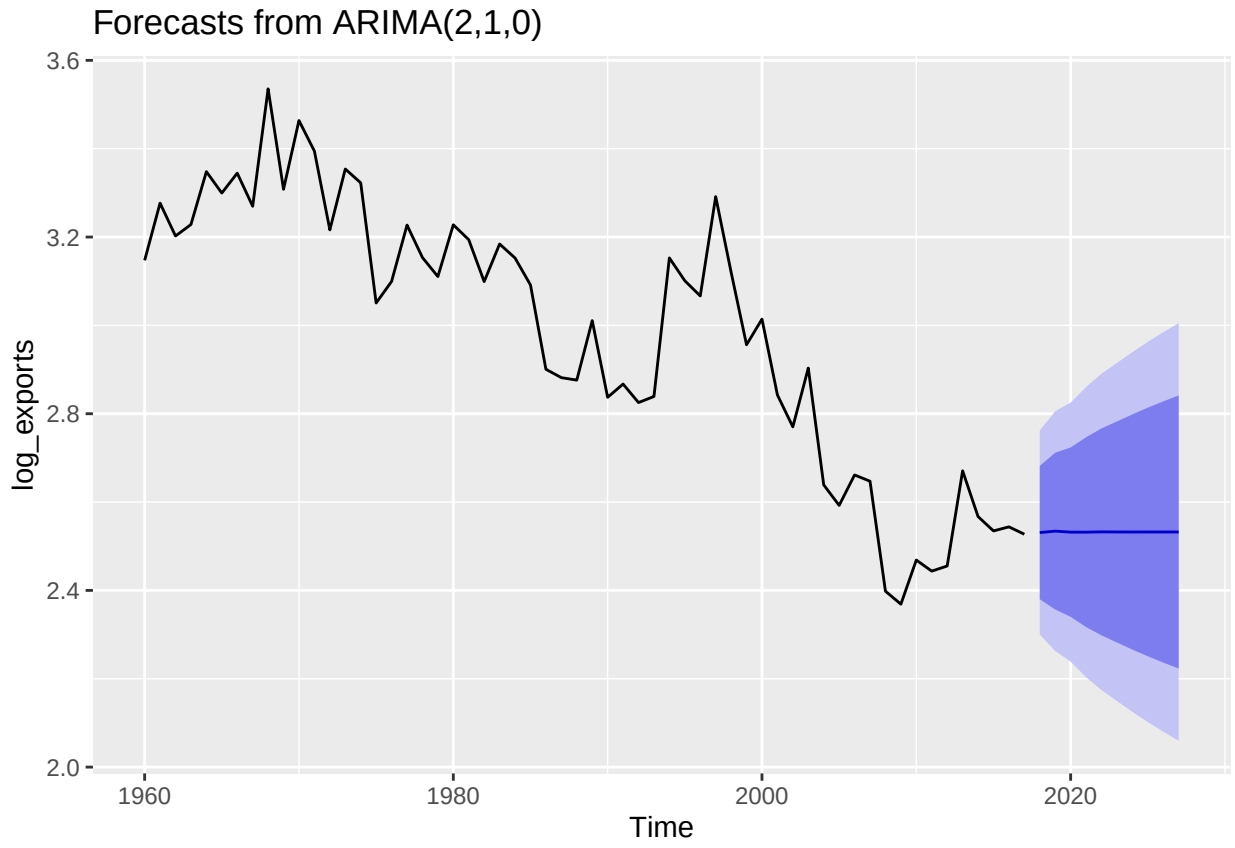
```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
```

```
##           95%
## [1,] 21.14132
## [2,] 21.00156
```

```
forecast_gdp$upper
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##           95%
## [1,] 21.67467
## [2,] 21.82008
```

```
forecast_exports <- forecast(log_exports_arima210, h=2, level = 95)
autoplot(forecast(log_exports_arima210))
```




```
forecast_exports$mean
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
## [1] 2.530936 2.534229
```

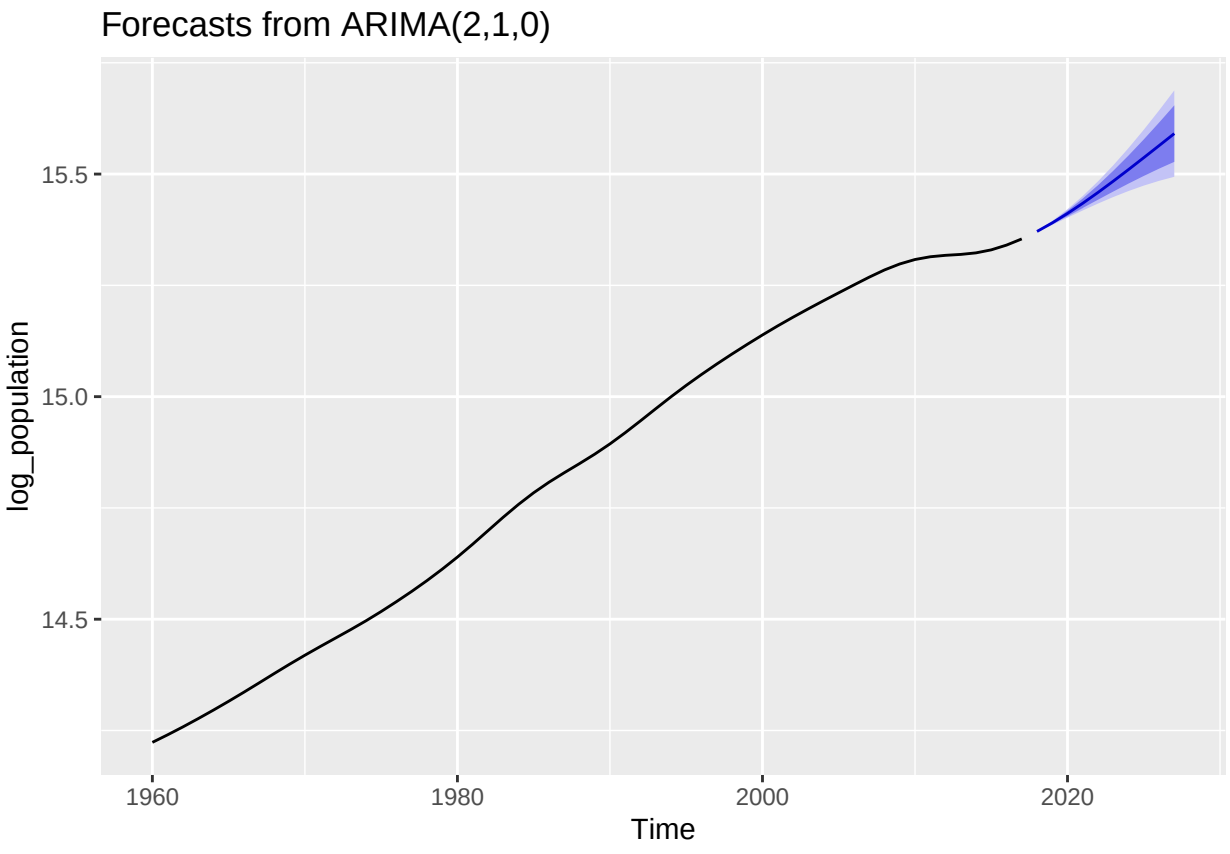
```
forecast_exports$lower
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##           95%
## [1,] 2.300132
## [2,] 2.262855
```

```
forecast_exports$upper
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##           95%
## [1,] 2.761741
## [2,] 2.805602
```

```
forecast_population <- forecast(log_population_arima210, h=2, level = 95)
autoplot(forecast(log_population_arima210))
```



```
forecast_population$mean
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
## [1] 15.37111 15.39032
```

```
forecast_population$lower
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##           95%
## [1,] 15.36962
## [2,] 15.38578
```

```
forecast_population$upper
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##           95%
## [1,] 15.37260
## [2,] 15.39487
```

Back transformed:

```
library(ggplot2)

gdp_mean_back  <- exp(forecast_gdp$mean)
gdp_mean_back
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
## [1] 1983237590 1988846394
```

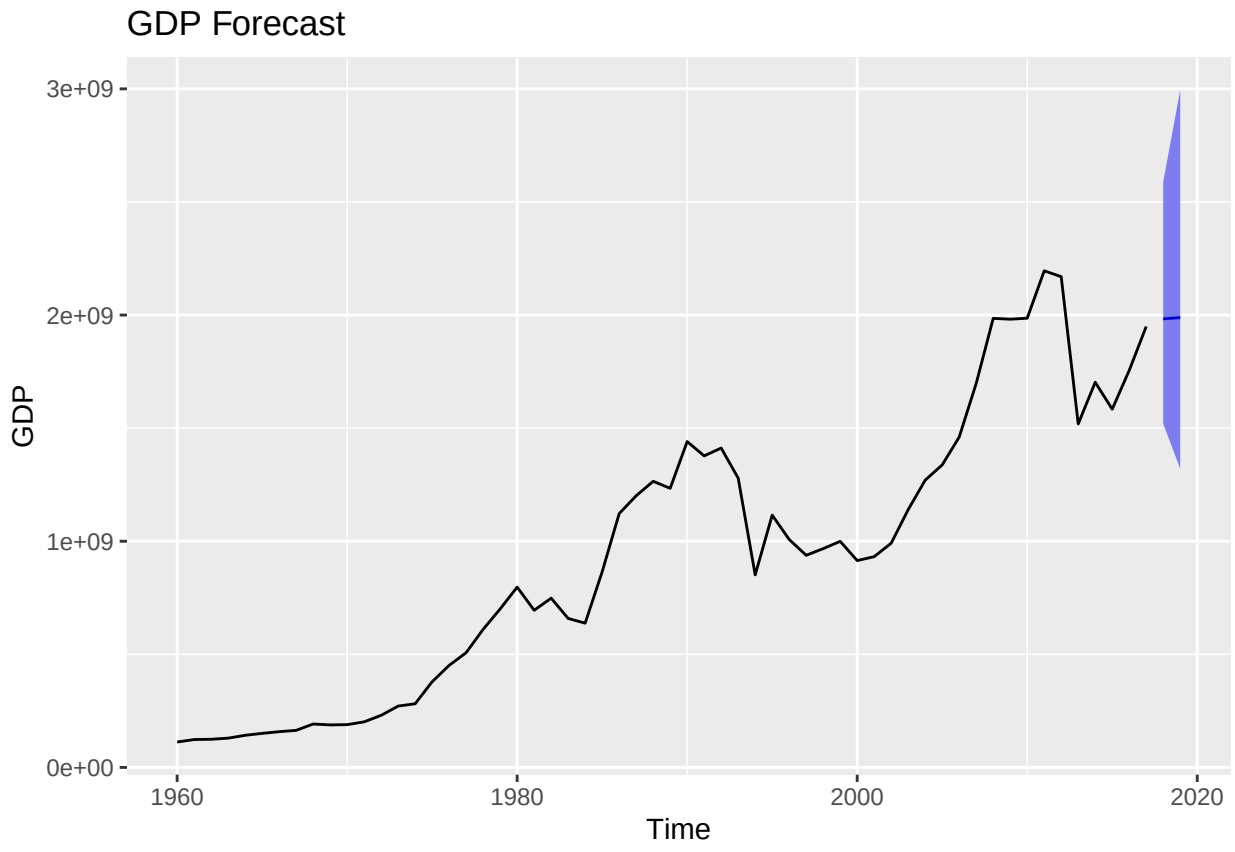
```
gdp_lower_back <- exp(forecast_gdp$lower)
gdp_lower_back
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##             95%
## [1,] 1519009254
## [2,] 1320875915
```

```
gdp_upper_back <- exp(forecast_gdp$upper)
gdp_upper_back
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##             95%
## [1,] 2589339944
## [2,] 2994611328
```

```
forecast_gdp$mean = exp(forecast_gdp$mean)
forecast_gdp$lower = exp(forecast_gdp$lower)
forecast_gdp$upper = exp(forecast_gdp$upper)
forecast_gdp$x = exp(forecast_gdp$x)
autoplot(forecast_gdp) +
  ggtitle("GDP Forecast") +
  xlab("Time") +
  ylab("GDP")
```




```
forecast_gdp$mean
```

```
## Time Series:
## Start = 2018
```

```
## End = 2019
## Frequency = 1
## [1] 1983237590 1988846394
```

```
forecast_gdp$lower
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##             95%
## [1,] 1519009254
## [2,] 1320875915
```

```
forecast_gdp$upper
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##             95%
## [1,] 2589339944
## [2,] 2994611328
```

```
exports_mean_back  <- exp(forecast_exports$mean)
exports_mean_back
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
## [1] 12.56527 12.60671
```

```
exports_lower_back <- exp(forecast_exports$lower)
exports_lower_back
```

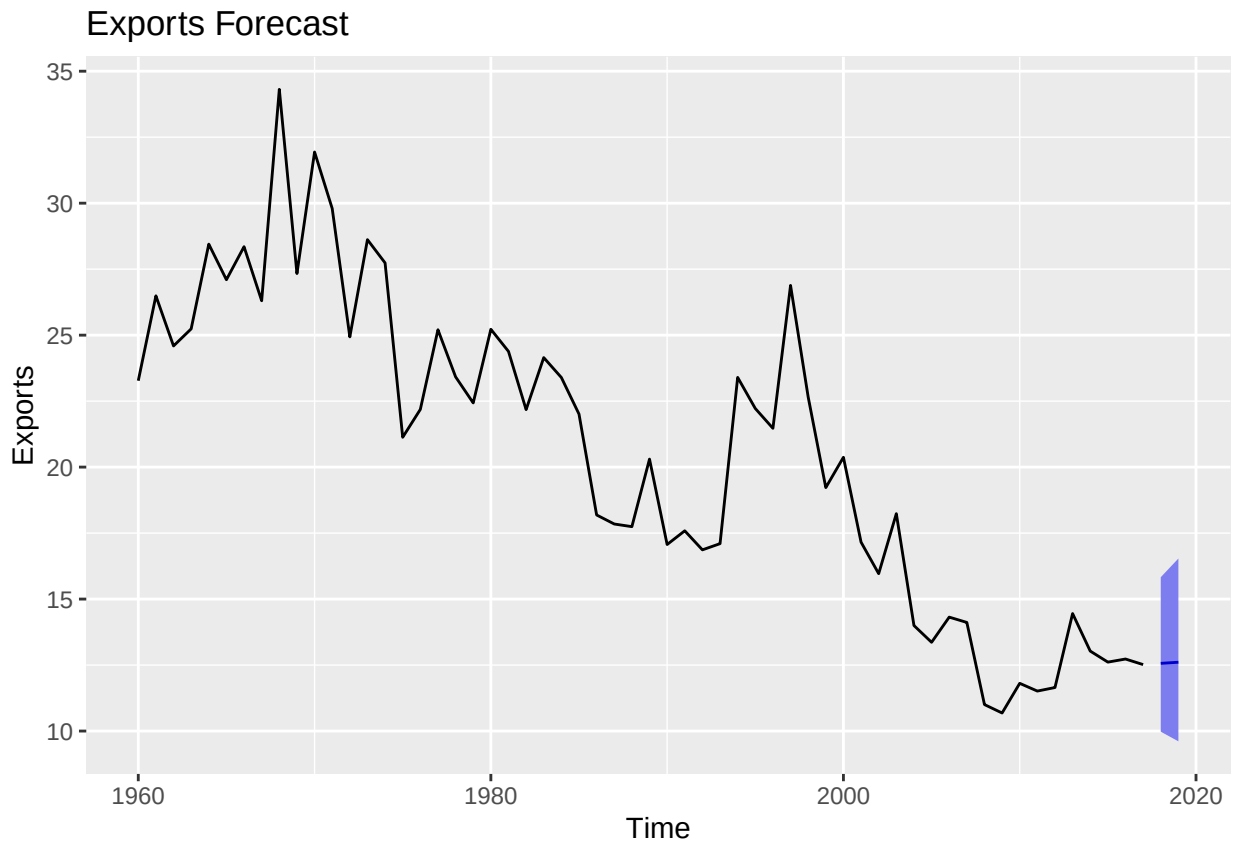
```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##           95%
## [1,] 9.975494
## [2,] 9.610491
```

```
exports_upper_back <- exp(forecast_exports$upper)
exports_upper_back
```

```
## Time Series:
## Start = 2018
## End = 2019
```

```
## Frequency = 1
##           95%
## [1,] 15.82738
## [2,] 16.53704
```

```
forecast_exports$mean = exp(forecast_exports$mean)
forecast_exports$lower = exp(forecast_exports$lower)
forecast_exports$upper = exp(forecast_exports$upper)
forecast_exports$x = exp(forecast_exports$x)
autoplot(forecast_exports) +
  ggtitle("Exports Forecast") +
  xlab("Time") +
  ylab("Exports")
```




```
forecast_exports$mean
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
## [1] 12.56527 12.60671
```

```
forecast_exports$lower
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##           95%
## [1,] 9.975494
## [2,] 9.610491
```

```
forecast_exports$upper
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##           95%
## [1,] 15.82738
## [2,] 16.53704
```

```
population_mean_back  <- exp(forecast_population$mean)
population_mean_back
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
## [1] 4737936 4829829
```

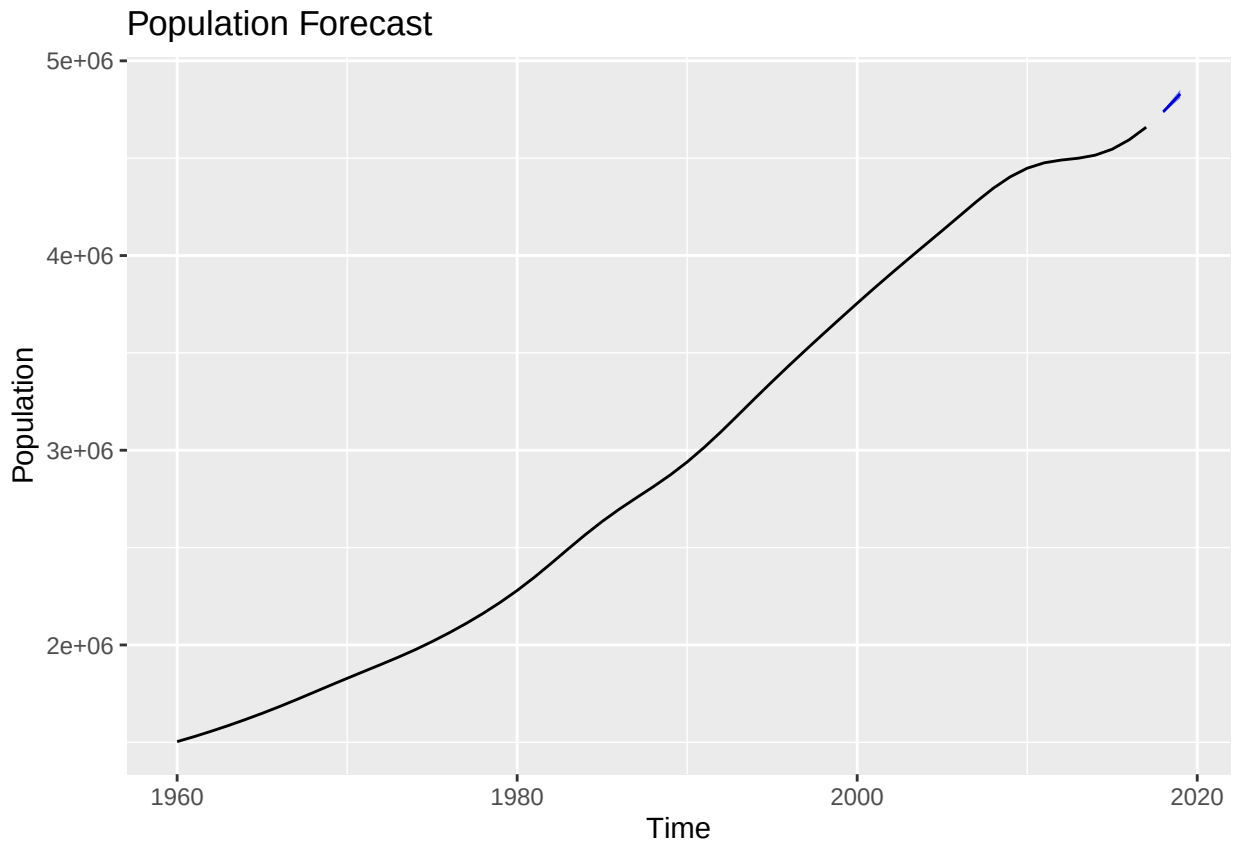
```
population_lower_back <- exp(forecast_population$lower)
population_lower_back
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##          95%
## [1,] 4730878
## [2,] 4807935
```

```
population_upper_back <- exp(forecast_population$upper)
population_upper_back
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##          95%
## [1,] 4745005
## [2,] 4851823
```

```r
forecast_population$mean = exp(forecast_population$mean)
forecast_population$lower = exp(forecast_population$lower)
forecast_population$upper = exp(forecast_population$upper)
forecast_population$x = exp(forecast_population$x)
autoplot(forecast_population) +
  ggtitle("Population Forecast") +
  xlab("Time") +
  ylab("Population")
```




```r
forecast_population$mean
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
## [1] 4737936 4829829
```

```r
forecast_population$lower
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##          95%
```

```
## [1,] 4730878
## [2,] 4807935
```

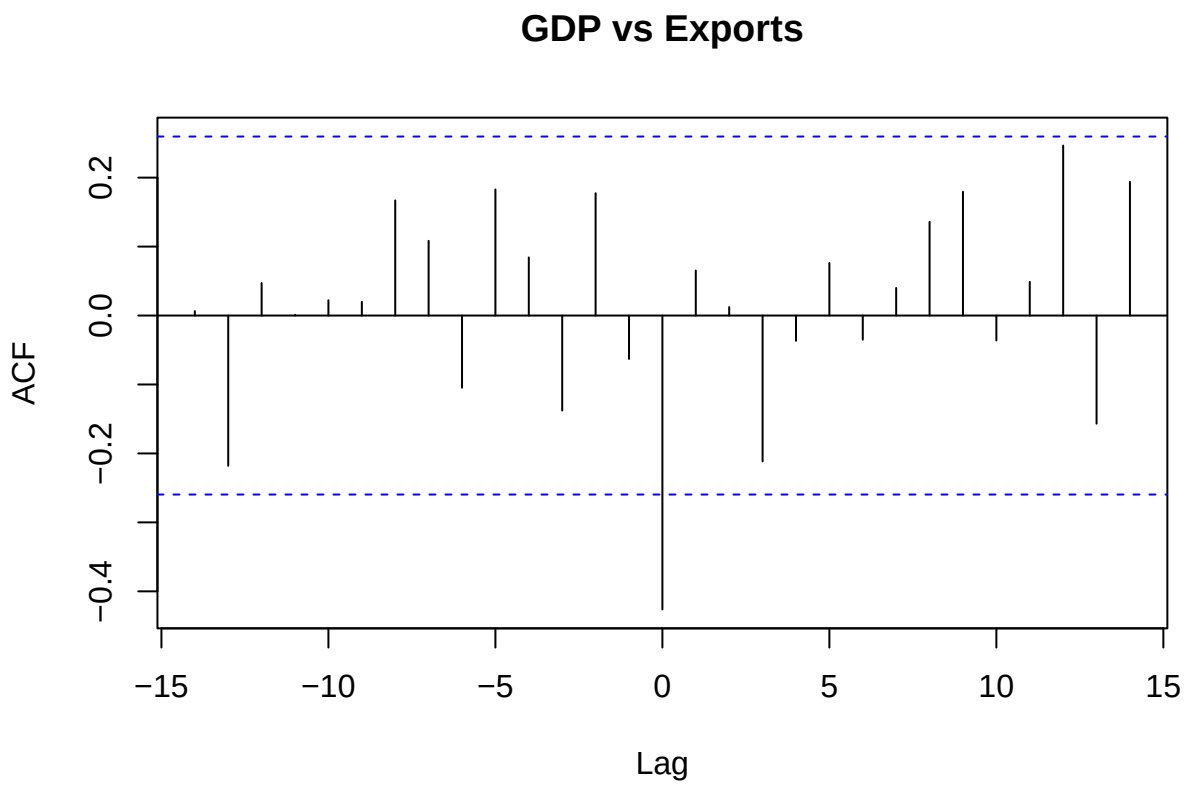
```
forecast_population$upper
```

```
## Time Series:
## Start = 2018
## End = 2019
## Frequency = 1
##          95%
## [1,] 4745005
## [2,] 4851823
```

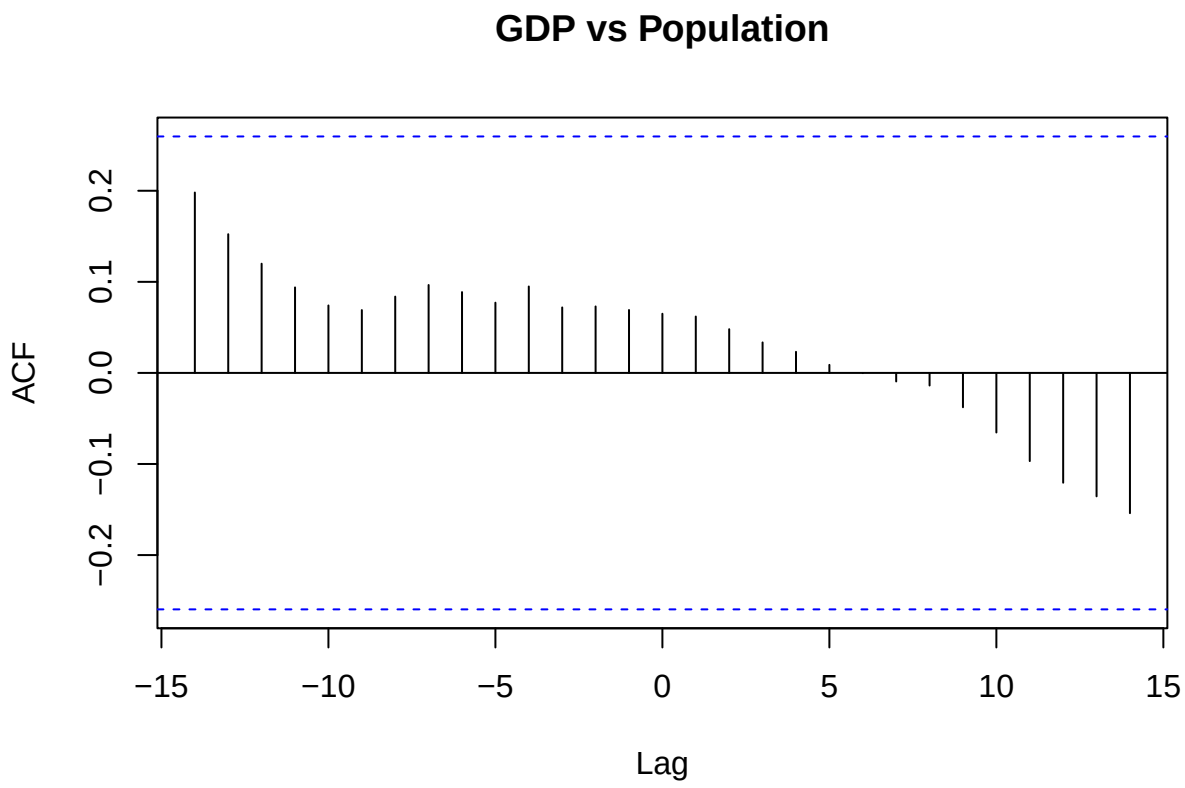
Comparitive Analysis/Joint Analysis: Variables don't move on their own independently and usually move together, so we analyze the cross correlation. Earlier we only analyzed GDP, Exports, and Population separately, so we do joint analysis to see how these variables interact with each other.Cross-correlation is how you quantify relationships between economic variables over time.

Cross-Correlation:

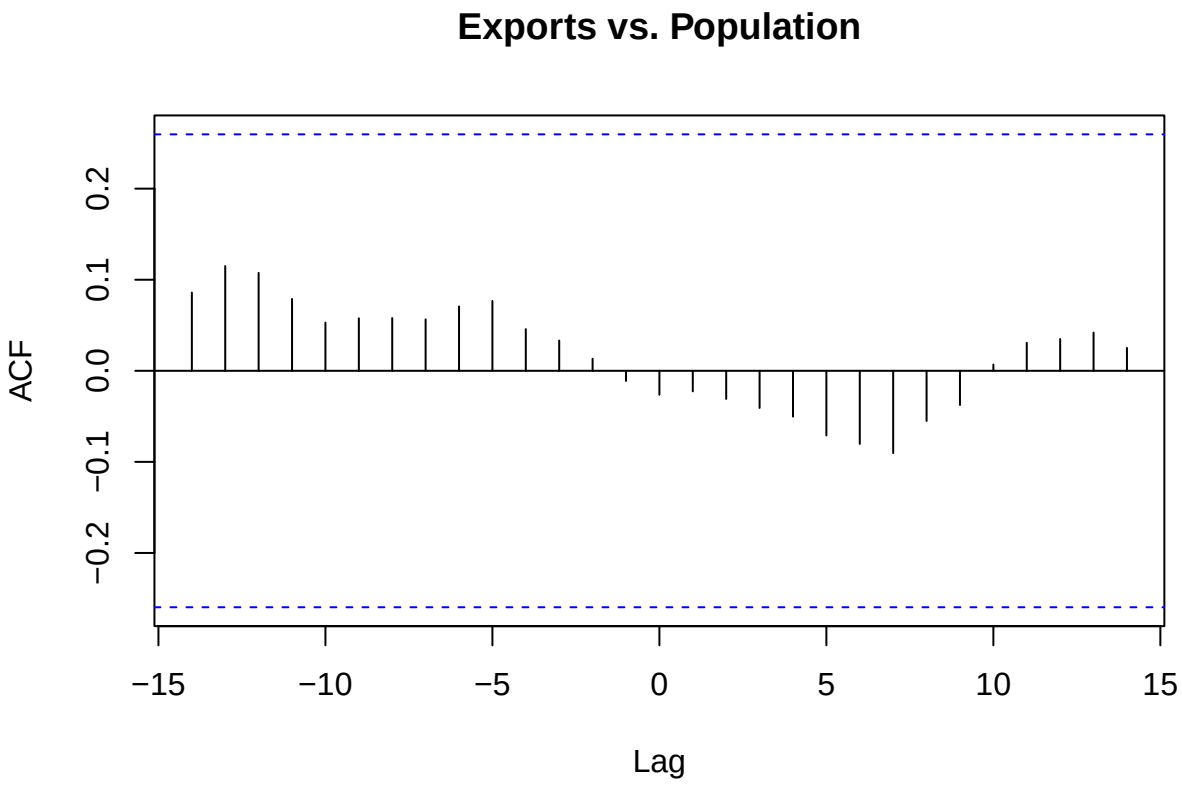
```
#GDP vs. Exports
ccf(log_gdp_diff1, log_exports_diff1, main = "GDP vs Exports")
```



```r
#GDP vs. Population
ccf(log_gdp_diff1, log_population_diff1, main = "GDP vs Population")
```




```r
#Exports vs. Population
ccf(log_exports_diff1, log_population_diff1, main = "Exports vs. Population")
```



GDP vs. Exports Analysis: Peak at lag 0 meaning that GDP and Exports move together, meaning that when exports increase, GDP tends to increase in the same year.

GDP vs. Population Analysis: Since there are no peaks above the dashed line, this means there is no statistically significant correlation at any lag, CHangs in population growth doesn't directly lead or lag GDP changes in this dataset. In the Central African Republic, GDP fluctuations seem to be driven by other factors (like exports) rather than population changes.

Exports vs. Population Analysis: There are no peaks above the dashed lines, meaning there is no statistically significant correlation at any lag. Changes in population growth do not appear to lead or lag changes in exports. Export performance seems independent of population growth over this period; other factors (like global demand, production capacity, or policy) likely drive exports more than demographic changes. Both GDP vs Population and Exports vs Population show no significant cross-correlation. This suggests that population growth is not a strong driver of either economic output or exports in the Central African Republic for this dataset.

Multivariate Analysis: We can capture dynamic interactions between variables. We can see how one variable affects others over time. Vector Autoregression (VAR) model.

```
library(vars)
```

```
## Warning: package 'vars' was built under R version 4.3.3
```

```
## Loading required package: MASS
```

```
## Loading required package: strucchange
```

```
## Warning: package 'strucchange' was built under R version 4.3.3
```

```
## Loading required package: zoo

##
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric

## Loading required package: sandwich

## Loading required package: urca

## Warning: package 'urca' was built under R version 4.3.3

## Loading required package: lmtest
```

```
multi_ts <- cbind(log_gdp_diff2, log_exports_diff2, log_population_diff2)
colnames(multi_ts) <- c("GDP", "Exports", "Population")

lag_selection <- VARselect(multi_ts, lag.max = 10, type = "const")
lag_selection$selection
```

```
## AIC(n)  HQ(n)  SC(n) FPE(n)
##     10      2      2      2
```

Pick the lag suggested by BIC for parsimonious model. Therefore 3.

VAR Model:

```
var_model <- VAR(multi_ts, p = 2 , type = "const")
summary(var_model)
```

```
##
## VAR Estimation Results:
## ========================
## Endogenous variables: GDP, Exports, Population
## Deterministic variables: const
## Sample size: 54
## Log Likelihood: 429.742
## Roots of the characteristic polynomial:
## 0.9866 0.9866 0.8005 0.8005 0.5729 0.5729
## Call:
## VAR(y = multi_ts, p = 2, type = "const")
##
##
## Estimation results for equation GDP:
## ====================================
## GDP = GDP.l1 + Exports.l1 + Population.l1 + GDP.l2 + Exports.l2 + Population.l2 + const
##
##                Estimate Std. Error t value Pr(>|t|)
```

```
## GDP.l1        -0.687424   0.152822  -4.498 4.48e-05 ***
## Exports.l1     0.089142   0.136014   0.655   0.5154
## Population.l1  2.757448  26.330165   0.105   0.9170
## GDP.l2        -0.293814   0.156752  -1.874   0.0671 .
## Exports.l2     0.163783   0.132307   1.238   0.2219
## Population.l2  3.132830  27.944917   0.112   0.9112
## const          0.002805   0.020715   0.135   0.8929
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.1507 on 47 degrees of freedom
## Multiple R-Squared: 0.4026,  Adjusted R-squared: 0.3263
## F-statistic: 5.279 on 6 and 47 DF,  p-value: 0.0003155
##
##
## Estimation results for equation Exports:
## ========================================
## Exports = GDP.l1 + Exports.l1 + Population.l1 + GDP.l2 + Exports.l2 + Population.l2 + const
##
##                 Estimate Std. Error t value Pr(>|t|)
## GDP.l1        -2.227e-01  1.393e-01  -1.599    0.117
## Exports.l1    -9.894e-01  1.240e-01  -7.982 2.71e-10 ***
## Population.l1 -2.310e+01  2.400e+01  -0.963    0.341
## GDP.l2        -7.156e-02  1.429e-01  -0.501    0.619
## Exports.l2    -6.775e-01  1.206e-01  -5.619 1.01e-06 ***
## Population.l2  2.181e+01  2.547e+01   0.856    0.396
## const         -8.303e-05  1.888e-02  -0.004    0.997
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.1373 on 47 degrees of freedom
## Multiple R-Squared: 0.6294,  Adjusted R-squared: 0.5821
## F-statistic:  13.3 on 6 and 47 DF,  p-value: 9.614e-09
##
##
## Estimation results for equation Population:
## ===========================================
## Population = GDP.l1 + Exports.l1 + Population.l1 + GDP.l2 + Exports.l2 + Population.l2 + const
##
##                 Estimate Std. Error t value Pr(>|t|)
## GDP.l1        -1.910e-04  3.360e-04  -0.568    0.572
## Exports.l1     1.572e-04  2.990e-04   0.526    0.602
## Population.l1  1.705e+00  5.789e-02  29.460   <2e-16 ***
## GDP.l2        -6.565e-05  3.446e-04  -0.190    0.850
## Exports.l2     1.743e-04  2.909e-04   0.599    0.552
## Population.l2 -9.745e-01  6.144e-02 -15.861   <2e-16 ***
## const         -4.213e-05  4.554e-05  -0.925    0.360
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.0003313 on 47 degrees of freedom
```

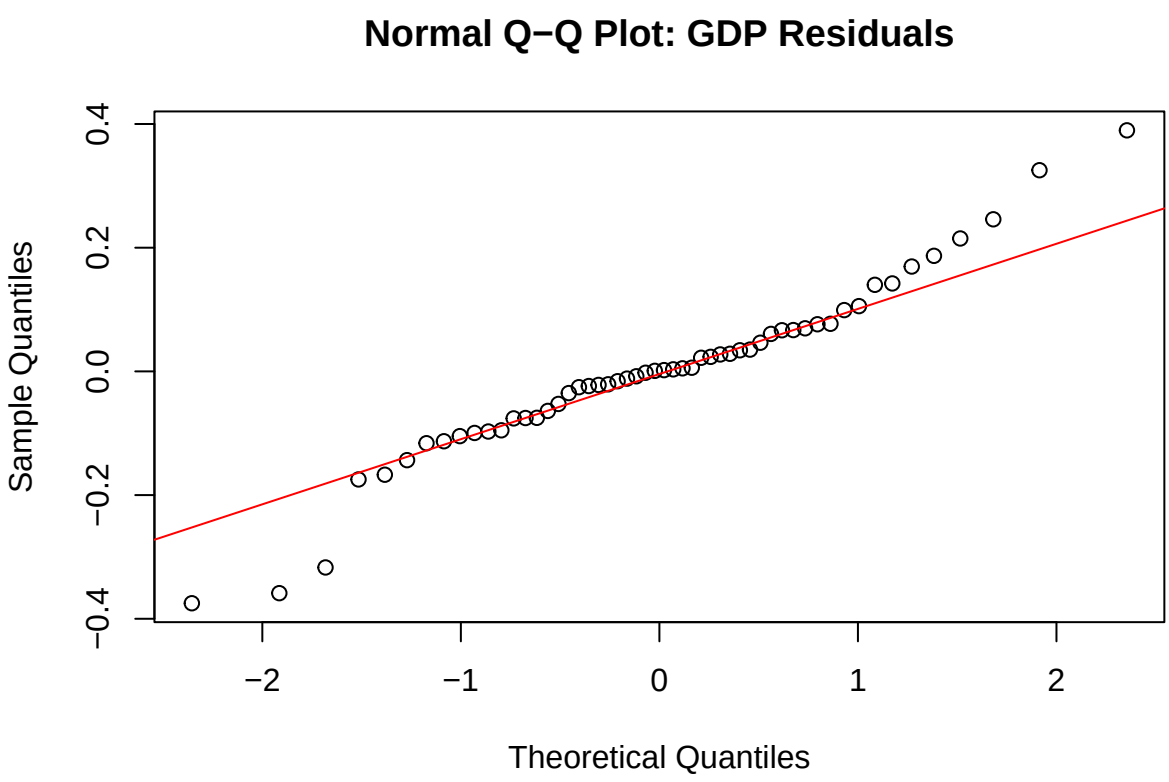
```
## Multiple R-Squared: 0.9636,  Adjusted R-squared: 0.9589
## F-statistic: 207.2 on 6 and 47 DF,  p-value: < 2.2e-16
##
##
##
## Covariance matrix of residuals:
##                    GDP    Exports Population
## GDP         2.271e-02 -8.615e-03 -5.795e-06
## Exports    -8.615e-03  1.886e-02  9.116e-06
## Population -5.795e-06  9.116e-06  1.098e-07
##
## Correlation matrix of residuals:
##                GDP Exports Population
## GDP         1.0000 -0.4163    -0.1161
## Exports    -0.4163  1.0000     0.2004
## Population -0.1161  0.2004     1.0000
```

Checking Residual

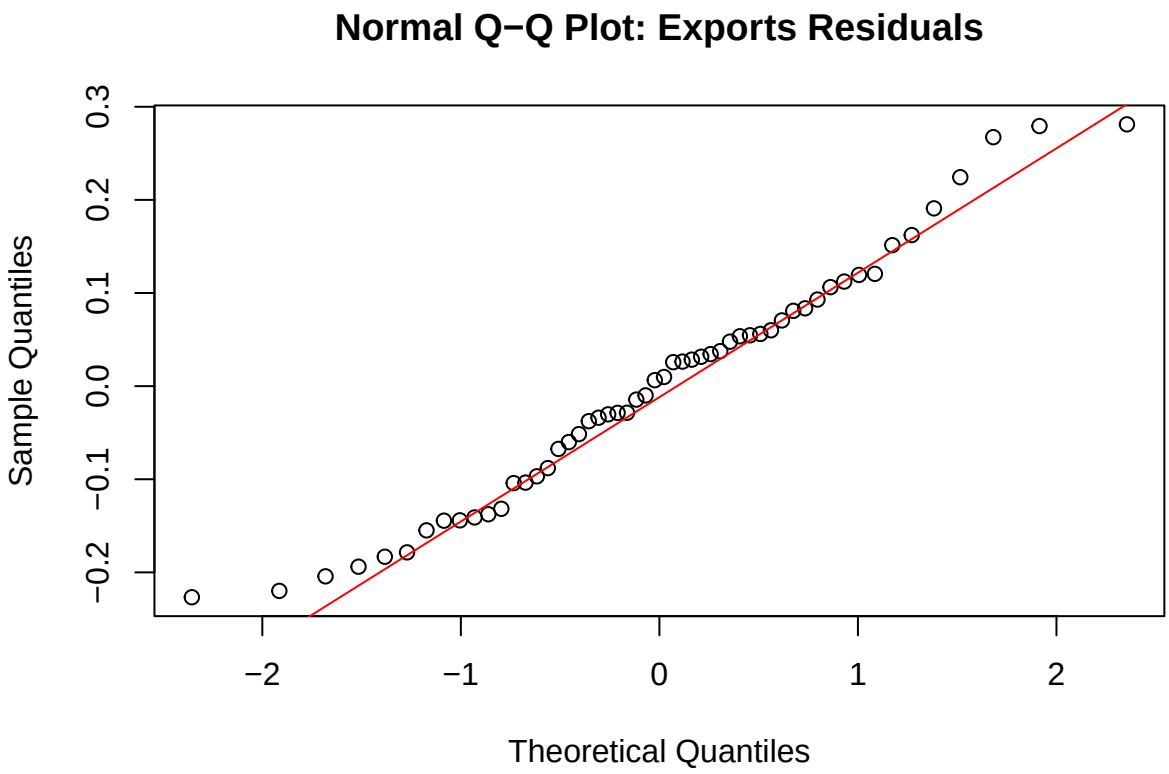
```
serial.test(var_model, lags.pt = 10, type = "PT.asymptotic")
```

```
##
##  Portmanteau Test (asymptotic)
##
## data:  Residuals of VAR object var_model
## Chi-squared = 69.213, df = 72, p-value = 0.5712
```

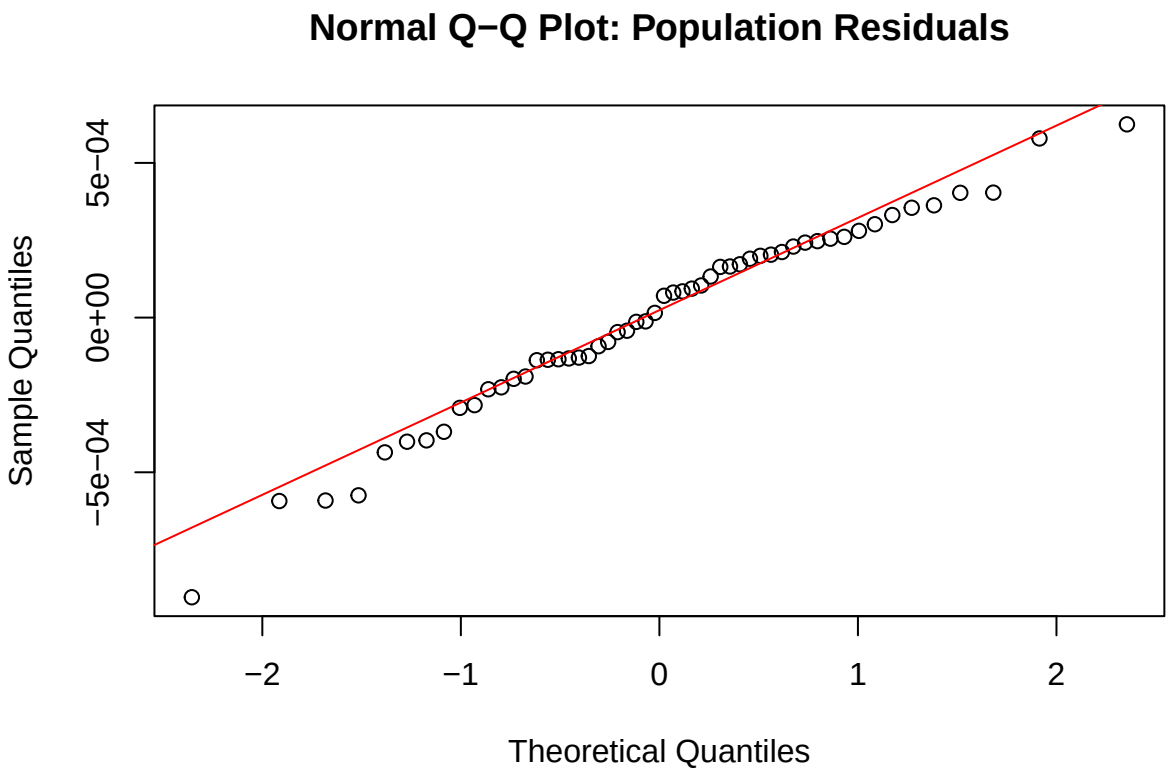
```
qqnorm(resid(var_model)[,1], main = "Normal Q-Q Plot: GDP Residuals")
qqline(resid(var_model)[,1], col = "red")
```



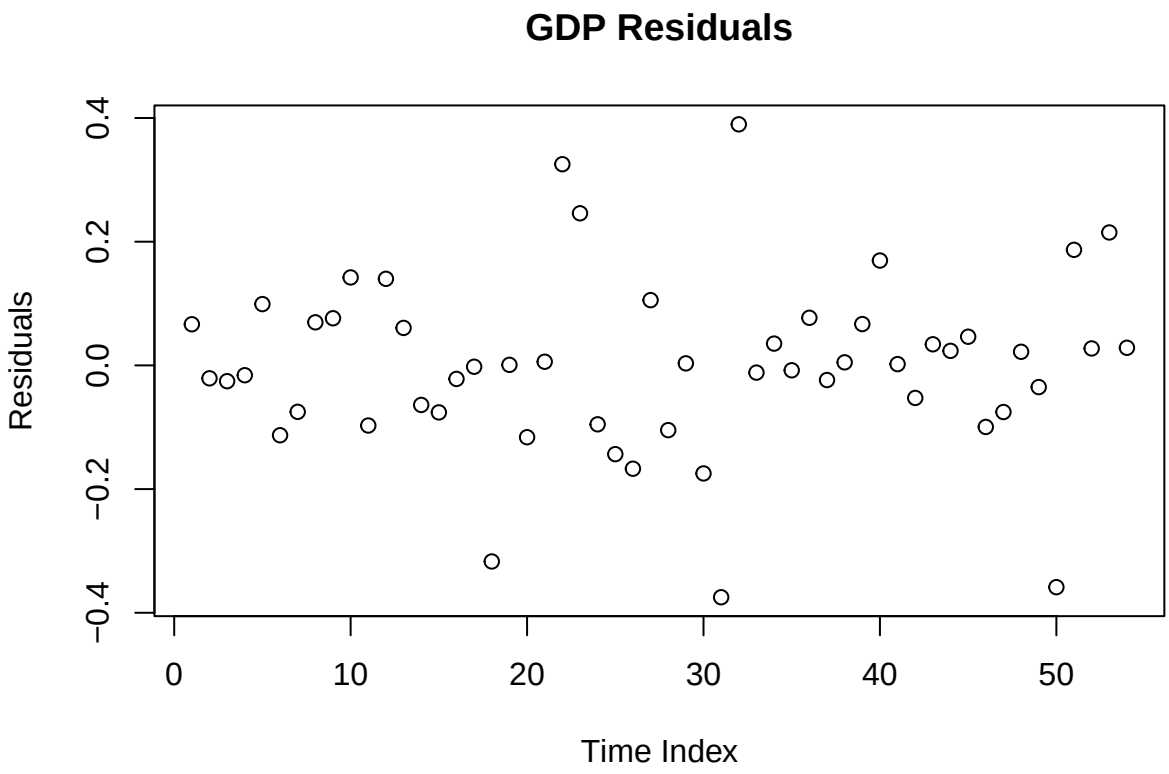

```r
qqnorm(resid(var_model)[,2], main = "Normal Q-Q Plot: Exports Residuals")
qqline(resid(var_model)[,2], col = "red")
```



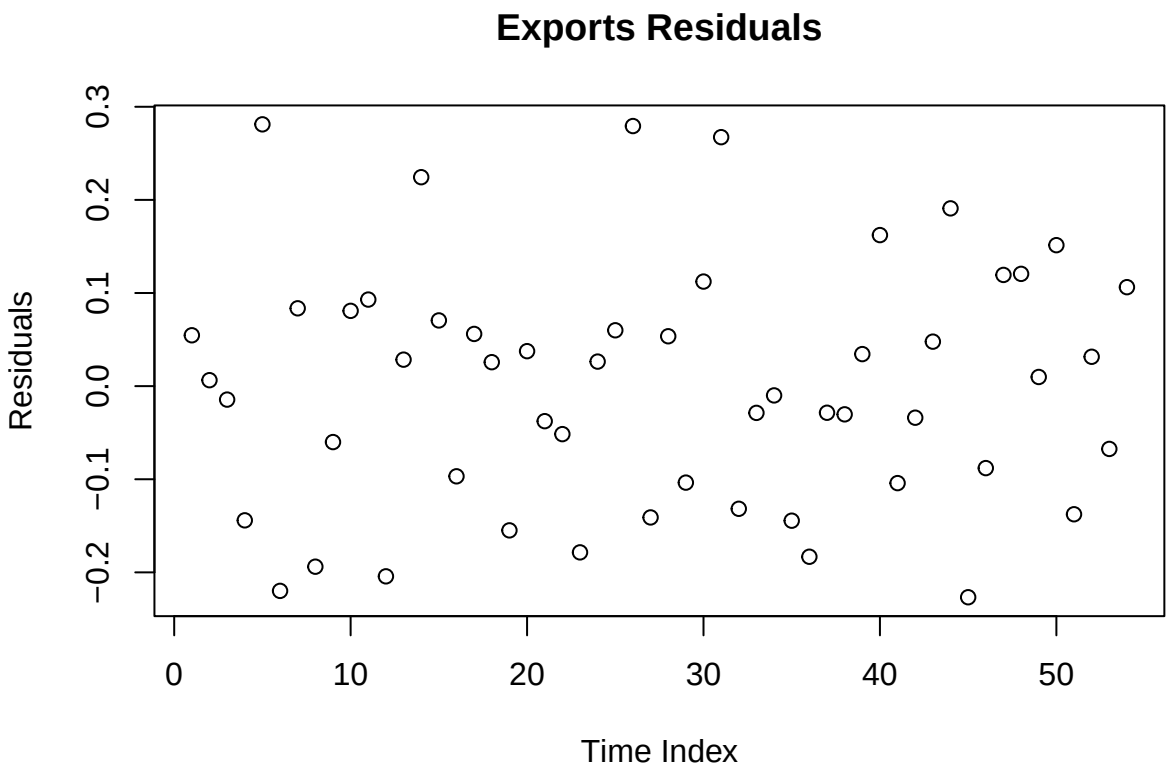

```
qqnorm(resid(var_model)[,3], main = "Normal Q-Q Plot: Population Residuals")
qqline(resid(var_model)[,3], col = "red")
```



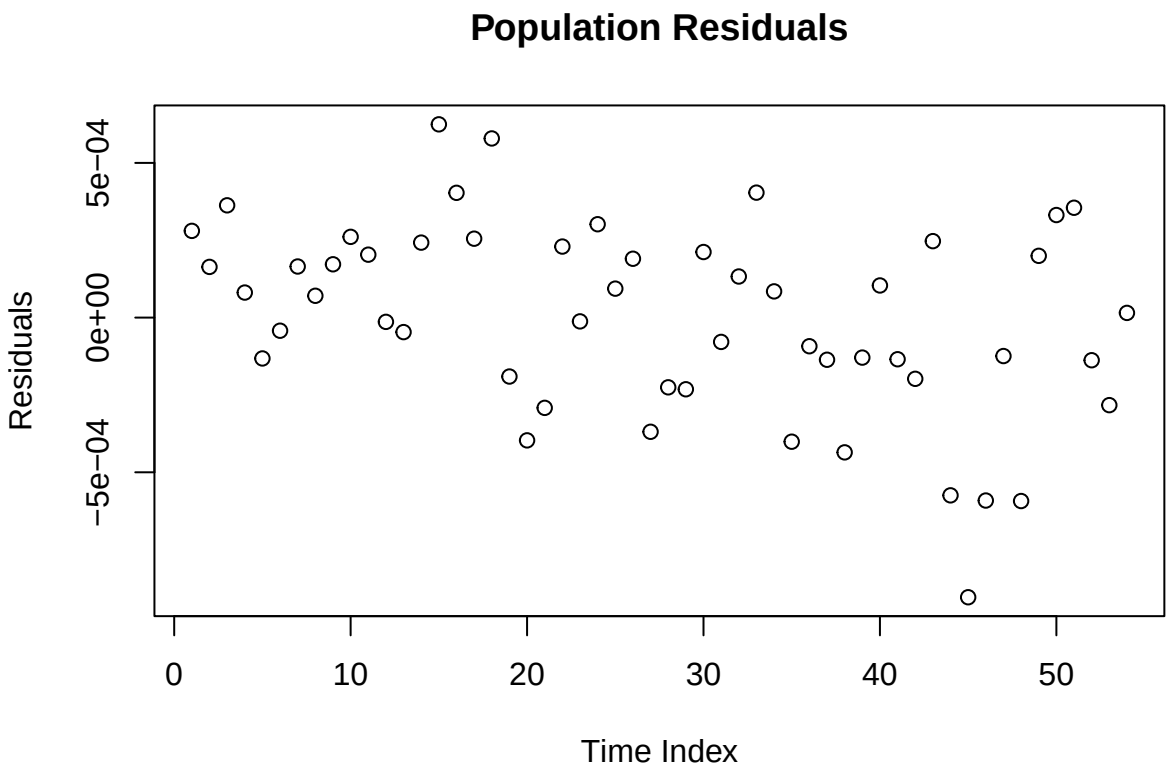

```r
plot(resid(var_model)[,1],
     main = "GDP Residuals",
     ylab = "Residuals",
     xlab = "Time Index")
```




```r
plot(resid(var_model)[,2],
     main = "Exports Residuals",
     ylab = "Residuals",
     xlab = "Time Index")
```




```
plot(resid(var_model)[,3],
     main = "Population Residuals",
     ylab = "Residuals",
     xlab = "Time Index")
```




```
normality.test(var_model)
```

```
## $JB
##
##  JB-Test (multivariate)
##
## data:  Residuals of VAR object var_model
## Chi-squared = 7.362, df = 6, p-value = 0.2887
##
##
## $Skewness
##
##  Skewness only (multivariate)
##
## data:  Residuals of VAR object var_model
## Chi-squared = 2.9838, df = 3, p-value = 0.3941
##
##
## $Kurtosis
##
##  Kurtosis only (multivariate)
##
## data:  Residuals of VAR object var_model
## Chi-squared = 4.3783, df = 3, p-value = 0.2234
```

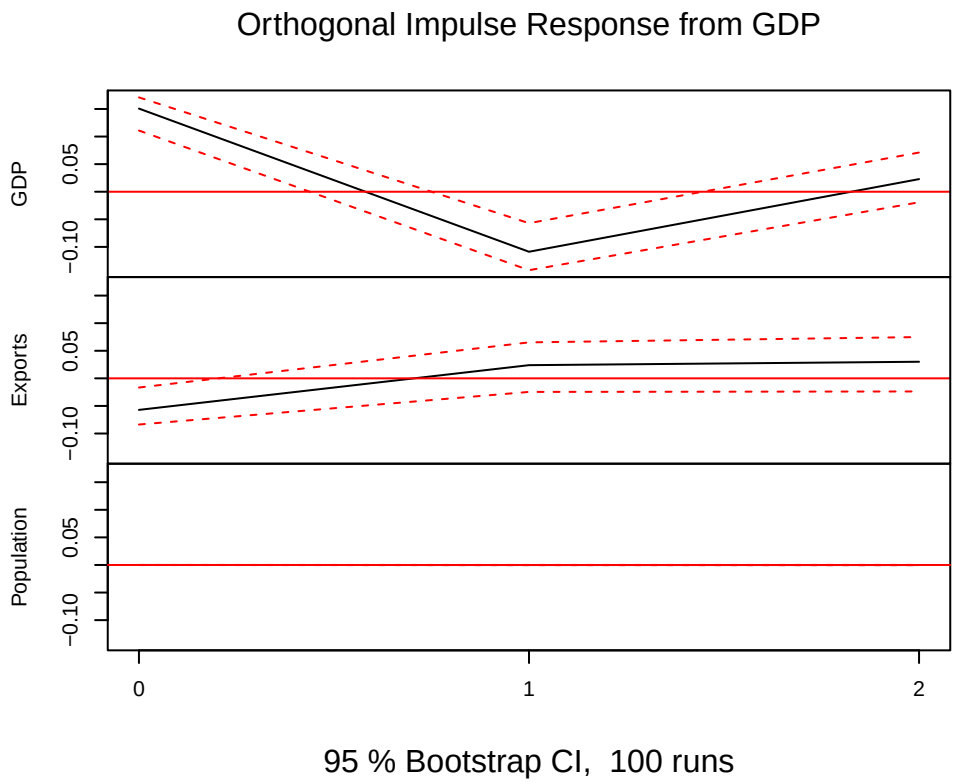
```
arch.test(var_model)
```

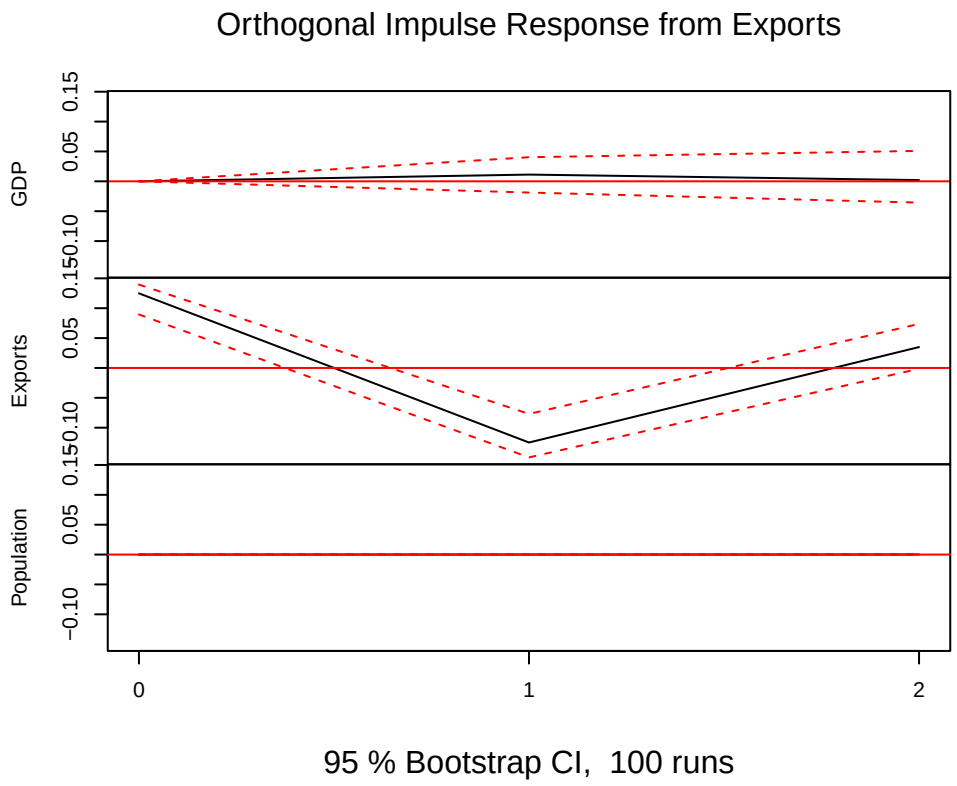
```
##
##  ARCH (multivariate)
##
## data:  Residuals of VAR object var_model
## Chi-squared = 174.58, df = 180, p-value = 0.5999
```

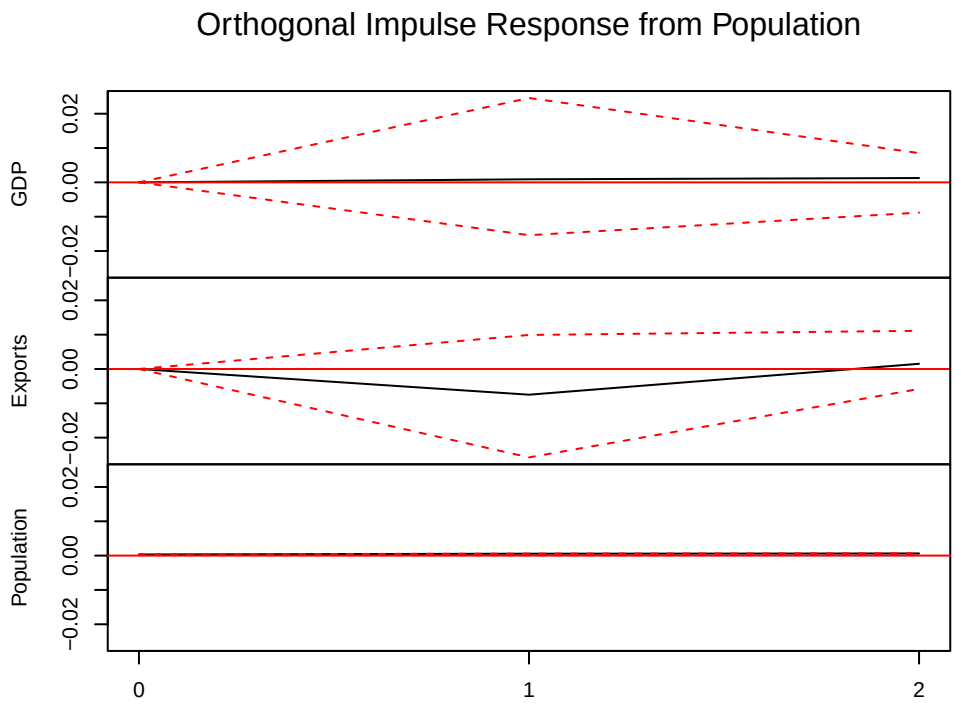
Greater than 0.05, therefore no residual autocorrelation.

Impulse Response Functions:

```
irf_all <- irf(var_model, impulse = NULL, response = NULL, n.ahead = 2
            , boot = TRUE)

plot(irf_all)
```




Orthogonal Impulse Response from Exports
GDP
Exports
Population
0.15
0.05
−0.10
0
1
2
95 % Bootstrap CI, 100 runs



Forecast:

```
forecast_var <- predict(var_model, n.ahead = 2, ci = 0.95)

forecast_var$fcst
```

```
## $GDP
##              fcst      lower     upper        CI
## [1,] -0.02431654 -0.3196656 0.2710326 0.2953491
## [2,]  0.02947463 -0.3354757 0.3944249 0.3649503
##
## $Exports
##               fcst      lower     upper        CI
## [1,] -0.008596438 -0.2777543 0.2605614 0.2691578
## [2,]  0.060579537 -0.3064308 0.4275899 0.3670104
##
## $Population
##                fcst        lower       upper           CI
## [1,]  1.861926e-03  0.001212605 0.002511246 0.0006493206
## [2,] -9.880374e-05 -0.001398033 0.001200426 0.0012992295
```

```
mean_back  <- exp(forecast_var$fcst$GDP[, "fcst"])
mean_back
```

```
## [1] 0.9759767 1.0299133
```

```r
lower_back <- exp(forecast_var$fcst$GDP[, "lower"])
lower_back
```

```
## [1] 0.7263919 0.7149979
```

```r
upper_back <- exp(forecast_var$fcst$GDP[, "upper"])
upper_back
```

```
## [1] 1.311318 1.483531
```

```r
mean_back_exports  <- exp(forecast_var$fcst$Exports[, "fcst"])
mean_back_exports
```

```
## [1] 0.9914404 1.0624521
```

```r
lower_back_exports <- exp(forecast_var$fcst$Exports[, "lower"])
lower_back_exports
```

```
## [1] 0.7574829 0.7360694
```

```r
upper_back_exports <- exp(forecast_var$fcst$Exports[, "upper"])
upper_back_exports
```

```
## [1] 1.297658 1.533557
```

```r
mean_back_pop  <- exp(forecast_var$fcst$Population[, "fcst"])
mean_back_pop
```

```
## [1] 1.0018637 0.9999012
```

```r
lower_back_pop <- exp(forecast_var$fcst$Population[, "lower"])
lower_back_pop
```

```
## [1] 1.0012133 0.9986029
```

```r
upper_back_pop <- exp(forecast_var$fcst$Population[, "upper"])
upper_back_pop
```

```
## [1] 1.002514 1.001201
```